

МИНИСТЕРСТВО ТРАНСПОРТА РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ УНИВЕРСИТЕТ ТРАНСПОРТА»
РУТ (МИИТ)
Институт управления и цифровых технологий
Кафедра «Цифровые технологии управления транспортными
процессами»

Курсовой проект

на тему:

«Студия мультипликации»

по дисциплине

Проектирование баз данных»

Выполнил:

студент группы УИС-311

Хохлова Елизавета Романовна

Проверил:

Доцент

Козьяков Павел Олегович
(ФИО)

Москва 2026

ЗАДАНИЕ НА КУРСОВОЙ ПРОЕКТ

по дисциплине

«Проектирование баз данных»

студента группы УИС-311

Хохловой Елизаветы Романовны

(ФИО студента полностью)

Наименование темы: «Студия мультипликации».

Необходимо изучить:

- 1) Принципы определения сущностей, связей между ними и атрибутов;
- 2) Нормальные формы: 1НФ, 2НФ, 3НФ, Бойса–Кодда (БКНФ).
- 3) Процедурное расширение языка SQL PostgreSQL;

Необходимо самостоятельно выполнить:

- 1) Отобразить заданную предметную область «Студия мультипликации», описанную вербально, в реляционную БД;
- 2) Организовать хранение исходных данных, в том числе проекты (мультфильмы), эпизоды, кадры, художники-аниматоры, этапы производства (скетч, раскраска, озвучка, композитинг);
- 3) Организовать операции ввода, редактирования, удаления и просмотра информации из БД посредством реализации пользовательского интерфейса в выбранной по желанию среде разработки (Python, C++, Java и др.);
- 4) Учесть при создании приложения и базы данных допущения: Каждый проект состоит из нескольких эпизодов, каждый эпизод – из кадров. Над каждым кадром работает один или несколько художников. Производственный процесс проходит через несколько этапов, каждый из которых требует проверки. Данные о прогрессе фиксируются ежедневно. Озвучка выполняется после завершения визуальной части.
- 5) Предусмотреть средствами приложения получение ответов на следующие вопросы:
 - а) Прогресс выполнения каждого проекта в процентах.
 - б) Список проектов, отстающих от графика более чем на 10 дней.
 - в) Количество кадров, созданных каждым художником за месяц.

- г) Среднее время выполнения этапа «раскраска» по проектам.
 - д) Список эпизодов, ожидающих проверки на этапе композитинга.
 - е) Распределение нагрузки по художникам.
 - ж) Прогнозируемая дата завершения проекта на основе текущих темпов.
- з) Отчёт о браке и доработках по этапам.;

Основные требования к прodelываемой работе:

- 1) Отношения создаваемой базы данных должны находиться, по крайней мере, в БКНФ2);
- 2) Разработку модели БД выполнить в нотации IDEF1X в среде ERwin;
- 3) Взаимодействие приложения с базой данных должно осуществляться через нативные запросы, использование ORM-фреймворков не допускается;
- 4) Заполнение идентификаторов в первичном ключе реализовать с использованием последовательностей;
- 5) Реализация обязательно должна содержать хотя бы один триггер, процедуру, функцию или анонимный блок с использованием процедурного расширения SQL;
- 6) После завершения озвучки эпизода пометать его как «готов к выпуску» и включать в график публикации;
- 7) Для оформления отчёта по курсовому проекту использовать ГОСТ 7.32-2017 «Отчет о научно-исследовательской работе. Структура и правила оформления».

Источники информации:

- 1 ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления = System of standards on information, librarianship and publishing. The research report. Structure and rules of presentation : издание официальное. – Москва : Стандартинформ, 2018. URL: <https://protect.gost.ru/document.aspx?control=7&baseC=6&page=0&month=3>

[&year=2022&search=7.32&RegNum=1&DocOnPageCount=15&id=218998](#)

(дата обращения: 24.10.2024) . - Текст: непосредственный.

- 2 Статьи, монографии, научно-исследовательские работы, официальные статистические данные по вопросам, затрагиваемым в курсовом проекте.

Материалы к защите:

- 1) Заполненный и подписанный лист индивидуального задания на курсовой проект.
- 2) Пояснительная записка (отчёт) по курсовому проекту.

Руководитель курсового проекта
Доцент кафедры ЦТУТП
П.О. Козьяков

(подпись) П.О. Козьяков

Студент группы УИС-311

(подпись) Е. Р. Хохлова

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1 Теоретические основы проектирования баз данных.....	8
1.1 Принципы определения сущностей, связей между ними и атрибутов .	8
1.2 Нормальные формы: 1НФ, 2НФ, 3НФ, Бойса–Кодда (БКНФ)	12
1.3 Процедурное расширение языка SQL PostgreSQL.....	17
2 Проектирование и реализация базы данных для предметной области «Студия мультипликации».....	20
2.1 Анализ предметной области и формирование требований к данным	20
2.2 Выделение сущностей, атрибутов и связей. Обоснование структуры таблиц	22
2.3 ER-модель базы данных и реализация схемы в PostgreSQL	27
3 Реализация приложения для работы с базой данных и демонстрация результатов.....	33
3.1 Среда выполнения и схема взаимодействия с базой данных	33
3.2 Пользовательский интерфейс приложения.....	34
3.3 Демонстрация выполнения аналитических запросов	42
ЗАКЛЮЧЕНИЕ	48
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ	49
Приложение А Скрипт SQL для создания структуры базы данных.....	51
Приложение Б Аналитические SQL запросы	55
Приложение В Функции, триггеры и процедуры	59
Приложение Г Полный код разработанного ПО	67

ВВЕДЕНИЕ

Производство анимационных проектов представляет собой сложный многоэтапный процесс, в рамках которого одновременно обрабатывается значительный объём взаимосвязанных данных: сведения о проектах (мультфильмах), их эпизодах и кадрах, этапах производственного цикла, задействованных сотрудниках, плановых и фактических сроках выполнения работ, результатах проверок качества и доработках. Каждый проект состоит из нескольких эпизодов, каждый эпизод – из набора кадров, над которыми могут работать один или несколько специалистов различных профилей, что существенно усложняет управление производственным процессом и контроль его состояния.

Производственный цикл анимации включает последовательность специализированных этапов, таких как разработка идеи и сценария, концепт-арт, анимация, раскраска, композитинг и озвучка. Выполнение этапов требует строгого соблюдения порядка, фиксации прогресса и обязательного контроля качества, поскольку результаты предыдущих работ напрямую влияют на возможность перехода к следующим стадиям. Нарушение сроков, неравномерная загрузка сотрудников или необходимость доработок могут привести к срыву графика проекта в целом, что делает актуальной задачу оперативного анализа хода производства.

Эффективное управление студией мультипликации невозможно без использования единой базы данных и информационной системы, обеспечивающих централизованное хранение информации и автоматизацию ключевых процессов. Такая система позволяет получать аналитические сведения, включая прогресс выполнения проектов в процентах, выявление проектов, отстающих от графика, оценку загрузки сотрудников, анализ средней длительности выполнения этапов, контроль эпизодов, ожидающих проверки, а также формирование отчётов по браку и доработкам. Автоматизация учёта повышает прозрачность производственного процесса,

снижает вероятность ошибок и способствует обоснованному принятию управленческих решений.

Цель курсового проекта – спроектировать и реализовать базу данных для предметной области «Студия мультипликации», обеспечивающую хранение исходных данных о проектах, эпизодах, кадрах, этапах производства и сотрудниках, а также получение ответов на предусмотренные заданием аналитические запросы, и разработать информационную систему на основе спроектированной базы данных.

1 Теоретические основы проектирования баз данных

Проектирование базы данных для любой предметной области начинается с опоры на фундаментальные принципы теории БД. Чтобы информационная система была устойчивой, удобной для сопровождения и корректно обрабатывала данные, необходимо правильно выделить объекты предметной области, определить их свойства и связи, а затем перенести эту структуру в реляционную модель. Не менее важным этапом является нормализация отношений, позволяющая снизить избыточность, устранить аномалии при добавлении, изменении и удалении данных и обеспечить целостность хранимой информации. Кроме того, для реализации правил предметной области на стороне СУБД применяется процедурное расширение SQL (в PostgreSQL – PL/pgSQL), с помощью которого создаются функции, процедуры и триггеры, автоматизирующие бизнес-логику.

Рассмотрение перечисленных теоретических положений формирует основу для дальнейшего проектирования структуры базы данных, выбора подхода к нормализации и реализации механизмов автоматизации в рамках разрабатываемой системы.

1.1 Принципы определения сущностей, связей между ними и атрибутов

Проектирование базы данных для предметной области «Студия мультипликации» начинается с анализа объектов, участвующих в производственном процессе, и определения перечня данных, подлежащих хранению в информационной системе. В реляционной теории такие объекты называются сущностями: это классы однотипных объектов с общими свойствами и правилами существования в рамках предметной области. Для рассматриваемой базы данных в качестве сущностей выступают, например, Project (проект/мультфильм), Episode (эпизод), Frame (кадр), Stage (этап производства), Employees (сотрудник), справочники StageType, StageStatus, Specialization, а также операционные сущности DailyProgress, StageCheck,

Rework, PublicationSchedule и другие объекты, фиксирующие прогресс, проверки качества, доработки и выпуск эпизодов [10–11].

Сущность описывает тип объекта, а конкретная строка таблицы является экземпляром сущности. Например, таблица "Project" задаёт структуру данных о проектах (название, даты, статус), а пример записи может выглядеть следующим образом: `id_project = 4`, `project_name = 'Гадкий я (учебный проект)'`, `start_date = '2025-02-01'`, `id_project_status =` (например, 'В производстве'). Аналогично, таблица "Frame" описывает кадры эпизода, а пример экземпляра: `id_frame = 15`, `id_episode = 3`, `frame_number_in_episode = 12`, `planned_due_date = '2025-02-05'`, `actual_done_date = NULL` (кадр ещё не завершён).

В реляционной модели сущности, как правило, отображаются в виде таблиц: каждая строка соответствует отдельному объекту, а столбцы отражают его свойства. При выделении сущностей целесообразно соблюдать ряд принципов. Во-первых, сущность должна быть значимой для процессов предметной области: например, Stage влияет на готовность кадра, эпизода и проекта, а также используется при формировании отчётов по срокам. Во-вторых, сущность должна описывать устойчивый объект, а не разовый вычисляемый показатель: процент прогресса проекта может вычисляться запросом или функцией, тогда как факты по этапам (статусы, даты, проверки) должны храниться в базе данных. В-третьих, если объект обладает самостоятельным назначением и собственным набором характеристик, его целесообразно выделять в отдельную таблицу: например, проверки качества StageCheck и доработки Rework оформляются отдельными сущностями, а не фиксируются как текстовые комментарии в записи этапа [2].

Атрибут представляет собой характеристику сущности и реализуется в таблице в виде столбца. Для таблицы "Stage" типичный состав атрибутов включает показатели планирования (`planned_end_date`, `start_datetime`), факта выполнения (`actual_end_datetime`), текущего состояния (`id_stage_status`), ответственности (`id_responsible_employee`), классификации (`stage_type_id`), а

также привязку к объекту производства (`id_project` / `id_episode` / `id_frame` в зависимости от уровня этапа).

При проектировании атрибутов необходимо обеспечивать атомарность значений. По этой причине сведения о сотрудниках оформляются отдельной таблицей "Employees" (фамилия, имя, контакты отдельно), а специализация выносится в справочник "Spesialization" и задаётся внешним ключом `id_spesialization`. Домены (допустимые значения) удобно задавать справочниками: например, `id_stage_status` может ссылаться только на записи "StageStatus", а результаты проверок – на "CheckResult". Такой подход снижает долю неконтролируемых текстовых значений и опирается на механизм ограничений целостности [5].

Для однозначной идентификации строк в каждой таблице используется первичный ключ. В схеме применяется подход с искусственными ключами BIGINT IDENTITY, что обеспечивает уникальные идентификаторы и упрощает построение связей между таблицами. В качестве примеров простых ключей выступают `Project.id_project`, `Episode.id_episode`, `Stage.id_stage`. В связующих таблицах может применяться составной ключ, где уникальность определяется сочетанием внешних ключей. Например, при наличии таблицы назначения сотрудников на кадры (FrameEmployee) естественная уникальность задаётся комбинацией `id_frame` + `id_employee`, исключающей повторное назначение одного сотрудника на один и тот же кадр [5].

После определения сущностей и ключей задаются связи между сущностями. В рассматриваемой предметной области типичны следующие связи: Project (1) – (M) Episode, поскольку одному проекту соответствует множество эпизодов, а эпизод принадлежит ровно одному проекту (в Episode хранится `id_project`); Episode (1) – (M) Frame, поскольку одному эпизоду соответствует множество кадров, а кадр принадлежит одному эпизоду (в Frame хранится `id_episode`); StageType (1) – (M) Stage, поскольку один тип этапа встречается во множестве этапов (в Stage хранится `stage_type_id`); Employees (M) – (M) Frame через связующую таблицу (FrameEmployee),

поскольку сотрудник может работать над несколькими кадрами, а выполнение одного кадра может выполняться несколькими специалистами [5].

Связь 1:М в реляционной схеме реализуется внешним ключом в дочерней таблице, а связь М:М – через ассоциативную таблицу, содержащую минимум два внешних ключа [5–6]. Для предметной области студии мультипликации это особенно важно, поскольку распределение работ по кадрам предполагает участие нескольких специалистов, что естественным образом формирует отношения многие-ко-многим [10–11].

Помимо кратности учитывается обязательность связей. Например, Episode не может существовать без Project, поскольку наличие `id_project` является обязательным. Для Stage некоторые атрибуты могут оставаться неопределёнными до наступления соответствующего события жизненного цикла: `actual_end_datetime` остаётся NULL до завершения этапа, а `id_responsible_employee` может быть NULL до назначения исполнителя. Корректность ссылок обеспечивается ограничениями ссылочной целостности и внешними ключами, предотвращающими появление некорректных ссылок и обеспечивающими непротиворечивость данных.

Отдельный класс требований предметной области связан с правилами выполнения работ, которые не всегда выражаются только внешними ключами. Например, правило выполнения озвучки после завершения визуальной части может обеспечиваться триггерами и функциями контроля порядка этапов и результатов проверок качества, реализованными средствами процедурного языка в СУБД. В таком подходе ER-схема фиксирует структуру и связи данных, а триггеры и процедуры задают порядок и условия изменения состояния объектов [3–4].

Для формализованного описания структуры базы данных применяется ER-модель; результаты проектирования и принятые решения фиксируются в отчёте, оформляемом по требованиям к структуре и правилам представления [1]. Использование ER-модели поддерживает согласованный переход от

концептуального описания к физической реализации в PostgreSQL и упрощает проверку соответствия сущностей и связей требованиям предметной области.

1.2 Нормальные формы: 1НФ, 2НФ, 3НФ, Бойса–Кодда (БКНФ)

Нормализация базы данных рассматривается как последовательное приведение отношений к таким формам, при которых уменьшается избыточность, устраняются аномалии вставки, обновления и удаления, а структура данных становится устойчивой к изменениям в процессе эксплуатации. В методической части по нормальным формам акцент делается на функциональных зависимостях, транзитивности и минимальности ключей; данный подход применим к модели производственного конвейера студии мультипликации. Практический эффект нормализации проявляется в том, что сведения о сотрудниках, статусах, типах этапов, результатах проверок и событиях жизненного цикла этапов не дублируются в рабочих таблицах, а задаются через справочники и связи; это повышает согласованность данных при смене статусов, появлении доработок и фиксации проверок качества [2, 5–6].

Первая нормальная форма (1НФ) (таблицы 1-3) требует атомарности значений и отсутствия повторяющихся групп: каждая ячейка хранит одно значение, а не список или набор. Атомарность достигается, например, разделением сведений о сотруднике на отдельные атрибуты `surname`, `pame`, `patronymic`, а не хранением объединённого поля «ФИО». Типичный случай нарушения 1НФ связан с попыткой хранить перечень исполнителей кадра в одном атрибуте кадра; корректным решением является использование связующей таблицы `FrameEmployee`, где каждая строка фиксирует одну пару `id_employee–id_frame` и атрибуты назначения [2].

Таблица 1 – Пример нарушения 1НФ: список сотрудников в одном поле кадра

id_frame	id_episode	frame_number_in_episode	planned_due_date	employees_list
15	3	12	2025-02-05	2:Иванов П.; 5:Смирнова А.

Таблица 2 – Пример соблюдения 1НФ: кадр и назначения сотрудников разделены

id_frame	id_episode	frame_number_in_episode	planned_due_date	actual_done_date
15	3	12	2025-02-05	NULL

Таблица 3 – Пример соблюдения 1НФ: назначения сотрудников на кадр в таблице FrameEmployee

id_frame	id_employee	role_in_frame	assigned_date
15	2	Animator	2025-02-01
15	5	Compositor	2025-02-02
15	9	Lighting artist	2025-02-02

Вторая нормальная форма (2НФ) (таблицы 4-6) требует полной функциональной зависимости: каждый неключевой атрибут должен зависеть от всего первичного ключа, а не от его части. Требование особенно важно для таблиц со составным ключом, используемых при реализации связей «многие-ко-многим». Для таблицы FrameEmployee (ключ id_employee + id_frame) допустимы атрибуты, описывающие факт назначения на конкретный кадр (например, role_in_frame, assigned_date). Хранение в FrameEmployee полей employee_email или employee_surname привело бы к частичной зависимости от id_employee и к дублированию данных при назначениях на разные кадры [2].

Таблица 4 – Пример нарушения 2НФ: свойства сотрудника повторяются в таблице назначений

id_frame	id_employee	employee_email	employee_surname	role_in_frame
15	2	p.ivanov@studio.local	Иванов	Animator
16	2	p.ivanov@studio.local	Иванов	Animator

Таблица 5 – Пример соблюдения 2НФ: свойства сотрудника хранятся в таблице Employees

id_employee	surname	name	email	id_specialization
2	Иванов	Павел	p.ivanov@studio.local	1

Таблица 6 – Пример соблюдения 2НФ: таблица FrameEmployee содержит только атрибуты назначения

id_frame	id_employee	role_in_frame
15	2	Animator
16	2	Animator

Третья нормальная форма (3НФ) (таблицы 7-9) направлена на устранение транзитивных зависимостей между неключевыми атрибутами: неключевые атрибуты должны зависеть только от ключа. В производственной схеме это достигается вынесением повторяющихся описательных значений в справочники и хранением в рабочих таблицах только ссылок. Например, таблица Stage хранит id_stage_status и stage_type_id, а названия статусов и типов этапов хранятся в StageStatus и StageType. Такое решение предотвращает необходимость массовых обновлений при изменении текста статуса и устраняет зависимость вида id_stage - id_stage_status - stage_status_name [2, 5–6].

Таблица 7 – Пример нарушения 3НФ: дублирование названия статуса в таблице Stage

id_stage	id_frame	stage_type_id	id_stage_status	stage_status_name	planned_end_date
701	15	4	2	В работе	2025-02-05
702	16	4	2	В работе	2025-02-06

Таблица 8 – Пример соблюдения 3НФ: таблица Stage хранит только ссылку id_stage_status

id_stage	id_frame	stage_type_id	id_stage_status	planned_end_date	actual_end_datetime
701	15	4	2	2025-02-05	NULL
702	16	4	2	2025-02-06	NULL

Таблица 9 – Пример соблюдения 3НФ: справочник StageStatus с названиями статусов

id_stage_status	stage_status_name
1	Не начат
2	В работе
3	На проверке
4	Требуется доработки
5	Завершён

Нормальная форма Бойса–Кодда (БКНФ) (таблицы 10-12) усиливает требования 3НФ: любой детерминант функциональной зависимости должен быть потенциальным ключом. На практике нарушение БКНФ часто связано со смешением фактов разных сущностей в одной таблице. Для таблицы проверок StageCheck корректным является хранение результата проверок ссылкой id_check_result на справочник CheckResult; в этом случае текстовое представление результата не дублируется в каждой записи проверки, а задаётся одним значением в справочнике [2].

Таблица 10 – Пример нарушения БКНФ: код результата определяет текст результата внутри StageCheck

id_ stage_ check	id_ stage	id_ checker_ employee	result_ code	result_ name
9001	701	12	A	Принято
9002	702	12	A	Принято

Таблица 11 – Пример соблюдения БКНФ: таблица StageCheck хранит ссылку на справочник результатов

id_ stage_ check	id_ stage	id_ checker_ employee	id_ check_ result	check_ datetime
9001	701	12	1	2025-02-05 18:20:00
9002	702	12	1	2025-02-06 11:10:00

Таблица 12 – Пример соблюдения БКНФ: справочник CheckResult с текстовыми наименованиями

id_check_result	check_result_name
1	Принято
2	Отклонено

Нормализация дополняется ограничениями предметной области (таблица 13), которые не относятся к нормальным формам, но поддерживают корректность данных. Примером является ограничение stage_scope_chk в таблице Stage (таблица 13), требующее заполнения ровно одного из атрибутов id_project / id_episode / id_frame (ровно один уровень привязки этапа). Это исключает появление двусмысленных записей этапов и повышает качество данных [5–6].

Таблица 13 – Пример контроля области этапа: корректная и некорректная привязка в таблице Stage

id_ stage	id_ project	id_ episode	id_ frame	comment
1001	4	NULL	NULL	корректно: этап проекта
1002	4	NULL	15	некорректно: заполнены два уровня
1003	NULL	3	NULL	корректно: этап эпизода
1004	NULL	NULL	15	корректно: этап кадра

Нормализация до 1НФ–3НФ и БКНФ обеспечивает структурное разделение справочных и операционных данных, устраняет дублирование (например, статусов и названий типов этапов), корректно оформляет связи многие-ко-многим (назначения сотрудников на кадры), а также снижает риск аномалий при изменениях в ходе производственного процесса. Теоретические определения нормальных форм и практические примеры декомпозиции согласуются с обзорными источниками по нормализации [2], а корректность ссылок и непротиворечивость данных обеспечиваются средствами ограничений целостности PostgreSQL [5–6] и возможностями процедурного языка для реализации бизнес-правил [3–4].

1.3 Процедурное расширение языка SQL PostgreSQL

Стандартный SQL в первую очередь предназначен для описания структуры данных и выполнения базовых операций с ними: выборки, добавления, изменения и удаления записей. Однако в прикладных информационных системах база данных обычно выполняет не только роль хранилища, но и роль контролирующего механизма, который обеспечивает соблюдение правил предметной области. На уровне СУБД требуется выполнять автоматические проверки корректности данных, поддерживать целостность при изменениях, фиксировать ключевые события жизненного цикла объектов, рассчитывать производные показатели и обеспечивать согласованность связанных сущностей. В PostgreSQL эти задачи решаются с помощью процедурного расширения PL/pgSQL, позволяющего писать исполняемый код, работающий непосредственно внутри базы данных.

PL/pgSQL расширяет SQL возможностью использовать переменные, выполнять ветвления (IF), организовывать циклы (LOOP, FOR), обрабатывать исключения и сохранять результаты запросов во временные переменные. Благодаря этому в PostgreSQL реализуются прикладные механизмы управления данными, которые сложно или неудобно выразить чистым SQL. На практике используются три основных типа объектов: функции, процедуры

и триггеры – каждый из них закрывает свой класс задач, и в курсовом проекте все три типа применены.

Функции (FUNCTION) возвращают скалярное значение или набор строк и могут вызываться в составе SQL-запросов. Такой формат удобен, когда требуется регулярно вычислять показатели или формировать отчётные выборки на основе накопленных данных. В проекте реализована функция `fn_project_progress_percent`, выполняющая расчёт процента готовности проекта по этапам производства с учётом иерархии данных (этапы проекта, этапы эпизодов и этапы кадров). Результат функции используется в аналитических запросах и в интерфейсных представлениях прогресса, позволяя получать показатель готовности без хранения вычисляемого значения в таблицах.

Процедуры (PROCEDURE) предназначены для выполнения последовательности действий и вызываются командой `CALL`. Процедуры подходят для операций сценарного характера, когда требуется создать или изменить набор связанных записей и обеспечить единый алгоритм внесения данных. В проекте реализованы две процедуры. Процедура `sp_reschedule_publication` переносит дату публикации эпизода: добавляет запись в расписание публикаций, фиксирует новое плановое значение и устанавливает статус публикации (например, «Отложено»), что поддерживает управляемое планирование выхода эпизодов и сохраняет историю переносов. Процедура `sp_create_default_stages_for_frame` формирует типовой набор этапов производства для кадра: создаёт записи `Stage` на основе справочника `StageType` с учётом порядка выполнения, что устраняет ручное создание однотипных этапов и обеспечивает единообразие конвейера по всем кадрам.

Триггеры (TRIGGER) запускают заданный код автоматически при событиях `INSERT`, `UPDATE` или `DELETE` и делают бизнес-правила обязательными на уровне базы данных, независимо от того, каким способом изменяются данные. В проекте реализовано пять триггеров, ориентированных на контроль конвейера и целостность данных:

1) триггер контроля порядка этапов и проверок качества (реализован через функцию `trg_stage_enforce_order_and_checks()`). Назначение – запрет изменения состояния этапа на «завершён» при нарушении последовательности выполнения и при отсутствии успешной проверки качества по требуемым предыдущим этапам;

2) триггер автоматической фиксации факта завершения этапа. Назначение – при переводе этапа в статус «завершён/принят» автоматически заполнять `actual_end_datetime`, если значение не было задано, обеспечивая корректную статистику по длительности и подтверждённый факт окончания работ;

3) триггер ведения истории статусов этапа. Назначение – при изменении `id_stage_status` добавлять запись в журнал (например, `StageStatusHistory`) с отметкой времени и новым статусом, чтобы сохранялась хронология переходов, включая возвраты на доработку и этапы проверки;

4) триггер «озвучка – готовность эпизода». Назначение – при успешном завершении этапа озвучки автоматически устанавливать признаки готовности эпизода к выпуску (`ready_for_release_flag`, `ready_for_release_date`) и при необходимости синхронизировать статус эпизода, связывая завершение ключевого производственного этапа с управленческим состоянием;

5) триггер поддержания производных показателей при изменениях этапов. Назначение – при вставке или обновлении этапов инициировать перерасчёт/синхронизацию показателей, используемых в отчётности и интерфейсе, на основе функции прогресса и актуальных статусов этапов, чтобы отображаемые состояния и признаки соответствовали фактическим данным.

2 Проектирование и реализация базы данных для предметной области «Студия мультипликации»

2.1 Анализ предметной области и формирование требований к данным

Предметная область «Студия мультипликации» удобнее всего описывается через метафору конвейера: на вход поступает проект как «заказ на выпуск», а на выходе должен появиться эпизод, прошедший финальную приёмку и готовый к публикации. Между входом и выходом находится цепочка взаимосвязанных операций, где результат каждого шага становится исходным материалом для следующего. В отличие от линейного производства, конвейер анимации многослоен: проект состоит из эпизодов, эпизоды – из кадров, а каждый кадр проходит набор этапов работ, требующих разных специалистов и разных правил контроля.

Конвейер начинается с формирования структуры производства. Проект задаёт рамки: сроки, статус, плановую дату завершения и общие требования. На следующем уровне эпизоды выступают как «партии продукции», которые должны быть готовы к определённым датам. Кадры становятся минимальными единицами производства: именно на них удобно назначать исполнителей, фиксировать прогресс и накапливать статистику. Один кадр может проходить через несколько специалистов, поэтому для учёта факта участия используется связь многие-ко-многим: сотрудник работает над несколькими кадрами, кадр создаётся усилиями нескольких сотрудников разных профилей.

Ключевая особенность конвейера – зависимость этапов и невозможность «перепрыгнуть» технологию. В данных это выражается правилом: переход статуса этапа вперёд допускается только при выполнении условий предыдущих стадий. Для части работ этого недостаточно – требуется подтверждение качества. Проверка качества выступает как контрольный пост на линии: этап может быть выполнен, но пока не пройден контроль, выпуск дальше блокируется. Если проверка не пройдена, конвейер не останавливается

полностью, а возвращает задачу на доработку: появляется запись о доработке или браке, фиксируется причина, а этап переводится в соответствующий статус, после чего запускается повторный цикл исправления и повторной проверки.

Управление конвейером невозможно без двух параллельных «измерительных линеек» – плановой и фактической. Плановые даты отражают производственный график, фактические – реальную скорость выполнения. На уровне этапов фиксируются даты старта и завершения, что позволяет вычислять длительность работ и находить узкие места. На уровне кадров и эпизодов прогресс собирается агрегированно: множество завершённых этапов и успешно пройденных проверок превращаются в показатель готовности, который нужен для управленческих решений и отчётности. Важное требование – хранить факты (статусы, даты, результаты проверок), а производные показатели рассчитывать на их основе, чтобы избежать расхождений между «как посчитали» и «что реально произошло».

Финальная приёмка эпизода выполняет роль последнего контрольного узла на конвейере. До неё эпизод может быть «почти готов», но критерий готовности должен быть однозначным: готовность наступает не после отдельного этапа вроде озвучки, а после успешного прохождения финальной приёмки, которая подтверждает, что комплект работ выполнен, замечания закрыты, а качество соответствует требованиям. В информационной системе это отражается как бизнес-правило: при успешном завершении этапа финальной приёмки автоматически устанавливается признак готовности эпизода и фиксируется дата готовности. Такой подход делает статус «готов к выпуску» не субъективной отметкой, а результатом контролируемого события.

На основе описанной логики конвейера формируются требования к данным. Необходимы сущности, отражающие «изделие» (проект, эпизод, кадр), «операции» (этапы производства), «ресурс» (сотрудники и специализации) и «контроль» (проверки качества, доработки, брак, история

изменений статусов, ежедневный прогресс). Требования к целостности обеспечивают, чтобы конвейер не «ломался» из-за некорректных записей: ссылки между объектами должны быть валидны, завершение этапа должно сопровождаться фиксацией факта завершения, порядок стадий должен соблюдаться, а проверки качества – быть обязательными там, где это предусмотрено технологией. Требования к аналитике превращают накопленные данные в управленческий инструмент: прогресс по проектам, отставания от графика, нагрузка сотрудников, очереди на проверку, статистика по длительности этапов и отчётность по браку и доработкам.

2.2 Выделение сущностей, атрибутов и связей. Обоснование структуры таблиц

По итогам анализа предметной области «Студия мультипликации» сформирована структура базы данных, предназначенная для хранения и обработки информации о проектах (мультфильмах), их эпизодах и кадрах, этапах производственного процесса, задействованных сотрудниках, результатах проверок качества и доработках. Разработанная модель отражает особенности анимационного производства: многоуровневую иерархию объектов (проект, эпизод, кадр), зависимость этапов друг от друга, необходимость контроля качества и фиксации фактических результатов выполнения работ на разных уровнях.

Проектирование выполнено с опорой на принципы нормализации. Повторяющиеся и категориальные значения вынесены в отдельные справочные таблицы: статусы, типы этапов, результаты проверок и специализации сотрудников представлены как самостоятельные сущности. Связи между объектами реализованы с использованием внешних ключей, а отношения многие-ко-многим оформлены через ассоциативные таблицы. Такое решение уменьшает избыточность, повышает согласованность данных и упрощает выполнение аналитических запросов, предусмотренных заданием.

Базовой сущностью модели является Project, представляющая анимационный продукт, находящийся в производстве. Проект объединяет

совокупность эпизодов, характеризуется плановыми и фактическими сроками выполнения, текущим статусом и используется как основной объект для расчёта прогресса и анализа соблюдения графика. Состояния проекта задаются через справочник ProjectStatus, что исключает использование произвольных строковых значений и обеспечивает единый перечень допустимых статусов.

Сущность Episode описывает логически завершённую часть проекта и связана с Project отношением один-ко-многим. Для эпизода хранятся наименование, порядковый номер в проекте, плановая и фактическая даты выпуска, а также признак готовности. Состояние эпизода задаётся через справочник EpisodeStatus. Дополнительно выделена сущность PublicationSchedule, предназначенная для хранения плановых и фактических дат публикации и статусов публикации, что позволяет учитывать переносы и хранить историю изменений расписания выхода эпизодов.

Сущность Frame предназначена для детального описания производственного процесса на уровне кадров и связана с Episode отношением один-ко-многим. Каждый эпизод включает набор кадров, для которых фиксируются плановые и фактические сроки выполнения. Такое разделение позволяет учитывать прогресс работ на уровне отдельных кадров и формировать детализированные аналитические отчёты.

Ключевым элементом модели, отражающим выполнение работ, является сущность Stage. Этап описывает выполнение конкретного вида работ и может относиться к проекту, эпизоду или кадру в зависимости от уровня этапа. Тип этапа задаётся через справочник StageType, в котором определены уровень применения, порядок выполнения, принадлежность к визуальной части и признак обязательности проверки качества. Текущий статус этапа хранится в справочнике StageStatus. Данная структура формализует последовательность выполнения работ и поддерживает контроль корректного прохождения этапов.

Назначение ответственных исполнителей реализовано через сущность Employees и внешний ключ в таблице Stage. Структура поддерживает распределение задач и учёт ответственности, а также позволяет анализировать загрузку персонала по этапам, кадрам и эпизодам. Специализация сотрудников нормализована через справочник Spesialization, а признак активности используется для учёта доступности сотрудников в производственном процессе.

Для фиксации ежедневного хода выполнения работ используется сущность DailyProgress, позволяющая хранить значение прогресса этапа на конкретную дату. Эти данные применяются для анализа темпов выполнения работ и прогнозирования сроков завершения. Результаты проверок качества хранятся в таблице StageCheck, где фиксируются этап, дата проверки, итог проверки и комментарии. Возможные итоги нормализованы через справочник CheckResult, что обеспечивает единые допустимые значения.

Для учёта повторных работ выделена сущность Rework, связанная с этапами производства. Таблица предназначена для фиксации доработок и случаев брака, а также последующего анализа причин возвратов и формирования отчётности по качеству в разрезе этапов и проектов.

Для автоматизации бизнес-логики в базе данных реализованы функции, триггеры и процедуры на языке PL/pgSQL. Реализованы механизмы расчёта прогресса проекта, контроля последовательности прохождения этапов и условий перехода между статусами, автоматического перевода эпизода в состояние готовности после успешного выполнения финальной приёмки, а также логирования смены статусов этапов.

CheckResult – предназначена для хранения вариантов результатов проверки:

- принято;
- отклонено;
- требует доработки.

EpisodeStatus – статусы эпизодов:

- в работе;
- готов к выпуску;
- опубликован.

ProjectStatus – статусы проектов:

- запланирован;
- в производстве;
- завершён.

PublicationStatus – статусы публикации.

- запланировано;
- опубликовано;
- отложено.

Spesialization – специализации сотрудников:

- режиссёр;
- сценарист;
- художник-постановщик;
- концепт-художник;
- художник персонажей;
- художник по окружению (фоны);
- 3D-моделлер;
- аниматор;
- актёр озвучивания;
- звукорежиссёр;
- компоузер (композитинг);
- vfx-специалист;
- технический художник;
- специалист по рендерингу.

StageStatus – статусы этапов производства:

- не начат;
- в работе;
- на проверке;

- принят;
- требует доработки;
- завершён.

StageType – типы этапов производственного процесса на уровне проекта, эпизода и кадра:

- разработка идеи и сценария;
- концепт-арт и дизайн;
- создание раскадровки;
- аниматик / превизуализация;
- озвучивание;
- звук и музыка;
- финальная приёмка эпизода;
- моделирование;
- анимация;
- композитинг;
- визуальные эффекты (vfx);
- техническая подготовка;
- рендеринг.

Атрибут stage_level определяет уровень, к которому относится этап, и показывает, применяется ли он ко всему проекту, отдельному эпизоду или конкретному кадру.

Атрибут is_visual_stage указывает, относится ли этап к визуальной части производства и используется для разделения визуальных и невизуальных работ.

Атрибут requires_quality_check показывает, требуется ли обязательная проверка качества после завершения этапа.

Атрибут execution_order задаёт порядковый номер выполнения этапа внутри соответствующего уровня и используется для определения корректной последовательности производственного процесса.

Employees – сотрудники студии.

Атрибут `is_active` в таблице сотрудников студии – это признак активности сотрудника (сотрудник может быть, например, в отпуске или на больничном):

- `true` – сотрудник в данный момент работает в студии и может быть назначен на этапы;

- `false` – сотрудник больше не участвует в производственном процессе (уволился, в отпуске, на длительном больничном и т.п.).

`Project` – информация о проектах.

`Episode` – информация об эпизодах.

`Frame` – информация о кадрах.

`Stage` – для хранения информации о выполнении конкретных этапов производственного процесса.

`PublicationSchedule` – хранение информации о планировании и фактическом выполнении публикации эпизодов.

`DailyProgress` – предназначена для фиксации ежедневного хода выполнения работ.

`StageCheck` – информация о проверках этапов производственного процесса.

`Rework` – для учёта доработок, возникающих по результатам проверок.

2.3 ER-модель базы данных и реализация схемы в PostgreSQL

В результате анализа предметной области и преобразования вербального описания производственного процесса в формализованную структуру была разработана реляционная модель базы данных, включающая 18 таблиц: 9 основных таблиц, предназначенных для хранения информации о проектах, эпизодах, кадрах, этапах производства, сотрудниках и ходе выполнения работ, и 9 справочных таблиц, обеспечивающих нормализацию и целостность данных.

Все таблицы приведены к нормальной форме Бойса–Кодда (БКНФ), что позволяет устранить избыточность данных и предотвратить аномалии вставки,

обновления и удаления. Каждая таблица имеет первичный ключ, формируемый автоматически с использованием последовательностей PostgreSQL, и связана с другими таблицами посредством внешних ключей, обеспечивающих ссылочную целостность данных. Структура базы данных представлена в виде ER-модели, выполненной в нотации IDEF1X (рисунок 2.3.1).

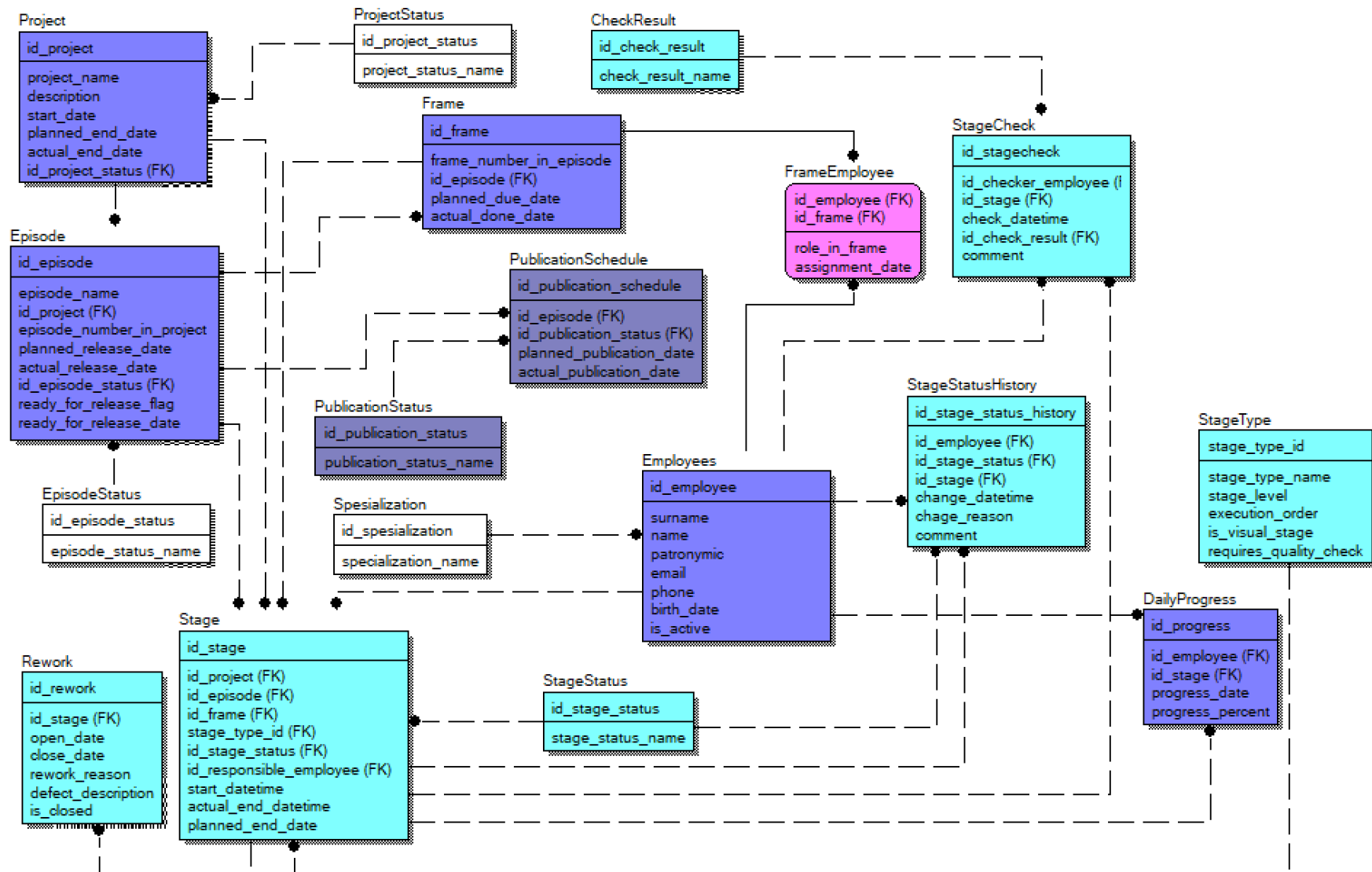


Рисунок 2.3.1 - ER-модель базы данных в нотации IDEF1X

При проектировании ER-модели для предметной области «Студия мультипликации» были приняты решения, направленные на снижение избыточности данных и обеспечение их логической целостности. Текстовые и категориальные характеристики объектов, такие как статусы проектов, эпизодов и этапов производства, типы этапов, результаты проверок качества, статусы публикации и специализации сотрудников, вынесены в отдельные справочные таблицы. В основных сущностях используются ссылки на соответствующие справочники, что обеспечивает единообразие значений, упрощает сопровождение базы данных и предотвращает ввод некорректных данных.

Базовой сущностью модели является Project (Проект), представляющий анимационный продукт, находящийся в производстве. Проект объединяет набор эпизодов, характеризуется плановыми и фактическими сроками выполнения и текущим статусом, задаваемым через справочник ProjectStatus. Проект выступает верхним уровнем иерархии и используется как основной объект для анализа прогресса и соблюдения графика производства.

Сущность Episode (Эпизод) связана с проектом отношением «один-ко-многим»: один проект может включать несколько эпизодов. Для эпизодов фиксируются наименование, порядковый номер в проекте, плановая и фактическая даты выпуска, а также признак готовности к публикации. Состояние эпизода нормализовано и задаётся через справочник EpisodeStatus. Планирование и фактическое выполнение публикации эпизодов реализовано через отдельную сущность PublicationSchedule, связанную со справочником PublicationStatus, что позволяет учитывать переносы дат и формировать историю публикаций.

Для детального учёта производственного процесса используется сущность Frame (Кадр), связанная с эпизодом отношением «один-ко-многим». Каждый эпизод состоит из набора кадров, для которых хранятся плановые и фактические сроки выполнения. Назначение сотрудников на выполнение работ по кадрам реализовано через ассоциативную таблицу FrameEmployee,

поскольку один кадр может разрабатываться несколькими специалистами, а один сотрудник может участвовать в создании множества кадров.

Ключевым элементом модели является сущность Stage (Этап производства), предназначенная для хранения информации о выполнении конкретных этапов производственного процесса. Этап может относиться к проекту, эпизоду или отдельному кадру в зависимости от уровня этапа. Тип этапа задаётся через справочник StageType, в котором определены уровень применения этапа, его порядковый номер в технологической последовательности, принадлежность к визуальной части производства и необходимость обязательной проверки качества. Текущее состояние этапа задаётся через справочник StageStatus.

Для фиксации ежедневного хода выполнения работ используется таблица DailyProgress, позволяющая хранить процент готовности этапа на конкретную дату. Эти данные используются для расчёта прогресса проектов и анализа динамики выполнения работ. История смены статусов этапов фиксируется в таблице StageStatusHistory, что обеспечивает трассируемость изменений и контроль действий ответственных сотрудников.

Контроль качества этапов реализован через сущность StageCheck, в которой хранятся сведения о проведённых проверках, дате проверки и её результате. Возможные результаты проверки нормализованы и хранятся в справочнике CheckResult. В случае выявления несоответствий используется сущность Rework, предназначенная для учёта доработок и повторных работ, связанных с конкретными этапами производства.

Сотрудники студии представлены сущностью Employees, содержащей персональные данные и признак активности сотрудника. Профессиональная принадлежность сотрудников нормализована через справочник Specialization, что позволяет корректно учитывать распределение ролей и специализаций в производственном процессе.

Физическая реализация базы данных выполнена в СУБД PostgreSQL. Для каждой таблицы определён первичный ключ, автоматически заполняемый

с использованием последовательностей. Связи между таблицами реализованы с помощью внешних ключей, обеспечивающих ссылочную целостность данных: проекты связаны со статусами проектов; эпизоды – с проектами и статусами эпизодов; кадры – с эпизодами; этапы производства – с типами этапов, статусами и ответственными сотрудниками; проверки и доработки – с этапами производства; публикации – с эпизодами и статусами публикации. Для предотвращения логически некорректного ввода данных предусмотрены ограничения обязательности ключевых атрибутов и бизнес-ограничения, реализованные средствами PL/pgSQL.

SQL-скрипт создания таблиц, первичных и внешних ключей, а также ограничений целостности приведён в приложении А и используется для воспроизводимого развёртывания базы данных и проверки корректности разработанной модели.

3 Реализация приложения для работы с базой данных и демонстрация результатов

3.1 Среда выполнения и схема взаимодействия с базой данных

Для демонстрации работы спроектированной базы данных по предметной области «Студия мультипликации» использовалась локальная среда разработки. СУБД PostgreSQL развернута на персональном компьютере, а доступ к данным выполнялся через локальный веб-интерфейс, открываемый по адресу localhost. Развёртывание структуры базы данных выполнялось запуском SQL-скрипта создания таблиц, ограничений целостности, справочников и программных объектов (функций, процедур, триггеров). Такой способ установки обеспечивает воспроизводимость: при наличии PostgreSQL база разворачивается на другой машине без ручной сборки схемы и без «донастройки» через интерфейс администратора.

Взаимодействие с базой данных организовано по клиент-серверной схеме. Локальный веб-интерфейс выполняет роль клиентского слоя: формирует SQL-команды и отправляет их в PostgreSQL по стандартному протоколу подключения. Работа с данными выполняется без использования ORM: запросы формируются явно и выполняются напрямую, что позволяет контролировать точный текст SQL, проверять корректность условий выборки и демонстрировать работу ограничений и бизнес-логики на уровне СУБД. Отдельный акцент сделан на том, что все изменения состояния производства (статусы этапов, проверки качества, доработки, готовность эпизода) фиксируются именно в базе данных и затем отображаются в интерфейсе как результат выполненных операций, а не как «логика приложения».

Демонстрация включает две группы действий.

Первая группа – выполнение подготовленных аналитических запросов по предметной области, ориентированных на управление производством и контроль сроков. В демонстрации выполнялись запросы: расчёт прогресса выполнения проектов и эпизодов в процентах на основе статусов этапов; выявление проектов и эпизодов, отстающих от планового графика более чем

на заданный срок; подсчёт количества кадров, выполненных каждым сотрудником за период; расчёт среднего времени выполнения этапов по проектам и по типам работ; формирование перечня кадров и эпизодов, ожидающих проверки качества, включая этап финальной приёмки; анализ распределения нагрузки между сотрудниками по активным этапам; оценка прогнозируемой даты завершения проекта по текущим темпам; отчёт по браку и доработкам в разрезе этапов производства; выборки по расписанию публикаций и истории переносов дат выпуска эпизодов. Тексты запросов приводятся в приложении с SQL-листингом.

Вторая группа – операции добавления, редактирования и удаления записей в основных таблицах с проверкой ограничений целостности и автоматизированной логики СУБД. В демонстрации выполнялись команды INSERT/UPDATE/DELETE для сущностей Project, Episode, Frame, Stage, Employees, а также для операционных сущностей DailyProgress, StageCheck, Rework и PublicationSchedule. Отдельно проверялась работа процедур и триггеров: создание типового набора этапов для кадра процедурой `sp_create_default_stages_for_frame`; перенос даты публикации эпизода процедурой `sp_reschedule_publication`; контроль порядка прохождения этапов и условий завершения через триггеры (например, запрет завершения этапа без обязательной успешной проверки качества); автоматическая фиксация фактических дат завершения при смене статуса этапа; логирование истории смены статусов; перевод эпизода в состояние готовности после успешного выполнения финальной приёмки. Такие операции позволяют воспроизводимо показать, что правила предметной области соблюдаются на уровне базы данных независимо от того, откуда приходит изменение – из интерфейса или из прямого SQL.

3.2 Пользовательский интерфейс приложения

Пользовательский интерфейс веб-приложения «Minion Studio» предназначен для оперативного контроля производства анимационных проектов. Интерфейс реализован в виде набора взаимосвязанных экранов

(списков и карточек объектов), объединённых единым стилем и верхним навигационным меню. Переходы между сущностями повторяют иерархию данных в базе: проект, эпизод, кадр, этап.

Во всех разделах используется единая шапка с логотипом студии и пунктами навигации: «Проекты», «Сотрудники», «Публикации», «Отчёты». Экранные формы построены по принципу «список, карточка, редактирование»: в списках отображаются ключевые атрибуты и показатели, а операции создания и изменения выполняются через отдельные формы. Для удобства контроля применяются визуальные индикаторы статусов и прогресса (бейджи и прогресс-бары), что позволяет оценивать состояние объектов без перехода в детальные карточки.

Раздел «Проекты» используется для просмотра перечня анимационных проектов и их текущего состояния. В таблице отображаются название проекта, статус, даты начала и планового завершения, фактическая дата окончания (при наличии), а также два показателя прогресса: общий прогресс проекта и прогресс за текущий день. Из списка доступны переход в карточку проекта, редактирование и удаление записи, а также создание нового проекта. Пример экрана приведён на рисунке 3.2.1.

Проект	Статус	Старт	План	Факт	Прогресс (общий / сегодня)	Действия
учебный проект	в производстве	2026-01-20	2026-01-31	—	общий 12% сегодня 12%	Редакт. Удалить
Мультфильм «Гадкий я» 3	в производстве	2026-01-22	2026-01-23	2026-01-23	общий 35% сегодня 0%	Редакт. Удалить
Мультфильм «Гадкий я»	в производстве	2025-09-25	2026-01-18	2026-01-17	общий 2% сегодня 0%	Редакт. Удалить
Мультсериал «Удивительный цифровой цирк»	в производстве	2025-01-15	2025-10-01	—	общий 95% сегодня 0%	Редакт. Удалить
Мультсериал «Рик и Морти»	в производстве	2025-01-10	2025-12-31	—	общий 72% сегодня 11%	Редакт. Удалить

Рисунок 3.2.1 - Раздел «Проекты»: список проектов с отображением статусов и прогресса

Форма редактирования проекта предназначена для изменения основных атрибутов проекта: названия, статуса, дат и описания. Валидация вводимых данных выполняется на уровне формы и ограничений базы данных; изменения

сохраняются только при соблюдении ссылочной и доменной целостности. Пример формы показан на рисунке 3.2.2.

Рисунок 3.2.2 - Форма редактирования проекта

Карточка проекта объединяет «паспорт» проекта и производственные показатели. В верхней части выводятся ключевые атрибуты и индикатор прогресса (рисунок 3.2.3). Ниже размещается таблица этапов проектного уровня, где доступно изменение статуса, назначение ответственного сотрудника и просмотр прогресса по этапу. Далее приводится список эпизодов проекта с ключевыми датами и признаком готовности к релизу (рисунок 3.2.4). Такая компоновка позволяет контролировать проект одновременно по «верхнему» уровню (проектные этапы) и по деталям (эпизоды).

Этап	Статус	Ответственный	Прогресс	План	Действия
Разработка идеи и сценария	Завершён	Соколова Мария	100%	2025-01-25	Назначить
Концепт-арт и дизайн	Завершён	Крылов Дмитрий	100%	2025-01-25	Назначить

Рисунок 3.2.3 - Карточка проекта: параметры и общий прогресс

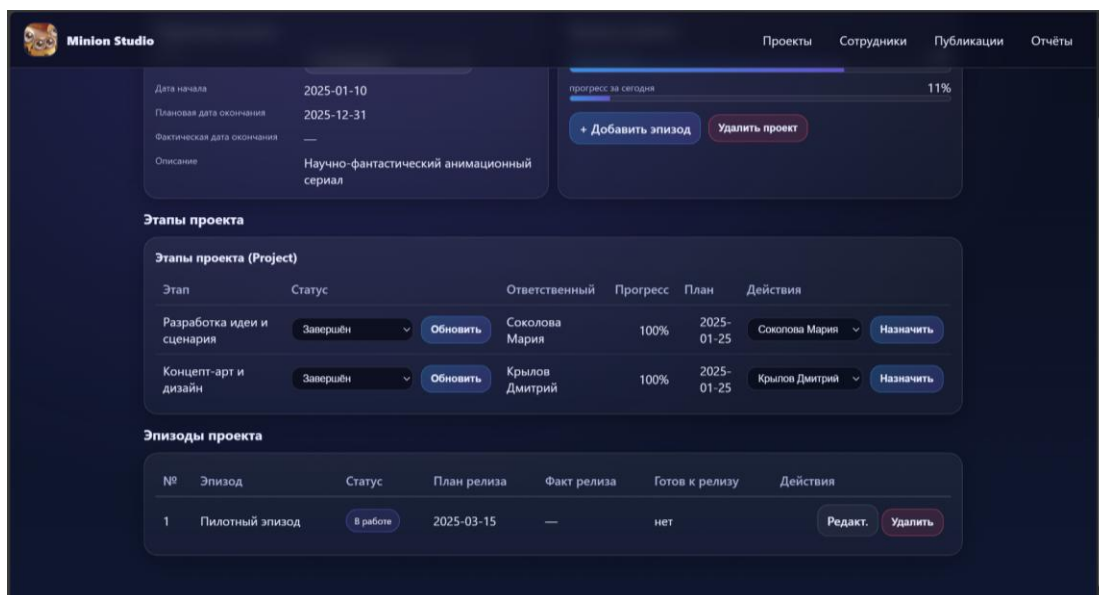


Рисунок 3.2.4 - Карточка проекта: этапы проекта и список эпизодов

Переход к эпизодам выполняется из карточки проекта. Экран редактирования эпизода используется для изменения названия, номера в проекте, плановой даты релиза и статуса, а также для установки признака готовности. Пример формы приведён на рисунке 3.2.5.

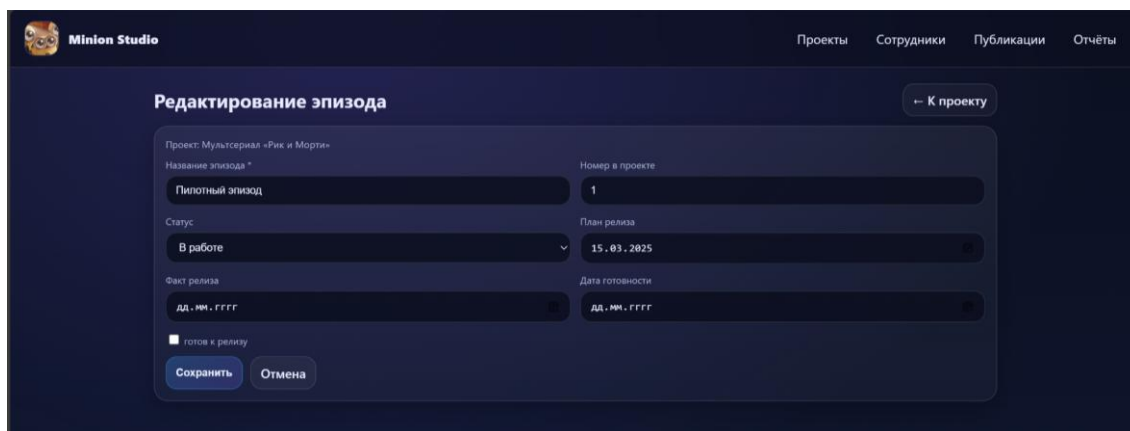


Рисунок 3.2.5 - Форма редактирования эпизода

Карточка эпизода содержит блок параметров эпизода и отдельный блок «График публикаций». В графике фиксируются статусы планирования публикации и даты (план/факт), что позволяет отражать переносы релиза и хранить историю изменений. Пример отображения графика приведён на рисунке 3.2.6.

Эпизод №1 — Пилотный эпизод → К проекту Редактировать

Проект: Мультсериал «Рик и Морти»
 Статус эпизода: В работе
 План релиза: 2025-03-15
 Факт релиза: —
 Готов к релизу: нет
 Дата готовности: —

+ Добавить кадр Удалить эпизод

График публикаций

ДД.ММ.ГГГГ Перенести публикацию

Статус	План	Факт
Запланировано	2025-03-15	—
Отложено	2025-03-25	—
Опубликовано	2025-03-25	2025-03-26
Отложено	2025-04-05	—

Рисунок 3.2.6 - Карточка эпизода: атрибуты эпизода и график публикаций

Ниже параметров располагаются этапы эпизода: для каждого этапа отображаются статус, ответственный сотрудник, текущий прогресс и плановая дата. На этом же экране размещён список кадров эпизода с плановыми и фактическими сроками сдачи, а также операциями редактирования. Пример размещения этапов и кадров приведён на рисунке 3.2.7.

Этапы эпизода

Этап	Статус	Действия	Ответственный	Прогресс	План	Действия
Создание раскадровки	Завершён	Обновить	Крылов Дмитрий	100%	2025-03-15	Крылов Дмитрий Назначить
Аниматик / превизуализация	Не начат	Обновить	Крылов Дмитрий	0%	2025-03-15	Крылов Дмитрий Назначить
Озвучивание	Не начат	Обновить	Морозов Артём	0%	2025-03-15	Морозов Артём Назначить
Звук и музыка	Не начат	Обновить	Кузнецов Дмитрий	0%	2025-03-15	Кузнецов Дмитрий Назначить
Финальная приёмка эпизода	Не начат	Обновить	Иванов Алексей	0%	2025-03-15	Иванов Алексей Назначить

Кадры эпизода

Кадр №	План сдачи	Факт сдачи	Действия
1	2025-02-01	2025-02-01	Редакт. Удалить
2	2025-02-05	2025-02-05	Редакт. Удалить

Рисунок 3.2.7 - Карточка эпизода: этапы эпизода и список кадров

Форма редактирования кадра предназначена для корректировки номера кадра в эпизоде и дат сдачи (план/факт). Эти значения используются при контроле сроков и в аналитических выборках по производительности. Пример формы приведён на рисунке 3.2.8.

Рисунок 3.2.8 - Форма редактирования кадра

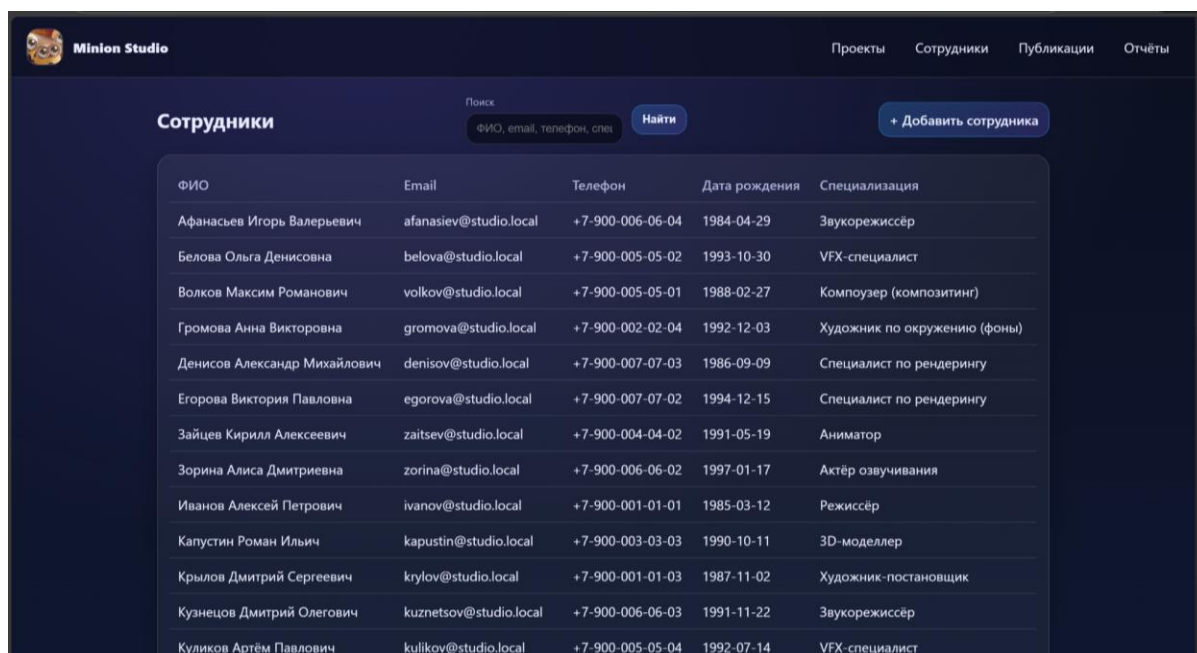
Карточка кадра является основным рабочим экраном для фиксации хода производства на детальном уровне. В таблице этапов кадра отображаются ответственный сотрудник, статус, прогресс и плановая дата. Доступны операции смены статуса этапа, назначение исполнителя и фиксация результата проверки качества с комментарием. Форма проверки качества позволяет выбирать проверяющего сотрудника и результат из справочника, после чего запись сохраняется в журнале проверок. Пример интерфейса приведён на рисунке 3.2.9.

Этап	Ответственный	Статус	Прогресс	План
Моделирование	Лебедев Сергей	Завершён	100%	2025-02-01
Анимация	Смирнова Анна	Завершён	100%	2025-02-01

Рисунок 3.2.9 - Карточка кадра: этапы кадра, смена статуса, назначение исполнителя и запись проверки качества

Раздел «Сотрудники» содержит справочник персонала студии и используется для поиска, просмотра и корректировки данных. В списке

отображаются ФИО, электронная почта, телефон, дата рождения и специализация; предусмотрен поисковый фильтр по ключевым полям и действие создания новой записи. Пример экрана приведён на рисунке 3.2.10.



ФИО	Email	Телефон	Дата рождения	Специализация
Афанасьев Игорь Валерьевич	afanasiev@studio.local	+7-900-006-06-04	1984-04-29	Звукорежиссёр
Белова Ольга Денисовна	belova@studio.local	+7-900-005-05-02	1993-10-30	VFX-специалист
Волков Максим Романович	volkov@studio.local	+7-900-005-05-01	1988-02-27	Композитер (композитинг)
Громова Анна Викторовна	gromova@studio.local	+7-900-002-02-04	1992-12-03	Художник по окружению (фоны)
Денисов Александр Михайлович	denisov@studio.local	+7-900-007-07-03	1986-09-09	Специалист по рендерингу
Егорова Виктория Павловна	egorova@studio.local	+7-900-007-07-02	1994-12-15	Специалист по рендерингу
Зайцев Кирилл Алексеевич	zaitsev@studio.local	+7-900-004-04-02	1991-05-19	Аниматор
Зорина Алиса Дмитриевна	zorina@studio.local	+7-900-006-06-02	1997-01-17	Актёр озвучивания
Иванов Алексей Петрович	ivanov@studio.local	+7-900-001-01-01	1985-03-12	Режиссёр
Капустин Роман Ильич	kapustin@studio.local	+7-900-003-03-03	1990-10-11	3D-моделлер
Крылов Дмитрий Сергеевич	krylov@studio.local	+7-900-001-01-03	1987-11-02	Художник-постановщик
Кузнецов Дмитрий Олегович	kuznetsov@studio.local	+7-900-006-06-03	1991-11-22	Звукорежиссёр
Куликов Артём Павлович	kulikov@studio.local	+7-900-005-05-04	1992-07-14	VFX-специалист

Рисунок 3.2.10 - Раздел «Сотрудники»: список сотрудников с поиском и справочными атрибутами

Карточка сотрудника объединяет справочные данные и производственную «загрузку». В блоке загрузки выводится количество назначенных этапов и их распределение по состояниям (активные, требующие доработки и др.), а также таблица этапов, где сотрудник указан ответственным, с привязкой к проекту/эпизоду/кадру и срокам. Ниже размещена форма редактирования атрибутов сотрудника. Пример карточки приведён на рисунке 3.2.11.

Афанасьев Игорь Валерьевич

→ К сотрудникам

Карточка сотрудника

Email

afanasiev@studio.local

Телефон

+7-900-006-06-04

Дата рождения

1984-04-29

Специализация

Звукорежиссёр

Загрузка сотрудника

Всего этапов: 3 Активных: 3 Требуется доработки: 0

Показаны этапы, где сотрудник указан ответственным.

Этап	Статус	Проект	Эпизод	Кадр	Прогресс	План
Звук и музыка	Не начат	—	Часть 1	—	0%	2025-12-14
Звук и музыка	Не начат	—	Часть 2	—	0%	2025-12-24
Звук и музыка	Не начат	—	Гадкий	—	—	—

Редактировать сотрудника

Фамилия

Афанасьев

Имя

Игорь

Отчество

Валерьевич

Email

afanasiev@studio.local

Телефон

+7-900-006-06-04

Дата рождения

29.04.1984

Специализация

Звукорежиссёр

Сохранить

Удалить сотрудника

Рисунок 3.2.11 - Карточка сотрудника: данные, редактирование и сводка загрузки по назначенным этапам

Раздел «Публикации» предназначен для централизованного контроля релизов эпизодов. Таблица содержит связку «проект – эпизод», статус публикации и даты (план/факт), а также действия редактирования и удаления записи. Предусмотрен фильтр по проекту, упрощающий мониторинг релизов внутри выбранного проекта. Пример экрана приведён на рисунке 3.2.12.

Minion Studio

Проекты Сотрудники Публикации Отчёты

Публикации

Фильтр по проекту

Название проекта

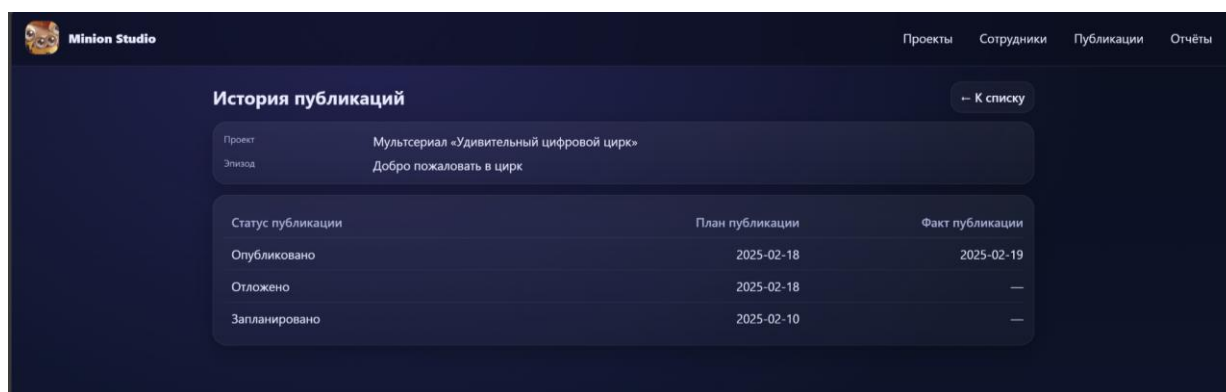
Найти

+ Добавить запись

Проект	Эпизод	Статус публикации	План публикации	Факт публикации	Действия
Мультсериал «Рик и Морти»	Пилотный эпизод	Отложено	2025-04-05	—	Редакт. Удалить
Мультсериал «Удивительный цифровой цирк»	Добро пожаловать в цирк	Опубликовано	2025-02-18	2025-02-19	Редакт. Удалить
Мультфильм «Гадкий я»	Часть 1	Запланировано	2026-01-23	—	Редакт. Удалить

Рисунок 3.2.12 - Раздел «Публикации»: список записей графика публикаций
Экран истории публикаций отображает последовательность изменений статусов публикации для выбранного эпизода, что позволяет фиксировать

переносы, публикации и повторные планирования без потери предыдущих значений. Пример истории публикаций приведён на рисунке 3.2.13.



Проект	Мультсериал «Удивительный цифровой цирк»		
Эпизод	Добро пожаловать в цирк		
Статус публикации	План публикации	Факт публикации	
Опубликовано	2025-02-18	2025-02-19	
Отложено	2025-02-18	—	
Запланировано	2025-02-10	—	

Рисунок 3.2.13 - История публикаций эпизода

3.3 Демонстрация выполнения аналитических запросов

Раздел «Отчёты» предназначен для демонстрации выполнения аналитических SQL-запросов и формирования сводной информации по проектам и этапам производства. Пользователь выбирает параметры отчёта (дата/месяц или порог отставания), после чего веб-приложение выполняет соответствующий запрос к PostgreSQL и отображает результат в табличной форме.

Набор отчётов охватывает ключевые управленческие показатели: текущий прогресс проектов на выбранную дату, выявление проектов с превышением порога отставания по плановым срокам, производительность художников по количеству выполненных кадров, среднюю длительность выбранного этапа («компози́тинг») по проектам, список эпизодов/кадров, ожидающих проверки, распределение текущей нагрузки по сотрудникам, прогнозируемую дату завершения проекта по фактическому темпу, а также статистику брака и доработок по этапам на основании результатов контроля качества.

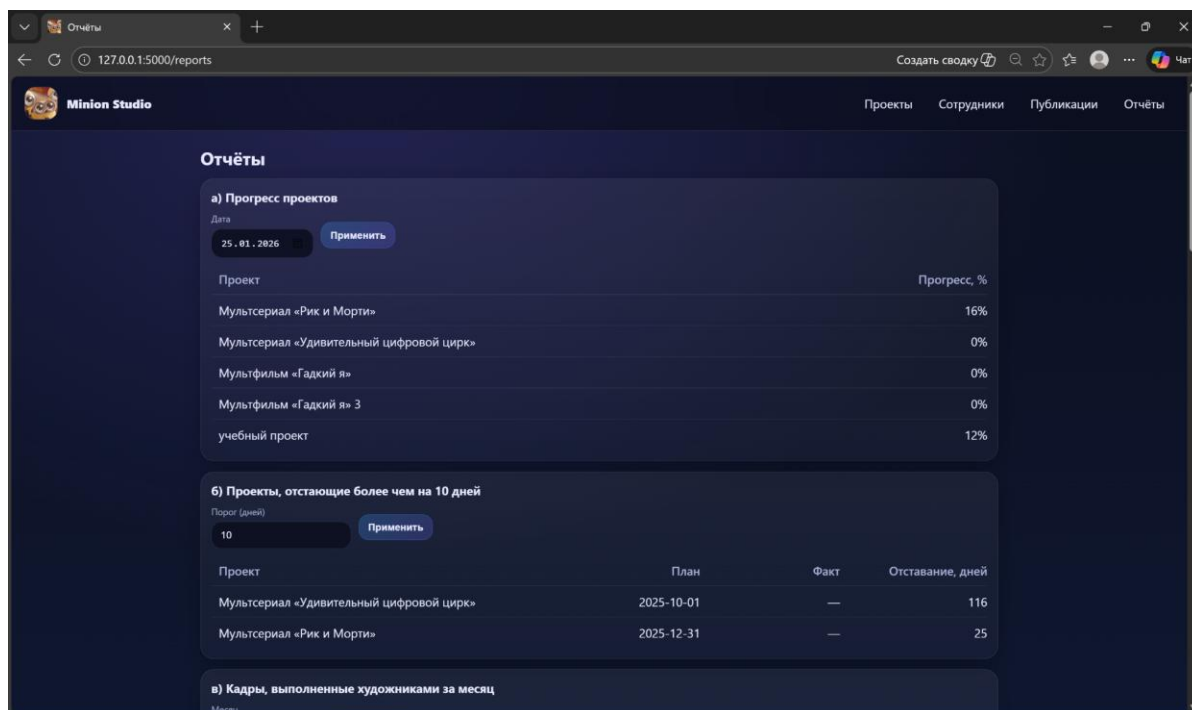


Рисунок 3.3.1 - Раздел «Отчёты»: прогресс проектов и проекты, отстающие более чем на заданный порог

Отчёт «Прогресс проектов» рассчитывается как среднее значение прогресса по всем этапам, относящимся к проекту, на выбранную календарную дату. Данные прогресса берутся из таблицы DailyProgress; при отсутствии записи по этапу на эту дату используется значение 0, что позволяет корректно отображать неприступившие этапы.

Отчёт «Проекты, отстающие более чем на N дней» сравнивает плановую дату завершения проекта с фактической датой (если проект ещё не завершён – с текущей датой) и выводит только проекты, превышающие введённый пользователем порог отставания.

в) Кадры, выполненные художниками за месяц

Месяц: Январь 2026 Применить

Период: 2026-01-01

Художник	Кадров
Белова Ольга	4
Лебедев Сергей	4
Савельев Роман	4
Смирнова Анна	4
Волков Максим	2
Егорова Виктория	2

г) Среднее время этапа «композитинг» по проектам

Месяц: Январь 2026 Применить

Проект	Среднее, часов
Мультсериал «Рик и Морти»	240.0
Мультсериал «Удивительный цифровой цирк»	—
Мультфильм «Гадкий я»	—

Рисунок 3.3.2 - Кадры, выполненные художниками за месяц, и среднее время этапа «композитинг» по проектам

Отчёт «Кадры, выполненные художниками за месяц» агрегирует завершённые этапы кадрового уровня за выбранный месяц и показывает количество уникальных кадров, по которым у сотрудника были выполнены этапы.

Отчёт «Среднее время этапа композитинг» вычисляет среднюю длительность (в часах) между датой начала и фактическим завершением этапа композитинга в пределах выбранного месяца, что используется для оценки трудоёмкости и планирования ресурсов.

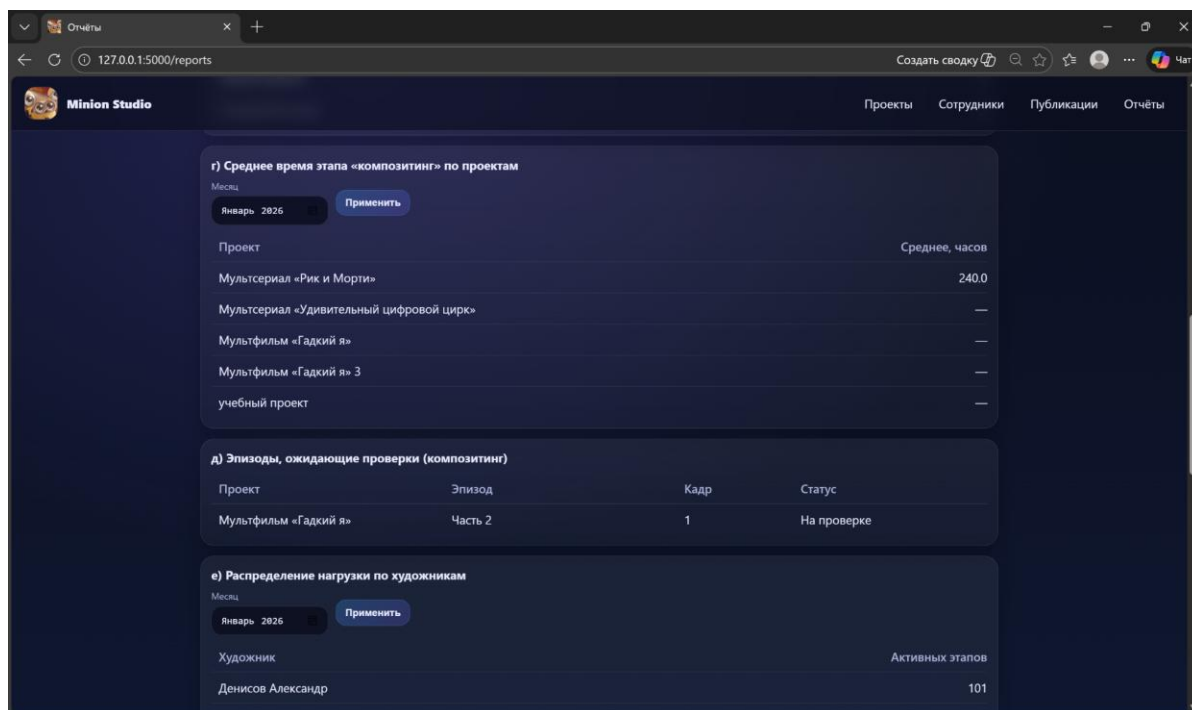


Рисунок 3.3.3 - Эпизоды, ожидающие проверки (композитинг), и распределение нагрузки по художникам

Отчёт «Эпизоды, ожидающие проверки» формирует список эпизодов и кадров, для которых этап композитинга находится в статусе проверки. Это позволяет оперативно выявлять «узкие места» на контроле качества и распределять внимание проверяющих.

Отчёт «Распределение нагрузки по художникам» показывает количество активных этапов, назначенных на каждого сотрудника (исключая статусы «Принят» и «Завершён»). Сводка используется для балансировки задач между исполнителями.

е) Распределение нагрузки по художникам

Месяц: **Январь 2026** Применить

Художник	Активных этапов
Денисов Александр	101
Савельев Роман	101
Белова Ольга	100
Волков Максим	100
Лебедев Сергей	99
Смирнова Анна	99
Иванов Алексей	15
Морозов Артём	4
Афанасьев Игорь	3
Крылов Дмитрий	3
Егорова Виктория	1
Кузнецов Дмитрий	1

Рисунок 3.3.4 - Распределение нагрузки по художникам: пример вывода для выбранного периода

ж) Прогноз завершения проекта

Проект: **Мультсериал «Рик и Морти»** Применить

Проект	Прогресс, %	Темп/день	Прогноз
Мультсериал «Рик и Морти»	74%	0.26	2026-05-05

Рисунок 3.3.5 - Прогноз завершения проекта на основе текущего прогресса и темпа выполнения

Отчёт «Прогноз завершения проекта» рассчитывает текущий процент выполнения проекта и темп выполнения по последним 7 дням. Темп определяется как прирост общего процента готовности за выбранный период, приведённый к значению «процент в день». На основе рассчитанного темпа определяется прогнозная дата достижения 100% готовности. Если за последние 7 дней прогресс не изменялся (темп равен нулю), прогноз не рассчитывается и выводится прочерк.

з) Брак и доработки по этапам			
Этап	Брак	Доработки	Не принят
Аниматик / превизуализация	0	0	0
Анимация	0	2	0
Визуальные эффекты (VFX)	0	1	0
Звук и музыка	0	0	0
Композитинг	0	1	0
Концепт-арт и дизайн	0	0	0
Моделирование	0	0	0
Озвучивание	0	0	0
Разработка идеи и сценария	0	0	0
Рендеринг	0	0	0
Создание раскадровки	0	1	0
Техническая подготовка	0	0	0
Финальная приёмка эпизода	0	1	0

Рисунок 3.3.6 - Брак и доработки по этапам на основании результатов проверок качества

Отчёт «Брак и доработки по этапам» строится по данным StageCheck и справочника результатов проверки (CheckResult) и отражает количество случаев брака, доработок и непринятия по каждому типу этапа. Сводка применяется для выявления наиболее проблемных участков производственного конвейера и уточнения регламентов контроля качества.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсового проекта была спроектирована и реализована реляционная база данных для управления производством анимационного контента, включающая сущности проектов, эпизодов, кадров, этапов, сотрудников, статусов и результатов проверок качества. Для обеспечения целостности данных и автоматизации типовых операций использованы ограничения, триггеры и процедурные объекты PostgreSQL.

Для демонстрации работы разработана веб-система (клиент–серверная архитектура), обеспечивающая ввод и просмотр данных, назначение ответственных исполнителей, ведение статусов, а также формирование аналитической отчётности. Реализованные отчёты позволяют контролировать прогресс проектов, выявлять отклонения от графика, оценивать загрузку сотрудников, анализировать качество выполнения этапов и прогнозировать сроки завершения. Полученные результаты подтверждают корректность выбранной структуры данных и применимость разработанного решения для поддержки управленческих задач в производственном конвейере анимационной студии.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

- 1 ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления = System of standards on information, librarianship and publishing. The research report. Structure and rules of presentation : издание официальное. – Москва : Стандартинформ, 2018. URL: <https://protect.gost.ru/document.aspx?control=7&baseC=6&page=0&month=3&year=2022&search=7.32&RegNum=1&DocOnPageCount=15&id=218998> (дата обращения: 24.10.2024) . - Текст: непосредственный.
- 2 Нормализация отношений. Шесть нормальных форм // Хабр. – 2 апр. 2015. – URL: <https://habr.com/ru/articles/254773/> (дата обращения: 20.01.2026).
- 3 Руководство по SQL. Процедурный язык PSQL // РЕД База Данных: документация. – URL: <https://rdb.red-soft.ru/docs/html/rdb50/Руководство%20по%20SQL/sec-psql.html> (дата обращения: 10.01.2026).
- 4 PostgreSQL Documentation. PL/pgSQL – SQL Procedural Language: Overview. – URL: <https://www.postgresql.org/docs/current/plpgsql-overview.html#PLPGSQL-ADVANTAGES> (дата обращения: 20.01.2026).
- 5 PostgreSQL Documentation. 5.5. Constraints : [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/current/ddl-constraints.html> (дата обращения: 11.01.2026).
- 6 PostgreSQL Documentation. 18.3.3. Foreign Keys (Tutorial) : [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/current/tutorial-fk.html> (дата обращения: 20.01.2026).
- 7 Psycopg 2.9.11 documentation. Basic module usage : [Электронный ресурс]. – URL: <https://www.psycopg.org/docs/usage.html> (дата обращения: 11.01.2026).
- 8 Python documentation. [Электронный ресурс]. – URL: <https://docs.python.org/> (дата обращения: 11.01.2026).

9 Кравченко К. В. СОЗДАНИЕ АНИМАЦИОННОГО МУЛЬТФИЛЬМА С ПОМОЩЬЮ СРЕДЫ CINEMA 4D // Теория и практика современной науки. 2020. №2 (56). URL: <https://cyberleninka.ru/article/n/sozдание-animatsionnogo-multfilma-s-pomoschyu-sredy-cinema-4d> (дата обращения: 24.01.2026).

10 Как создают 3D-мультфильмы: полное руководство [Электронный ресурс]. – Contented. – 18.12.2025. – URL: <https://media.contented.ru/opyt/instrukcii/kak-sozdayut-3d-multfilmy/> (дата обращения: 20.01.2026).

11 Как создаются мультфильмы: раскрываем секреты производства [Электронный ресурс]. – Смотрим.ру. – 12.09.2025. – URL: <https://smotrim.ru/article/4683180> (дата обращения: 20.01.2026).

Приложение А

Скрипт SQL для создания структуры базы данных

```
-- 1) Reference / dictionary tables first

CREATE TABLE "CheckResult"
(
    "id_check_result"    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    "check_result_name" VARCHAR(50)
);

CREATE TABLE "EpisodeStatus"
(
    "id_episode_status"  BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    "episode_status_name" VARCHAR(50)
);

CREATE TABLE "ProjectStatus"
(
    "id_project_status"  BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    "project_status_name" VARCHAR(50)
);

CREATE TABLE "PublicationStatus"
(
    "id_publication_status"  BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    "publication_status_name" VARCHAR(50)
);

CREATE TABLE "Spesialization"
(
    "id_spesialization"    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    "spesialization_name"  VARCHAR(100)
);

CREATE TABLE "StageStatus"
(
    "id_stage_status"      BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    "stage_status_name"    VARCHAR(50)
);

CREATE TABLE "StageType"
(
    "stage_type_id"        BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    "stage_type_name"      VARCHAR(100),
    "stage_level"          VARCHAR(50),
    "is_visual_stage"      BOOLEAN,
    "is_voice_stage"       BOOLEAN,
    "requires_quality_check" BOOLEAN,
    "execution_order"      INTEGER
);

-- 2) Main entities

CREATE TABLE "Employees"
(
    "id_employee"          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    "surname"              VARCHAR(50),
    "name"                 VARCHAR(50),
    "patronymic"           VARCHAR(50),
    "email"                VARCHAR(100),
    "phone"                VARCHAR(20),
    "birth_date"           DATE,
    "is_active"            BOOLEAN DEFAULT TRUE,
    "id_spesialization"    BIGINT NOT NULL,
    CONSTRAINT "R_40" FOREIGN KEY ("id_spesialization")
        REFERENCES "Spesialization" ("id_spesialization")
);
```

```

CREATE TABLE "Project"
(
    "id_project"          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    "project_name"        VARCHAR(100),
    "description"         TEXT,
    "start_date"          DATE,
    "planned_end_date"    DATE,
    "actual_end_date"     DATE,
    "id_project_status"   BIGINT NOT NULL,
    CONSTRAINT "R_45" FOREIGN KEY ("id_project_status")
        REFERENCES "ProjectStatus" ("id_project_status")
);

CREATE TABLE "Episode"
(
    "id_episode"          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    "episode_name"        VARCHAR(100),
    "episode_number_in_project" INTEGER,
    "planned_release_date" DATE,
    "id_project"          BIGINT NOT NULL,
    "actual_release_date" DATE,
    "ready_for_release_flag" BOOLEAN DEFAULT FALSE,
    "ready_for_release_date" DATE,
    "id_episode_status"   BIGINT NOT NULL,
    CONSTRAINT "R_16" FOREIGN KEY ("id_project")
        REFERENCES "Project" ("id_project"),
    CONSTRAINT "R_46" FOREIGN KEY ("id_episode_status")
        REFERENCES "EpisodeStatus" ("id_episode_status")
);

CREATE TABLE "Frame"
(
    "id_frame"            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    "id_episode"          BIGINT NOT NULL,
    "frame_number_in_episode" INTEGER,
    "planned_due_date"    DATE,
    "actual_done_date"    DATE,
    CONSTRAINT "R_17" FOREIGN KEY ("id_episode")
        REFERENCES "Episode" ("id_episode")
);

CREATE TABLE "FrameEmployee"
(
    "id_employee"         BIGINT NOT NULL,
    "id_frame"            BIGINT NOT NULL,
    "role_in_frame"       VARCHAR(50),
    "assignment_date"     DATE,
    PRIMARY KEY ("id_employee", "id_frame"),
    CONSTRAINT "R_28" FOREIGN KEY ("id_employee")
        REFERENCES "Employees" ("id_employee"),
    CONSTRAINT "R_29" FOREIGN KEY ("id_frame")
        REFERENCES "Frame" ("id_frame")
);

-- 3) Stage-related tables

CREATE TABLE "Stage"
(
    "id_stage"            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    "id_project"          BIGINT,
    "id_episode"          BIGINT,
    "id_frame"            BIGINT,
    "id_responsible_employee" BIGINT NOT NULL,
    "stage_type_id"       BIGINT NOT NULL,
    "id_stage_status"     BIGINT NOT NULL,
    "start_datetime"      TIMESTAMP,
    "actual_end_datetime" TIMESTAMP,
    "planned_end_date"    DATE,

```

```

CONSTRAINT "R_47" FOREIGN KEY ("id_project")
REFERENCES "Project" ("id_project"),
CONSTRAINT "R_48" FOREIGN KEY ("id_episode")
REFERENCES "Episode" ("id_episode"),
CONSTRAINT "R_49" FOREIGN KEY ("id_frame")
REFERENCES "Frame" ("id_frame"),
CONSTRAINT "R_50" FOREIGN KEY ("id_responsible_employee")
REFERENCES "Employees" ("id_employee"),
CONSTRAINT "R_51" FOREIGN KEY ("stage_type_id")
REFERENCES "StageType" ("stage_type_id"),
CONSTRAINT "R_53" FOREIGN KEY ("id_stage_status")
REFERENCES "StageStatus" ("id_stage_status")
);

-- Exactly one of id_project / id_episode / id_frame must be filled
ALTER TABLE "Stage"
ADD CONSTRAINT "stage_scope_chk" CHECK (
((CASE WHEN "id_project" IS NOT NULL THEN 1 ELSE 0 END) +
(CASE WHEN "id_episode" IS NOT NULL THEN 1 ELSE 0 END) +
(CASE WHEN "id_frame" IS NOT NULL THEN 1 ELSE 0 END)) = 1
);

CREATE TABLE "DailyProgress"
(
    "id_progress"          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    "id_employee"          BIGINT NOT NULL,
    "id_stage"             BIGINT NOT NULL,
    "progress_date"        DATE,
    "progress_percent"      INTEGER,

    CONSTRAINT "R_31" FOREIGN KEY ("id_employee")
REFERENCES "Employees" ("id_employee"),
    CONSTRAINT "R_57" FOREIGN KEY ("id_stage")
REFERENCES "Stage" ("id_stage")
);

CREATE TABLE "StageCheck"
(
    "id_stagecheck"        BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    "id_checker_employee"  BIGINT NOT NULL,
    "id_stage"             BIGINT NOT NULL,
    "id_check_result"      BIGINT NOT NULL,
    "comment"              VARCHAR(255),
    "check_datetime"       TIMESTAMP,

    CONSTRAINT "R_38" FOREIGN KEY ("id_checker_employee")
REFERENCES "Employees" ("id_employee"),
    CONSTRAINT "R_55" FOREIGN KEY ("id_stage")
REFERENCES "Stage" ("id_stage"),
    CONSTRAINT "R_59" FOREIGN KEY ("id_check_result")
REFERENCES "CheckResult" ("id_check_result")
);

-- StageStatusHistory as M:N between Stage and StageStatus (associative table)
CREATE TABLE "StageStatusHistory"
(
    "id_stage"             BIGINT NOT NULL,
    "id_stage_status"      BIGINT NOT NULL,
    "change_datetime"      TIMESTAMP NOT NULL,
    "id_employee"          BIGINT NOT NULL,
    "change_reason"        VARCHAR(255),
    "comment"              VARCHAR(255),

    CONSTRAINT "pk_stage_status_history"
PRIMARY KEY ("id_stage", "id_stage_status", "change_datetime"),

    CONSTRAINT "R_56" FOREIGN KEY ("id_stage")
REFERENCES "Stage" ("id_stage"),

    CONSTRAINT "R_52" FOREIGN KEY ("id_stage_status")

```

```

        REFERENCES "StageStatus" ("id_stage_status"),

CONSTRAINT "R_36" FOREIGN KEY ("id_employee")
    REFERENCES "Employees" ("id_employee")
);

CREATE TABLE "Rework"
(
    "id_rework"          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    "id_stage"           BIGINT NOT NULL,
    "rework_reason"      VARCHAR(255),
    "defect_description" VARCHAR(255),
    "open_date"          DATE,
    "close_date"         DATE,
    "is_closed"          BOOLEAN DEFAULT FALSE,

    CONSTRAINT "R_58" FOREIGN KEY ("id_stage")
        REFERENCES "Stage" ("id_stage")
);

-- 4) Publication schedule

CREATE TABLE "PublicationSchedule"
(
    "id_publication_schedule" BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    "id_episode"              BIGINT NOT NULL,
    "id_publication_status"   BIGINT NOT NULL,
    "planned_publication_date" DATE,
    "actual_publication_date"  DATE,

    CONSTRAINT "R_20" FOREIGN KEY ("id_episode")
        REFERENCES "Episode" ("id_episode"),
    CONSTRAINT "R_60" FOREIGN KEY ("id_publication_status")
        REFERENCES "PublicationStatus" ("id_publication_status")
);

```

Приложение Б

Аналитические SQL запросы

Б1) Прогресс проектов на выбранную дату

```
WITH stage_project AS (
    SELECT
        s."id_stage",
        COALESCE(s."id_project", e."id_project", e2."id_project") AS id_project
    FROM "Stage" s
    LEFT JOIN "Episode" e ON e."id_episode" = s."id_episode"
    LEFT JOIN "Frame" f ON f."id_frame" = s."id_frame"
    LEFT JOIN "Episode" e2 ON e2."id_episode" = f."id_episode"
),
today_dp AS (
    SELECT
        dp."id_stage",
        dp."progress_percent",
        ROW_NUMBER() OVER (PARTITION BY dp."id_stage"
                           ORDER BY dp."progress_date" DESC, dp."id_progress" DESC) AS
rn
    FROM "DailyProgress" dp
    JOIN stage_project sp ON sp."id_stage" = dp."id_stage"
    WHERE dp."progress_date" = DATE '2026-01-25'
),
totals AS (
    SELECT
        sp.id_project,
        COUNT(*) AS total_stages,
        COALESCE(SUM(COALESCE(t.progress_percent, 0)), 0) AS today_sum
    FROM stage_project sp
    LEFT JOIN today_dp t ON t."id_stage" = sp."id_stage" AND t.rn = 1
    GROUP BY sp.id_project
)
SELECT
    p."id_project",
    p."project_name",
    CASE
        WHEN COALESCE(t.total_stages, 0) = 0 THEN 0
        ELSE ROUND((t.today_sum::numeric / t.total_stages::numeric), 2)
    END AS overall_percent
FROM "Project" p
LEFT JOIN totals t ON t.id_project = p."id_project"
ORDER BY p."project_name";
```

Б2) Проекты, отстающие от графика более чем на заданный порог

(в днях)

```
SELECT
    p."project_name",
    p."planned_end_date",
    p."actual_end_date",
    (COALESCE(p."actual_end_date", CURRENT_DATE) - p."planned_end_date")::int AS
delay_days
FROM "Project" p
WHERE p."planned_end_date" IS NOT NULL
    AND (COALESCE(p."actual_end_date", CURRENT_DATE) - p."planned_end_date") > 10
ORDER BY delay_days DESC;
```

Б3) Кадры, выполненные художниками за месяц

```
SELECT
    e."surname" || ' ' || e."name" AS artist,
    COUNT(DISTINCT s."id_frame") AS frames_done
FROM "Stage" s
JOIN "Employees" e ON e."id_employee" = s."id_responsible_employee"
WHERE s."id_frame" IS NOT NULL
```

```

AND s."actual_end_datetime" >= DATE '2026-01-01'
AND s."actual_end_datetime" < DATE '2026-02-01'
GROUP BY artist
ORDER BY frames_done DESC, artist;

```

Б4) Среднее время этапа «компози́тинг» по проектам (в часах) за месяц

```

WITH stage_project AS (
    SELECT
        s."id_stage",
        s."stage_type_id",
        s."start_datetime",
        s."actual_end_datetime",
        COALESCE(s."id_project", e."id_project", e2."id_project") AS id_project
    FROM "Stage" s
    LEFT JOIN "Episode" e ON e."id_episode" = s."id_episode"
    LEFT JOIN "Frame" f ON f."id_frame" = s."id_frame"
    LEFT JOIN "Episode" e2 ON e2."id_episode" = f."id_episode"
)
SELECT
    p."project_name",
    AVG(
        CASE
            WHEN st."stage_type_id" IS NOT NULL
            THEN EXTRACT(EPOCH FROM (sp."actual_end_datetime" - sp."start_datetime")) /
3600.0
            ELSE NULL
        END
    ) AS avg_hours
FROM "Project" p
LEFT JOIN stage_project sp ON sp.id_project = p."id_project"
AND sp."start_datetime" IS NOT NULL
AND sp."actual_end_datetime" IS NOT NULL
AND sp."actual_end_datetime" >= DATE '2026-01-01'
AND sp."actual_end_datetime" < DATE '2026-02-01'
LEFT JOIN "StageType" st ON st."stage_type_id" = sp."stage_type_id"
AND st."stage_type_name" ILIKE '%компо́зит%'
GROUP BY p."project_name"
ORDER BY p."project_name";

```

Б5) Эпизоды, ожидающие проверки (компози́тинг)

```

WITH stage_ctx AS (
    SELECT
        s."id_stage",
        s."id_episode",
        s."id_frame"
    FROM "Stage" s
),
ep_ctx AS (
    SELECT
        sc."id_stage",
        COALESCE(sc."id_episode", f."id_episode") AS id_episode
    FROM stage_ctx sc
    LEFT JOIN "Frame" f ON f."id_frame" = sc."id_frame"
)
SELECT DISTINCT
    p."project_name",
    e."episode_name",
    f."frame_number_in_episode",
    ss."stage_status_name"
FROM "Stage" s
JOIN "StageType" st ON st."stage_type_id" = s."stage_type_id"
JOIN ep_ctx ec ON ec."id_stage" = s."id_stage"
JOIN "Episode" e ON e."id_episode" = ec.id_episode
JOIN "Project" p ON p."id_project" = e."id_project"

```



```

LEFT JOIN "Frame" f ON f."id_frame" = s."id_frame"
LEFT JOIN "StageStatus" ss ON ss."id_stage_status" = s."id_stage_status"
WHERE st."stage_type_name" ILIKE '%%композит%%'
      AND ss."stage_status_name" ILIKE '%%провер%%'
      AND (s."start_datetime" IS NULL OR s."start_datetime" < DATE '2026-02-01')
ORDER BY p."project_name", e."episode_name", f."frame_number_in_episode";

```

Б6) Распределение нагрузки по художникам (количество активных этапов)

```

SELECT
    e."surname" || ' ' || e."name" AS artist,
    COUNT(DISTINCT s."id_stage") AS active_stages
FROM "Stage" s
JOIN "Employees" e ON e."id_employee" = s."id_responsible_employee"
JOIN "StageStatus" ss ON ss."id_stage_status" = s."id_stage_status"
WHERE ss."stage_status_name" NOT IN ('Принят', 'Завершён')
GROUP BY artist
ORDER BY active_stages DESC, artist;

```

Б7) Прогноз завершения проекта по темпу выполнения

```

# g) Forecast finish date (pace from last 7 days)
forecast_rows = fetch_all(''
    WITH stage_project AS (
        SELECT
            s."id_stage",
            s."id_stage_status",
            COALESCE(s."id_project", e."id_project", e2."id_project") AS
id_project
        FROM "Stage" s
        LEFT JOIN "Episode" e ON e."id_episode" = s."id_episode"
        LEFT JOIN "Frame" f ON f."id_frame" = s."id_frame"
        LEFT JOIN "Episode" e2 ON e2."id_episode" = f."id_episode"
    ),
    progress_at AS (
        -- прогресс проекта на двух датах: сегодня и 7 дней назад
        SELECT
            p."id_project",
            x.as_of,
            AVG(
                COALESCE(
                    ld."progress_percent",
                    CASE
                        WHEN ss."stage_status_name" ILIKE '%%не начат%%' THEN 0
                        WHEN ss."stage_status_name" ILIKE '%%в работе%%' THEN 50
                        WHEN ss."stage_status_name" ILIKE '%%доработ%%' THEN 70
                        WHEN ss."stage_status_name" ILIKE '%%заверш%%' OR
ss."stage_status_name" ILIKE '%%принят%%' THEN 100
                        WHEN ss."stage_status_name" ILIKE '%%провер%%' THEN 90
                        ELSE 0
                    END
                )
            ) AS overall_percent_raw
        FROM "Project" p
        LEFT JOIN stage_project sp ON sp.id_project = p."id_project"
        CROSS JOIN (VALUES (CURRENT_DATE), (CURRENT_DATE - 7)) AS x(as_of)
        LEFT JOIN LATERAL (
            SELECT dp."progress_percent"
            FROM "DailyProgress" dp
            WHERE dp."id_stage" = sp."id_stage"
            AND dp."progress_date" <= x.as_of
            ORDER BY dp."progress_date" DESC, dp."id_progress" DESC
            LIMIT 1
        ) ld ON true
        LEFT JOIN "StageStatus" ss ON ss."id_stage_status" = sp."id_stage_status"
        GROUP BY p."id_project", x.as_of
    )

```

```

    ),
    pivot AS (
        SELECT
            "id_project",
            MAX(overall_percent_raw) FILTER (WHERE as_of = CURRENT_DATE) AS
p_today,
            MAX(overall_percent_raw) FILTER (WHERE as_of = CURRENT_DATE - 7) AS
p_prev
        FROM progress_at
        GROUP BY "id_project"
    )
    SELECT
        p."project_name",
        ROUND(COALESCE(v.p_today, 0), 0) AS overall_percent,
        ROUND(GREATEST(0, (COALESCE(v.p_today, 0) - COALESCE(v.p_prev, 0)) / 7.0),
2) AS daily_rate,
        CASE
            WHEN GREATEST(0, (COALESCE(v.p_today, 0) - COALESCE(v.p_prev, 0)) / 7.0)
> 0
            THEN (CURRENT_DATE
                + CEIL((100 - COALESCE(v.p_today, 0))
                    / GREATEST(0, (COALESCE(v.p_today, 0) - COALESCE(v.p_prev, 0)) /
7.0)
                ) * INTERVAL '1 day')::date
            ELSE NULL
        END AS прогноз
    FROM "Project" p
    LEFT JOIN pivot v ON v."id_project" = p."id_project"
    WHERE (% (project_id)s IS NULL OR p."id_project" = %(project_id)s)
    ORDER BY p."project_name";
    '', {"project_id": project_j})

```

Б8) Брак и доработки по этапам (по результатам проверок

качества)

```

SELECT
    st."stage_type_name",
    COUNT(*) FILTER (WHERE cr."check_result_name" ILIKE '%%брак%%') AS defects,
    COUNT(*) FILTER (WHERE cr."check_result_name" ILIKE '%%доработ%%') AS reworks,
    COUNT(*) FILTER (WHERE cr."check_result_name" ILIKE '%%не принят%%') AS rejected
FROM "StageType" st
LEFT JOIN "Stage" s ON s."stage_type_id" = st."stage_type_id"
LEFT JOIN "StageCheck" sc ON sc."id_stage" = s."id_stage"
LEFT JOIN "CheckResult" cr ON cr."id_check_result" = sc."id_check_result"
GROUP BY st."stage_type_name"
ORDER BY st."stage_type_name";

```

Приложение В

Функции, триггеры и процедуры

В1) Функция расчёта прогресса проекта на дату

```

CREATE OR REPLACE FUNCTION "fn_project_progress_percent"
(p_project_id BIGINT, p_as_of DATE DEFAULT CURRENT_DATE)
RETURNS NUMERIC
LANGUAGE plpgsql
AS $$
DECLARE v_result NUMERIC;
BEGIN
    WITH stages_in_project AS (
        SELECT DISTINCT s.*
        FROM "Stage" s
        LEFT JOIN "Episode" e ON e."id_episode" = s."id_episode"
        LEFT JOIN "Frame" f ON f."id_frame" = s."id_frame"
        LEFT JOIN "Episode" e2 ON e2."id_episode" = f."id_episode"
        WHERE
            s."id_project" = p_project_id
            OR e."id_project" = p_project_id      -- этап уровня эпизода
            OR e2."id_project" = p_project_id    -- этап уровня кадра (через Frame ->
Episode)
    ),
    last_dp AS (
        SELECT DISTINCT ON (dp."id_stage")
            dp."id_stage",
            dp."progress_percent"
        FROM "DailyProgress" dp
        JOIN stages_in_project s ON s."id_stage" = dp."id_stage"
        WHERE dp."progress_date" <= p_as_of
        ORDER BY dp."id_stage", dp."progress_date" DESC
    ),
    stage_progress AS (
        SELECT
            s."id_stage",
            COALESCE(
                ld."progress_percent"::numeric,
                CASE ss."stage_status_name"
                    WHEN 'Не начат' THEN 0
                    WHEN 'В работе' THEN 50
                    WHEN 'Требуется доработки' THEN 70
                    WHEN 'Завершён' THEN 100
                    WHEN 'Принят' THEN 100
                    WHEN 'На проверке' THEN 90
                    ELSE 0
                END)::numeric
            ) AS progress_percent
        FROM stages_in_project s
        JOIN "StageStatus" ss ON ss."id_stage_status" = s."id_stage_status"
        LEFT JOIN last_dp ld ON ld."id_stage" = s."id_stage"
    )
    SELECT
        CASE WHEN COUNT(*) = 0 THEN 0
            ELSE ROUND(AVG(progress_percent), 2)
        END
    INTO v_result
    FROM stage_progress;

    RETURN COALESCE(v_result, 0);
END $$;

```

В2) Запрет начала озвучки до завершения визуальной части

эпизода

```
CREATE OR REPLACE FUNCTION "trg_voice_only_after_visual_done"()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
    v_stage_name TEXT;
    v_new_status TEXT;
    v_episode_id BIGINT;
    v_not_ready_cnt INT;
BEGIN
    SELECT st."stage_type_name"
    INTO v_stage_name
    FROM "StageType" st
    WHERE st."stage_type_id" = NEW."stage_type_id";

    IF v_stage_name NOT IN ('Озвучивание', 'Звук и музыка') THEN
        RETURN NEW;
    END IF;

    SELECT ss."stage_status_name"
    INTO v_new_status
    FROM "StageStatus" ss
    WHERE ss."id_stage_status" = NEW."id_stage_status";

    IF v_new_status IN ('В работе', 'На проверке', 'Требуется доработки', 'Принят', 'Завершён') THEN
        v_episode_id := NEW."id_episode";

        SELECT COUNT(*) INTO v_not_ready_cnt
        FROM "Frame" f
        WHERE f."id_episode" = v_episode_id
        AND EXISTS (
            SELECT 1
            FROM "Stage" s
            JOIN "StageType" st2 ON st2."stage_type_id" = s."stage_type_id"
            JOIN "StageStatus" ss2 ON ss2."id_stage_status" = s."id_stage_status"
            WHERE s."id_frame" = f."id_frame"
            AND st2."stage_type_name" = 'Рендеринг'
            AND ss2."stage_status_name" NOT IN ('Принят', 'Завершён')
        );

        IF v_not_ready_cnt > 0 THEN
            RAISE EXCEPTION
            'Нельзя начать озвучивание: визуальная часть эпизода не завершена.';
        END IF;
    END IF;

    RETURN NEW;
END $$;

CREATE TRIGGER "tg_voice_only_after_visual_done"
BEFORE INSERT OR UPDATE OF "id_stage_status" ON "Stage"
FOR EACH ROW
EXECUTE FUNCTION "trg_voice_only_after_visual_done"();
```

В3) Автоматическое обновление эпизода и графика публикации

после финальной проверки

```
CREATE OR REPLACE FUNCTION "trg_after_final_episode_check"()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
    v_result_name TEXT;
```

```

v_stage_name TEXT;
v_episode_id BIGINT;
v_episode_status_id BIGINT;
v_pub_status_id BIGINT;
v_planned_date DATE;
BEGIN
    SELECT cr."check_result_name"
    INTO v_result_name
    FROM "CheckResult" cr
    WHERE cr."id_check_result" = NEW."id_check_result";

    IF v_result_name <> 'Принято' THEN
        RETURN NEW;
    END IF;

    SELECT st."stage_type_name", s."id_episode"
    INTO v_stage_name, v_episode_id
    FROM "Stage" s
    JOIN "StageType" st ON st."stage_type_id" = s."stage_type_id"
    WHERE s."id_stage" = NEW."id_stage";

    IF v_stage_name <> 'Финальная приёмка эпизода' THEN
        RETURN NEW;
    END IF;

    SELECT "id_episode_status"
    INTO v_episode_status_id
    FROM "EpisodeStatus"
    WHERE "episode_status_name" = 'Готов к выпуску';

    UPDATE "Episode"
    SET
        "ready_for_release_flag" = TRUE,
        "ready_for_release_date" = NEW."check_datetime"::date,
        "id_episode_status" = v_episode_status_id
    WHERE "id_episode" = v_episode_id;

    SELECT "planned_release_date"
    INTO v_planned_date
    FROM "Episode"
    WHERE "id_episode" = v_episode_id;

    SELECT "id_publication_status"
    INTO v_pub_status_id
    FROM "PublicationStatus"
    WHERE "publication_status_name" = 'Запланировано';

    INSERT INTO "PublicationSchedule"
    ("id_episode", "id_publication_status", "planned_publication_date", "actual_publication_d
ate")
    VALUES
    (v_episode_id, v_pub_status_id, v_planned_date, NULL);

    RETURN NEW;
END $$;

CREATE TRIGGER "tg_after_final_episode_check"
AFTER INSERT ON "StageCheck"
FOR EACH ROW
EXECUTE FUNCTION "trg_after_final_episode_check"();

```

В4) Логирование смены статусов этапов

```

CREATE OR REPLACE FUNCTION "trg_log_stage_status_change"()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW."id_stage_status" IS DISTINCT FROM OLD."id_stage_status" THEN
        INSERT INTO "StageStatusHistory"

```

```

("id_stage","id_stage_status","change_datetime","id_employee","change_reason")
VALUES
(
    NEW."id_stage",
    NEW."id_stage_status",
    NOW(),
    NEW."id_responsible_employee",
    'Изменение статуса'
);
END IF;

RETURN NEW;
END $$;

CREATE TRIGGER "tg_log_stage_status_change"
AFTER UPDATE OF "id_stage_status" ON "Stage"
FOR EACH ROW
EXECUTE FUNCTION "trg_log_stage_status_change"();

```

В5) Соблюдение очерёдности в этапах

```

CREATE OR REPLACE FUNCTION "trg_stage_must_follow_order"()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
    v_stage_level TEXT;
    v_exec_order INT;

    v_prev_stage_type_id BIGINT;
    v_prev_stage_status TEXT;
    v_new_stage_status TEXT;
BEGIN
    -- новый статус (текстом)
    SELECT ss."stage_status_name"
    INTO v_new_stage_status
    FROM "StageStatus" ss
    WHERE ss."id_stage_status" = NEW."id_stage_status";

    -- проверяем только если этап "двигают вперёд"
    IF v_new_stage_status NOT IN ('В работе', 'На проверке', 'Принят', 'Завершён') THEN
        RETURN NEW;
    END IF;

    -- уровень и порядок текущего этапа
    SELECT st."stage_level", st."execution_order"
    INTO v_stage_level, v_exec_order
    FROM "StageType" st
    WHERE st."stage_type_id" = NEW."stage_type_id";

    -- если это первый этап уровня - проверка не нужна
    IF v_exec_order = 1 THEN
        RETURN NEW;
    END IF;

    -- предыдущий тип этапа на этом же уровне
    SELECT st2."stage_type_id"
    INTO v_prev_stage_type_id
    FROM "StageType" st2
    WHERE st2."stage_level" = v_stage_level
    AND st2."execution_order" = v_exec_order - 1
    LIMIT 1;

    IF v_prev_stage_type_id IS NULL THEN
        RETURN NEW;
    END IF;

    -- получаем статус предыдущего этапа
    IF v_stage_level = 'Project' THEN
        SELECT ss."stage_status_name"
        INTO v_prev_stage_status

```

```

FROM "Stage" s
JOIN "StageStatus" ss ON ss."id_stage_status" = s."id_stage_status"
WHERE s."id_project" = NEW."id_project"
      AND s."stage_type_id" = v_prev_stage_type_id
LIMIT 1;

ELSIF v_stage_level = 'Episode' THEN
  SELECT ss."stage_status_name"
  INTO v_prev_stage_status
  FROM "Stage" s
  JOIN "StageStatus" ss ON ss."id_stage_status" = s."id_stage_status"
  WHERE s."id_episode" = NEW."id_episode"
        AND s."stage_type_id" = v_prev_stage_type_id
  LIMIT 1;

ELSE -- Frame
  SELECT ss."stage_status_name"
  INTO v_prev_stage_status
  FROM "Stage" s
  JOIN "StageStatus" ss ON ss."id_stage_status" = s."id_stage_status"
  WHERE s."id_frame" = NEW."id_frame"
        AND s."stage_type_id" = v_prev_stage_type_id
  LIMIT 1;
END IF;

-- если предыдущий этап есть и он не завершён - ошибка
IF v_prev_stage_status IS NOT NULL
  AND v_prev_stage_status NOT IN ('Принят', 'Завершён') THEN
  RAISE EXCEPTION
    'Нельзя изменить статус этапа: предыдущий этап (order=%) ещё не завершён
(текущий статус="%").',
    v_exec_order - 1,
    v_prev_stage_status;
END IF;

RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS "tg_stage_must_follow_order" ON "Stage";

CREATE TRIGGER "tg_stage_must_follow_order"
BEFORE INSERT OR UPDATE OF "id_stage_status"
ON "Stage"
FOR EACH ROW
EXECUTE FUNCTION "trg_stage_must_follow_order"();

```

В6) Запрет переключать статус без проверки предыдущего и без порядка

```

CREATE OR REPLACE FUNCTION "trg_stage_enforce_order_and_checks"()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
  v_cur_order int;
  v_prev_missing int;
  v_prev_need_check_missing int;
  v_done_status bigint;
  v_check_ok bigint;
BEGIN
  -- id статуса "Завершён" (подстрой под твои значения)
  SELECT "id_stage_status" INTO v_done_status
  FROM "StageStatus"
  WHERE "stage_status_name" = 'Завершён'
  LIMIT 1;

  -- id результата проверки "Принято/ОК" (подстрой под твои значения)

```

```

SELECT "id_check_result" INTO v_check_ok
FROM "CheckResult"
WHERE "check_result_name" IN ('Принято', 'OK', 'Passed')
LIMIT 1;

-- проверяем только когда пытаются поставить "Завершён"
IF NEW."id_stage_status" = v_done_status AND (OLD."id_stage_status" IS DISTINCT FROM
NEW."id_stage_status") THEN

    SELECT stt."execution_order" INTO v_cur_order
    FROM "StageType" stt
    WHERE stt."stage_type_id" = NEW."stage_type_id";

    -- 1) Все предыдущие этапы должны быть завершены
    SELECT COUNT(*) INTO v_prev_missing
    FROM "Stage" s2
    JOIN "StageType" t2 ON t2."stage_type_id" = s2."stage_type_id"
    WHERE s2."id_frame" = NEW."id_frame"
        AND t2."execution_order" < v_cur_order
        AND s2."id_stage_status" <> v_done_status;

    IF v_prev_missing > 0 THEN
        RAISE EXCEPTION 'Нельзя завершить этап: предыдущие этапы ещё не завершены';
    END IF;

    -- 2) Если предыдущие этапы требуют QC - должна быть успешная проверка
    SELECT COUNT(*) INTO v_prev_need_check_missing
    FROM "Stage" s2
    JOIN "StageType" t2 ON t2."stage_type_id" = s2."stage_type_id"
    WHERE s2."id_frame" = NEW."id_frame"
        AND t2."execution_order" < v_cur_order
        AND t2."requires_quality_check" = TRUE
        AND NOT EXISTS (
            SELECT 1
            FROM "StageCheck" sc
            WHERE sc."id_stage" = s2."id_stage"
                AND sc."id_check_result" = v_check_ok
        );

    IF v_prev_need_check_missing > 0 THEN
        RAISE EXCEPTION 'Нельзя завершить этап: нет успешной проверки качества по
        предыдущим этапам';
    END IF;

END IF;

RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS "tg_stage_enforce_order_and_checks" ON "Stage";
CREATE TRIGGER "tg_stage_enforce_order_and_checks"
BEFORE UPDATE OF "id_stage_status" ON "Stage"
FOR EACH ROW
EXECUTE FUNCTION "trg_stage_enforce_order_and_checks"();

```

В7) Процедура переноса публикации эпизода

```

CREATE OR REPLACE PROCEDURE "sp_reschedule_publication"
(
    p_episode_id BIGINT,
    p_new_planned_date DATE
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_postponed_status BIGINT;
BEGIN
    SELECT "id_publication_status"
    INTO v_postponed_status

```



```

FROM "PublicationStatus"
WHERE "publication_status_name" = 'Отложено';

INSERT INTO "PublicationSchedule"

("id_episode","id_publication_status","planned_publication_date","actual_publication_d
ate")
VALUES
(p_episode_id, v_postponed_status, p_new_planned_date, NULL);
END $$;

```

Пример вызова:

```
CALL "sp_reschedule_publication"(1, DATE '2025-04-05');
```

Проверка:

```

SELECT
    ps."id_publication_schedule",
    es."episode_name",
    pub."publication_status_name",
    ps."planned_publication_date",
    ps."actual_publication_date"
FROM "PublicationSchedule" ps
JOIN "Episode" es ON es."id_episode" = ps."id_episode"
JOIN "PublicationStatus" pub ON pub."id_publication_status" =
ps."id_publication_status"
WHERE ps."id_episode" = 1
ORDER BY ps."planned_publication_date";

```

В8) Процедура - Создание этапов для кадра

```

CREATE OR REPLACE PROCEDURE "sp_create_default_stages_for_frame"(p_frame_id bigint)
LANGUAGE plpgsql
AS $$
DECLARE
    v_planned_status bigint;
BEGIN
    -- статус этапа "Не начат" / "Запланирован" (подхватим любой из этих)
    SELECT "id_stage_status"
        INTO v_planned_status
    FROM "StageStatus"
    WHERE "stage_status_name" IN ('Не начат', 'Запланирован', 'В работе') -- подстрой,
если нужно
    ORDER BY CASE
        WHEN "stage_status_name"='Не начат' THEN 1
        WHEN "stage_status_name"='Запланирован' THEN 2
        ELSE 3
    END
    LIMIT 1;

    IF v_planned_status IS NULL THEN
        RAISE EXCEPTION 'Не найден стартовый статус этапа (StageStatus). Добавь "Не начат"
или "Запланирован".';
    END IF;

    -- если этапы уже есть - ничего не делаем (чтобы не дублировать)
    IF EXISTS (SELECT 1 FROM "Stage" s WHERE s."id_frame" = p_frame_id) THEN
        RETURN;
    END IF;

    -- создаём этапы по справочнику StageType для этого кадра
    INSERT INTO "Stage"
    ("id_project","id_episode","id_frame",
    "id_responsible_employee","stage_type_id","id_stage_status",
    "start_datetime","planned_end_date")
    SELECT
        f."id_project",
        f."id_episode",
        f."id_frame",
        NULL,

```

```
        stt."stage_type_id",
        v_planned_status,
        NOW(),
        f."planned_due_date"
FROM "Frame" f
JOIN "StageType" stt ON TRUE
WHERE f."id_frame" = p_frame_id;

END;
$;$
```

Приложение Г

Полный код разработанного ПО

Г1) Файл employee.py

```
import psycpg2
from flask import Blueprint, render_template, redirect, url_for, flash, request

from db import get_conn
from tokens import make_token, parse_token

bp = Blueprint("employee", __name__, url_prefix="/employees")

def fetch_all(sql, params=None):
    with get_conn() as conn, conn.cursor() as cur:
        cur.execute(sql, params or {})
        return cur.fetchall()

def fetch_one(sql, params=None):
    with get_conn() as conn, conn.cursor() as cur:
        cur.execute(sql, params or {})
        return cur.fetchone()

def execute(sql, params=None):
    with get_conn() as conn, conn.cursor() as cur:
        cur.execute(sql, params or {})

def _employees_query_with_specialization(where_clause=""):
    return f'''
        SELECT
            e."id_employee",
            e."surname",
            e."name",
            e."patronymic",
            e."email",
            e."phone",
            e."birth_date",
            sp."specialization_name" AS specialization
        FROM "Employees" e
        LEFT JOIN "Specialization" sp
            ON sp."id_specialization" = e."id_specialization"
        {where_clause}
        ORDER BY e."surname", e."name", e."patronymic";
    '''

def _employees_query_with_specialization_column(where_clause=""):
    return f'''
        SELECT
            e."id_employee",
            e."surname",
            e."name",
            e."patronymic",
            e."email",
            e."phone",
            e."birth_date",
            e."specialization" AS specialization
        FROM "Employees" e
        {where_clause}
        ORDER BY e."surname", e."name", e."patronymic";
    '''

def _search_clause(search_term: str, specialization_expr: str):
    if not search_term:
```

```

        return "", {}
    return (
        f'''
        WHERE (
            e."surname" ILIKE %(q)s
            OR e."name" ILIKE %(q)s
            OR COALESCE(e."patronymic", '') ILIKE %(q)s
            OR (e."surname" || ' ' || e."name" || ' ' || COALESCE(e."patronymic", ''))
        ILIKE %(q)s
            OR COALESCE(e."email", '') ILIKE %(q)s
            OR COALESCE(e."phone", '') ILIKE %(q)s
            OR COALESCE({specialization_expr}, '') ILIKE %(q)s
        )
        ''',
        {"q": f"%{search_term}%"},
    )

def _fetch_employees(search_term: str):
    where_clause, params = _search_clause(search_term, 'sp."specialization_name"')
    try:
        return fetch_all(_employees_query_with_specialization(where_clause), params)
    except psycopg2.Error:
        where_clause, params = _search_clause(search_term, 'e."specialization"')
        try:
            return
        fetch_all(_employees_query_with_specialization_column(where_clause), params)
    except psycopg2.Error:
        where_clause, params = _search_clause(search_term, '')
        rows = fetch_all(f'''
            SELECT
                e."id_employee",
                e."surname",
                e."name",
                e."patronymic",
                e."email",
                e."phone",
                e."birth_date"
            FROM "Employees" e
            {where_clause}
            ORDER BY e."surname", e."name", e."patronymic";
        ''', params)
        for r in rows:
            r["specialization"] = None
        return rows

def _fetch_employee(id_employee: int):
    try:
        return fetch_one(_employees_query_with_specialization('WHERE e."id_employee" =
%(id_employee)s'), {"id_employee": id_employee})
    except psycopg2.Error:
        try:
            return fetch_one(_employees_query_with_specialization_column('WHERE
e."id_employee" = %(id_employee)s'), {"id_employee": id_employee})
        except psycopg2.Error:
            row = fetch_one(f'''
                SELECT
                    e."id_employee",
                    e."surname",
                    e."name",
                    e."patronymic",
                    e."email",
                    e."phone",
                    e."birth_date"
                FROM "Employees" e
                WHERE e."id_employee" = %(id_employee)s;
            ''', {"id_employee": id_employee})
            if row:
                row["specialization"] = None

```

```

        return row

def _attach_token(row):
    row["token"] = make_token("employee", row["id_employee"])
    return row

def _fetch_specializations():
    try:
        return fetch_all('''
            SELECT "id_spesialization", "specialization_name"
            FROM "Spesialization"
            ORDER BY "specialization_name";
        ''')
    except psycopg2.Error:
        return []

@bp.get("/")
def list_employees():
    search = request.args.get("q", "").strip()
    rows = [_attach_token(r) for r in _fetch_employees(search)]
    return render_template("employees_list.html", rows=rows, q=search)

@bp.get("/new")
def new_employee_form():
    specializations = _fetch_specializations()
    return render_template("employees_form.html", specializations=specializations)

@bp.post("/new")
def create_employee():
    data = {
        "surname": request.form.get("surname", "").strip(),
        "name": request.form.get("name", "").strip(),
        "patronymic": request.form.get("patronymic", "").strip() or None,
        "email": request.form.get("email", "").strip() or None,
        "phone": request.form.get("phone", "").strip() or None,
        "birth_date": request.form.get("birth_date") or None,
        "id_spesialization": request.form.get("id_spesialization") or None,
        "specialization": request.form.get("specialization", "").strip() or None,
    }

    if not data["surname"] or not data["name"]:
        flash("Фамилия и имя обязательны.", "error")
        return redirect(url_for("employee.new_employee_form"))

    try:
        execute('''
            INSERT INTO "Employees"

("surname", "name", "patronymic", "email", "phone", "birth_date", "id_spesialization")
VALUES

(%(surname)s, %(name)s, %(patronymic)s, %(email)s, %(phone)s, %(birth_date)s, %(id_spesializ
ation)s);
        ''', data)
    except psycopg2.Error:
        execute('''
            INSERT INTO "Employees"

("surname", "name", "patronymic", "email", "phone", "birth_date", "specialization")
VALUES

(%(surname)s, %(name)s, %(patronymic)s, %(email)s, %(phone)s, %(birth_date)s, %(specializati
on)s);
        ''', data)

```

```

flash("Сотрудник добавлен.", "ok")
return redirect(url_for("employee.list_employees"))

@bp.get("/<token>")
def view_employee(token: str):
    try:
        id_employee = parse_token(token, "employee")
    except ValueError:
        flash("Неверная ссылка на сотрудника.", "error")
        return redirect(url_for("employee.list_employees"))

    emp = _fetch_employee(id_employee)
    if not emp:
        flash("Сотрудник не найден.", "error")
        return redirect(url_for("employee.list_employees"))

    load_rows = fetch_all('''
        SELECT
            s."id_stage",
            st."stage_type_name",
            ss."stage_status_name",
            s."planned_end_date",
            s."start_datetime",
            s."actual_end_datetime",
            COALESCE(p."project_name", '') AS project_name,
            COALESCE(ep."episode_name", '') AS episode_name,
            fr."frame_number_in_episode" AS frame_no,
            (
                SELECT dp."progress_percent"
                FROM "DailyProgress" dp
                WHERE dp."id_stage" = s."id_stage"
                ORDER BY dp."progress_date" DESC, dp."id_progress" DESC
                LIMIT 1
            ) AS last_progress_percent
        FROM "Stage" s
        JOIN "StageType" st ON st."stage_type_id" = s."stage_type_id"
        JOIN "StageStatus" ss ON ss."id_stage_status" = s."id_stage_status"
        LEFT JOIN "Project" p ON p."id_project" = s."id_project"
        LEFT JOIN "Episode" ep ON ep."id_episode" = s."id_episode"
        LEFT JOIN "Frame" fr ON fr."id_frame" = s."id_frame"
        WHERE s."id_responsible_employee" = %(eid)s
        ORDER BY
            ss."stage_status_name" IN ('Принят','Завершён') ASC,
            s."planned_end_date" NULLS LAST,
            s."id_stage" DESC;
    ''', {"eid": id_employee})

    summary = fetch_one('''
        SELECT
            COUNT(*) AS total,
            COUNT(*) FILTER (WHERE ss."stage_status_name" NOT IN ('Принят','Завершён'))
        AS active,
            COUNT(*) FILTER (WHERE ss."stage_status_name" = 'Требуется доработки') AS
        needs_rework
        FROM "Stage" s
        JOIN "StageStatus" ss ON ss."id_stage_status" = s."id_stage_status"
        WHERE s."id_responsible_employee" = %(eid)s;
    ''', {"eid": id_employee})

    specializations = _fetch_specializations()
    return render_template(
        "employees_view.html",
        emp=emp,
        token=token,
        load_rows=load_rows,
        summary=summary,
        specializations=specializations,
    )

```

```

@bp.post("/<token>/edit")
def update_employee(token: str):
    try:
        id_employee = parse_token(token, "employee")
    except ValueError:
        flash("Неверная ссылка на сотрудника.", "error")
        return redirect(url_for("employee.list_employees"))

    data = {
        "id_employee": id_employee,
        "surname": request.form.get("surname", "").strip(),
        "name": request.form.get("name", "").strip(),
        "patronymic": request.form.get("patronymic", "").strip() or None,
        "email": request.form.get("email", "").strip() or None,
        "phone": request.form.get("phone", "").strip() or None,
        "birth_date": request.form.get("birth_date") or None,
        "id_specialization": request.form.get("id_specialization") or None,
        "specialization": request.form.get("specialization", "").strip() or None,
    }

    if not data["surname"] or not data["name"]:
        flash("Фамилия и имя обязательны.", "error")
        return redirect(url_for("employee.view_employee", token=token))

    try:
        execute('''
            UPDATE "Employees"
            SET "surname"=%(surname)s,
                "name"=%(name)s,
                "patronymic"=%(patronymic)s,
                "email"=%(email)s,
                "phone"=%(phone)s,
                "birth_date"=%(birth_date)s,
                "id_specialization"=%(id_specialization)s
            WHERE "id_employee"=%(id_employee)s;
        ''', data)
    except psycopg2.Error:
        execute('''
            UPDATE "Employees"
            SET "surname"=%(surname)s,
                "name"=%(name)s,
                "patronymic"=%(patronymic)s,
                "email"=%(email)s,
                "phone"=%(phone)s,
                "birth_date"=%(birth_date)s,
                "specialization"=%(specialization)s
            WHERE "id_employee"=%(id_employee)s;
        ''', data)

    flash("Данные сотрудника обновлены.", "ok")
    return redirect(url_for("employee.view_employee", token=token))

@bp.post("/<token>/delete")
def delete_employee(token: str):
    try:
        id_employee = parse_token(token, "employee")
    except ValueError:
        flash("Неверная ссылка на сотрудника.", "error")
        return redirect(url_for("employee.list_employees"))

    try:
        with get_conn() as conn, conn.cursor() as cur:
            cur.execute('''
                UPDATE "Stage"
                SET "id_responsible_employee" = NULL
                WHERE "id_responsible_employee" = %(eid)s;
            ''')

```

```

        '', {"eid": id_employee})
    cur.execute(''
        DELETE FROM "DailyProgress" WHERE "id_employee" = %(eid)s;
        '', {"eid": id_employee})
    cur.execute(''
        DELETE FROM "StageCheck" WHERE "id_checker_employee" = %(eid)s;
        '', {"eid": id_employee})
    cur.execute(''
        DELETE FROM "Employees" WHERE "id_employee" = %(eid)s;
        '', {"eid": id_employee})
    flash("Сотрудник удалён.", "ok")
except Exception as err:
    flash(f"Ошибка при удалении сотрудника: {err}", "error")

return redirect(url_for("employee.list_employees"))

```

Г2) Файл episode.py

```

from flask import Blueprint, render_template, request, redirect, url_for, flash
from db import get_conn
from tokens import make_token, parse_token
from routes.role_rules import validate_stage_responsible, allowed_specializations,
normalize_value

bp = Blueprint("episode", __name__, url_prefix="/episodes")

def ensure_episode_stages(id_episode: int, planned_release_date=None) -> bool:
    """Создаёт этапы (Stage) для эпизода, если их нет. Возвращает True, если этапы
    добавлены."""
    cnt = fetch_one('SELECT COUNT(*)::int AS c FROM "Stage" WHERE "id_episode"=%(eid)s
    AND "id_frame" IS NULL;', {"eid": id_episode})
    if cnt and cnt.get("c", 0) > 0:
        return False

    emp = fetch_one(''
        SELECT "id_employee"
        FROM "Employees"
        WHERE COALESCE("is_active", FALSE) = TRUE
        ORDER BY "id_employee"
        LIMIT 1;
        '')
    if not emp:
        emp = fetch_one('SELECT "id_employee" FROM "Employees" ORDER BY "id_employee"
    LIMIT 1;')
    if not emp:
        return False

    st = fetch_one(''
        SELECT "id_stage_status"
        FROM "StageStatus"
        WHERE "stage_status_name" = 'Не начат'
        LIMIT 1;
        '')
    if not st:
        st = fetch_one('SELECT "id_stage_status" FROM "StageStatus" ORDER BY
    "id_stage_status" LIMIT 1;')
    if not st:
        return False

    execute(''
        INSERT INTO "Stage"
        ("id_project", "id_episode", "id_frame",
        "id_responsible_employee", "stage_type_id", "id_stage_status",
        "start_datetime", "actual_end_datetime", "planned_end_date")
        SELECT
        NULL, %(eid)s, NULL,
        %(emp)s, stt."stage_type_id", %(sid)s,

```



```

        NULL, NULL, %(planned)s
    FROM "StageType" stt
    WHERE LOWER(COALESCE(stt."stage_level", '')) = 'episode'
    ORDER BY stt."execution_order" NULLS LAST, stt."stage_type_id";
    '', {"eid": id_episode, "emp": emp["id_employee"], "sid": st["id_stage_status"],
    "planned": planned_release_date})
    return True

def fetch_all(sql, params=None):
    with get_conn() as conn, conn.cursor() as cur:
        cur.execute(sql, params or {})
        return cur.fetchall()

def fetch_one(sql, params=None):
    with get_conn() as conn, conn.cursor() as cur:
        cur.execute(sql, params or {})
        return cur.fetchone()

def execute(sql, params=None):
    with get_conn() as conn, conn.cursor() as cur:
        cur.execute(sql, params or {})

@bp.get("/<episode_token>")
def view_episode(episode_token: str):
    try:
        id_episode = parse_token(episode_token, "episode")
    except ValueError:
        flash("Неверная ссылка на эпизод.", "error")
        return redirect(url_for("project.list_projects"))

    ep = fetch_one('''
        SELECT
            e.*,
            es."episode_status_name",
            p."project_name"
        FROM "Episode" e
        LEFT JOIN "EpisodeStatus" es ON es."id_episode_status" = e."id_episode_status"
        JOIN "Project" p ON p."id_project" = e."id_project"
        WHERE e."id_episode" = %(eid)s;
    ''', {"eid": id_episode})

    if not ep:
        flash("Эпизод не найден.", "error")
        return redirect(url_for("project.list_projects"))

    project_token = make_token("project", ep["id_project"])

    # Если этапов эпизода нет - создаём автоматически
    ensure_episode_stages(id_episode, ep.get("planned_release_date"))

    frames = fetch_all('''
        SELECT
            f."id_frame",
            f."frame_number_in_episode",
            f."planned_due_date",
            f."actual_done_date"
        FROM "Frame" f
        WHERE f."id_episode" = %(eid)s
        ORDER BY f."frame_number_in_episode";
    ''', {"eid": id_episode})

    for fr in frames:
        fr["token"] = make_token("frame", fr["id_frame"])

    pub = fetch_all('''
        SELECT
            ps."id_publication_schedule",
            pstat."publication_status_name",

```

```

        ps."planned_publication_date",
        ps."actual_publication_date"
    FROM "PublicationSchedule" ps
    JOIN "PublicationStatus" pstat ON pstat."id_publication_status" =
ps."id_publication_status"
    WHERE ps."id_episode" = %(eid)s
    ORDER BY ps."planned_publication_date", ps."id_publication_schedule";
    '', {"eid": id_episode})

episode_stages = fetch_all('''
    SELECT
        s."id_stage",
        s."id_responsible_employee",
        st."stage_type_name",
        COALESCE(ss."stage_status_name", '-') AS stage_status_name,
        s."planned_end_date",
        COALESCE((emp."surname" || ' ' || emp."name"), '-') AS responsible,
        (
            SELECT dp."progress_percent"
            FROM "DailyProgress" dp
            WHERE dp."id_stage" = s."id_stage"
            ORDER BY dp."progress_date" DESC, dp."id_progress" DESC
            LIMIT 1
        ) AS last_progress_percent
    FROM "Stage" s
    JOIN "StageType" st ON st."stage_type_id" = s."stage_type_id"
    LEFT JOIN "StageStatus" ss ON ss."id_stage_status" = s."id_stage_status"
    LEFT JOIN "Employees" emp ON emp."id_employee" = s."id_responsible_employee"
    WHERE s."id_episode" = %(eid)s
    ORDER BY st."execution_order", s."id_stage";
    '', {"eid": id_episode})

for s in episode_stages:
    s["token"] = make_token("stage", s["id_stage"])

    statuses = fetch_all('SELECT "id_episode_status","episode_status_name" FROM
"EpisodeStatus" ORDER BY 1;')

try:
    employees = fetch_all('''
        SELECT
            e."id_employee",
            (e."surname" || ' ' || e."name") AS fio,
            sp."specialization_name" AS specialization
        FROM "Employees" e
        LEFT JOIN "Specialization" sp
            ON sp."id_specialization" = e."id_specialization"
        WHERE COALESCE(e."is_active", TRUE) = TRUE
        ORDER BY e."surname","name";
    ''')
except Exception:
    employees = fetch_all('''
        SELECT
            e."id_employee",
            (e."surname" || ' ' || e."name") AS fio,
            NULL AS specialization
        FROM "Employees" e
        WHERE COALESCE(e."is_active", TRUE) = TRUE
        ORDER BY e."surname","name";
    ''')

employees_by_stage = {}
for s in episode_stages:
    allowed = allowed_specializations(s.get("stage_type_name"))
    if allowed:
        employees_by_stage[s["id_stage"]] = [
            e for e in employees if normalize_value(e.get("specialization")) in
allowed
    ]

```

```

        else:
            employees_by_stage[s["id_stage"]] = employees

    stage_statuses = fetch_all('SELECT "id_stage_status","stage_status_name" FROM
"StageStatus" ORDER BY 1;')

    return render_template(
        "episodes_view.html",
        ep=ep,
        episode_token=episode_token,
        project_token=project_token,
        frames=frames,
        pub=pub,
        episode_stages=episode_stages,
        employees=employees,
        employees_by_stage=employees_by_stage,
        stage_statuses=stage_statuses,
        statuses=statuses
    )

@bp.post("/<episode_token>/stages/<stage_token>/responsible")
def update_episode_stage_responsible(episode_token: str, stage_token: str):
    try:
        id_episode = parse_token(episode_token, "episode")
        id_stage = parse_token(stage_token, "stage")
    except ValueError:
        flash("Неверная ссылка.", "error")
        return redirect(url_for("project.list_projects"))

    employee_id = request.form.get("id_responsible_employee") or None
    if employee_id:
        ok, error = validate_stage_responsible(id_stage, int(employee_id))
        if not ok:
            flash(error, "error")
            return redirect(url_for("episode.view_episode",
episode_token=episode_token))
        execute('''
            UPDATE "Stage"
            SET "id_responsible_employee" = %(eid)s
            WHERE "id_stage" = %(id_stage)s AND "id_episode" = %(eid2)s;
        ''', {"eid": employee_id, "id_stage": id_stage, "eid2": id_episode})

    flash("Ответственный назначен.", "ok")
    return redirect(url_for("episode.view_episode", episode_token=episode_token))

@bp.post("/<episode_token>/stages/<stage_token>/status")
def update_episode_stage_status(episode_token: str, stage_token: str):
    """Обновление статуса этапа эпизода + фиксация прогресса в DailyProgress.
    Это нужно, чтобы прогресс в интерфейсе менялся автоматически после переключения
    статуса.
    """
    try:
        id_episode = parse_token(episode_token, "episode")
        id_stage = parse_token(stage_token, "stage")
    except ValueError:
        flash("Неверная ссылка.", "error")
        return redirect(url_for("project.list_projects"))

    id_stage_status = request.form.get("id_stage_status") or None

    # Маппинг статуса -> процент прогресса (можно подстроить под твою шкалу)
    def status_to_progress(name: str | None) -> int:
        if not name:
            return 0
        n = name.lower()

```

```

        if "не нач" in n:
            return 0
        if "в работ" in n:
            return 50
        if "требует" in n or "доработ" in n:
            return 70
        if "заверш" in n or "готов" in n:
            return 100
        return 0

    try:
        # обновляем статус
        execute('''
            UPDATE "Stage"
            SET "id_stage_status" = %(sid)s
            WHERE "id_stage" = %(st)s AND "id_episode" = %(ep)s;
        ''', {"sid": id_stage_status, "st": id_stage, "ep": id_episode})

        # определяем имя статуса, чтобы посчитать процент
        ss = fetch_one('''
            SELECT "stage_status_name"
            FROM "StageStatus"
            WHERE "id_stage_status" = %(sid)s;
        ''', {"sid": id_stage_status})
        pct = status_to_progress(ss["stage_status_name"] if ss else None)

        # фиксируем прогресс на сегодня (новая запись)
        execute('''
            INSERT INTO "DailyProgress"
            ("id_employee", "id_stage", "progress_date", "progress_percent")
            VALUES (
                (SELECT "id_responsible_employee" FROM "Stage" WHERE
                "id_stage"=%(st)s),
                %(st)s,
                CURRENT_DATE,
                %(pct)s
            );
        ''', {"st": id_stage, "pct": pct})

        flash(f"Статус этапа обновлён, прогресс: {pct}%", "ok")
    except Exception as e:
        flash(f"Ошибка при обновлении статуса этапа: {e}", "error")

    return redirect(url_for("episode.view_episode", episode_token=episode_token))
@bp.post("/<episode_token>/status")
def update_episode_status(episode_token: str):
    try:
        id_episode = parse_token(episode_token, "episode")
    except ValueError:
        flash("Неверная ссылка на эпизод.", "error")
        return redirect(url_for("project.list_projects"))

    new_status = request.form.get("id_episode_status") or None
    try:
        execute('''
            UPDATE "Episode"
            SET "id_episode_status" = %(sid)s
            WHERE "id_episode" = %(eid)s;
        ''', {"sid": new_status, "eid": id_episode})
        flash("Статус эпизода обновлён. (Триггеры БД выполнены)", "ok")
    except Exception as e:
        flash(f"Ошибка при обновлении статуса: {e}", "error")

    return redirect(url_for("episode.view_episode", episode_token=episode_token))

@bp.get("/new/<project_token>")
def new_episode_form(project_token: str):
    try:

```

```

        id_project = parse_token(project_token, "project")
    except ValueError:
        flash("Неверная ссылка на проект.", "error")
        return redirect(url_for("project.list_projects"))

    project = fetch_one('SELECT "id_project","project_name" FROM "Project" WHERE
"id_project"=%(pid)s;', {"pid": id_project})
    if not project:
        flash("Проект не найден.", "error")
        return redirect(url_for("project.list_projects"))

    statuses = fetch_all('SELECT "id_episode_status", "episode_status_name" FROM
"EpisodeStatus" ORDER BY 1;')
    return render_template("episodes_form.html", mode="new", project=project,
project_token=project_token, statuses=statuses, ep=None, episode_token=None)

@bp.post("/new/<project_token>")
def create_episode(project_token: str):
    try:
        id_project = parse_token(project_token, "project")
    except ValueError:
        flash("Неверная ссылка на проект.", "error")
        return redirect(url_for("project.list_projects"))

    data = {
        "id_project": id_project,
        "episode_name": request.form.get("episode_name","").strip(),
        "episode_number_in_project": request.form.get("episode_number_in_project") or
None,
        "planned_release_date": request.form.get("planned_release_date") or None,
        "actual_release_date": request.form.get("actual_release_date") or None,
        "ready_for_release_flag": True if request.form.get("ready_for_release_flag")
== "on" else False,
        "ready_for_release_date": request.form.get("ready_for_release_date") or None,
        "id_episode_status": request.form.get("id_episode_status") or None,
    }

    if not data["episode_name"]:
        flash("Поле «Название эпизода» обязательно.", "error")
        return redirect(url_for("episode.new_episode_form",
project_token=project_token))

    execute('''
        INSERT INTO "Episode"

("episode_name","episode_number_in_project","planned_release_date","id_project",

"actual_release_date","ready_for_release_flag","ready_for_release_date","id_episode_st
atus")

VALUES

(%(episode_name)s,%(episode_number_in_project)s,%(planned_release_date)s,%(id_project)
s,

%(actual_release_date)s,%(ready_for_release_flag)s,%(ready_for_release_date)s,%(id_epi
sode_status)s);
''', data)

    flash("Эпизод добавлен.", "ok")
    return redirect(url_for("project.view_project", token=project_token))

@bp.get("/<episode_token>/edit")
def edit_episode_form(episode_token: str):
    try:
        id_episode = parse_token(episode_token, "episode")
    except ValueError:
        flash("Неверная ссылка на эпизод.", "error")
        return redirect(url_for("project.list_projects"))

```

```

    ep = fetch_one('SELECT * FROM "Episode" WHERE "id_episode"=%(eid)s;', {"eid":
id_episode}))
    if not ep:
        flash("Эпизод не найден.", "error")
        return redirect(url_for("project.list_projects"))

    project = fetch_one('SELECT "id_project","project_name" FROM "Project" WHERE
"id_project"=%(pid)s;', {"pid": ep["id_project"]})
    project_token = make_token("project", ep["id_project"])
    statuses = fetch_all('SELECT "id_episode_status", "episode_status_name" FROM
"EpisodeStatus" ORDER BY 1;')
    return render_template("episodes_form.html", mode="edit", project=project,
project_token=project_token, statuses=statuses, ep=ep, episode_token=episode_token)

@bp.post("/<episode_token>/edit")
def update_episode(episode_token: str):
    try:
        id_episode = parse_token(episode_token, "episode")
    except ValueError:
        flash("Неверная ссылка на эпизод.", "error")
        return redirect(url_for("project.list_projects"))

    # Get project id to redirect back
    cur_ep = fetch_one('SELECT "id_project" FROM "Episode" WHERE
"id_episode"=%(eid)s;', {"eid": id_episode})
    if not cur_ep:
        flash("Эпизод не найден.", "error")
        return redirect(url_for("project.list_projects"))
    project_token = make_token("project", cur_ep["id_project"])

    data = {
        "id_episode": id_episode,
        "episode_name": request.form.get("episode_name","").strip(),
        "episode_number_in_project": request.form.get("episode_number_in_project") or
None,
        "planned_release_date": request.form.get("planned_release_date") or None,
        "actual_release_date": request.form.get("actual_release_date") or None,
        "ready_for_release_flag": True if request.form.get("ready_for_release_flag")
== "on" else False,
        "ready_for_release_date": request.form.get("ready_for_release_date") or None,
        "id_episode_status": request.form.get("id_episode_status") or None,
    }

    if not data["episode_name"]:
        flash("Поле «Название эпизода» обязательно.", "error")
        return redirect(url_for("episode.edit_episode_form",
episode_token=episode_token))

    execute('''
UPDATE "Episode"
SET "episode_name"=%(episode_name)s,
    "episode_number_in_project"=%(episode_number_in_project)s,
    "planned_release_date"=%(planned_release_date)s,
    "actual_release_date"=%(actual_release_date)s,
    "ready_for_release_flag"=%(ready_for_release_flag)s,
    "ready_for_release_date"=%(ready_for_release_date)s,
    "id_episode_status"=%(id_episode_status)s
WHERE "id_episode"=%(id_episode)s;
''', data)

    flash("Эпизод обновлён.", "ok")
    return redirect(url_for("episode.view_episode", episode_token=episode_token))

@bp.post("/<episode_token>/delete")
def delete_episode(episode_token: str):
    try:
        id_episode = parse_token(episode_token, "episode")

```

```

except ValueError:
    flash("Неверная ссылка на эпизод.", "error")
    return redirect(url_for("project.list_projects"))

    ep = fetch_one('SELECT "id_project" FROM "Episode" WHERE "id_episode"=%(eid)s;',
{"eid": id_episode})
    if not ep:
        flash("Эпизод не найден.", "error")
        return redirect(url_for("project.list_projects"))
    project_token = make_token("project", ep["id_project"])

    try:
        with get_conn() as conn, conn.cursor() as cur:
            cur.execute('''
                DELETE FROM "StageStatusHistory"
                WHERE "id_stage" IN (
                    SELECT s."id_stage"
                    FROM "Stage" s
                    LEFT JOIN "Frame" f ON f."id_frame" = s."id_frame"
                    WHERE s."id_episode" = %(eid)s
                        OR f."id_episode" = %(eid)s
                );
            ''', {"eid": id_episode})
            cur.execute('''
                DELETE FROM "StageCheck"
                WHERE "id_stage" IN (
                    SELECT s."id_stage"
                    FROM "Stage" s
                    LEFT JOIN "Frame" f ON f."id_frame" = s."id_frame"
                    WHERE s."id_episode" = %(eid)s
                        OR f."id_episode" = %(eid)s
                );
            ''', {"eid": id_episode})
            cur.execute('''
                DELETE FROM "DailyProgress"
                WHERE "id_stage" IN (
                    SELECT s."id_stage"
                    FROM "Stage" s
                    LEFT JOIN "Frame" f ON f."id_frame" = s."id_frame"
                    WHERE s."id_episode" = %(eid)s
                        OR f."id_episode" = %(eid)s
                );
            ''', {"eid": id_episode})
            cur.execute('''
                DELETE FROM "Stage"
                WHERE "id_episode" = %(eid)s
                OR "id_frame" IN (
                    SELECT f."id_frame" FROM "Frame" f WHERE f."id_episode" =
%(eid)s
                );
            ''', {"eid": id_episode})
            cur.execute('''
                DELETE FROM "PublicationSchedule" WHERE "id_episode" = %(eid)s;
            ''', {"eid": id_episode})
            cur.execute('''
                DELETE FROM "Frame" WHERE "id_episode" = %(eid)s;
            ''', {"eid": id_episode})
            cur.execute('''
                DELETE FROM "Episode" WHERE "id_episode"=%(eid)s;
            ''', {"eid": id_episode})
            flash("Эпизод удалён.", "ok")
        except Exception as err:
            flash(f"Ошибка при удалении эпизода: {err}", "error")
        return redirect(url_for("project.view_project", token=project_token))

@bp.post("/<episode_token>/reschedule")
def reschedule_publication(episode_token: str):
    try:

```

```

        id_episode = parse_token(episode_token, "episode")
    except ValueError:
        flash("Неверная ссылка на эпизод.", "error")
        return redirect(url_for("project.list_projects"))

    new_date = request.form.get("new_planned_date") or None
    if not new_date:
        flash("Выберите новую плановую дату.", "error")
        return redirect(url_for("episode.view_episode", episode_token=episode_token))

    try:
        execute('CALL "sp_reschedule_publication"(%(eid)s, %(d)s::date);', {"eid":
id_episode, "d": new_date})
        flash("Публикация перенесена (вызвана процедура БД).", "ok")
    except Exception as e:
        flash(f"Ошибка при вызове процедуры: {e}", "error")

    return redirect(url_for("episode.view_episode", episode_token=episode_token))

```

Г3) Файл frame.py

```

from flask import Blueprint, render_template, request, redirect, url_for, flash
from db import get_conn
from tokens import make_token, parse_token
from routes.role_rules import validate_stage_responsible, allowed_specializations,
normalize_value

bp = Blueprint("frame", __name__, url_prefix="/frames")

def ensure_frame_stages(id_frame: int, planned_end_date=None) -> bool:
    """Создаёт этапы (Stage) для кадра, если их нет. Возвращает True, если этапы
    добавлены."""
    cnt = fetch_one('SELECT COUNT(*)::int AS c FROM "Stage" WHERE
"id_frame"=%(fid)s;', {"fid": id_frame})
    if cnt and cnt.get("c", 0) > 0:
        return False

    # 1) попытка вызвать процедуру БД (если существует)
    try:
        execute('CALL "sp_create_default_stages_for_frame"(%(fid)s);', {"fid":
id_frame})
        return True
    except Exception:
        pass

    # 2) фолбэк: создаём этапы через INSERT..SELECT
    emp = fetch_one('''
        SELECT "id_employee"
        FROM "Employees"
        WHERE COALESCE("is_active", FALSE) = TRUE
        ORDER BY "id_employee"
        LIMIT 1;
    ''')
    if not emp:
        emp = fetch_one('SELECT "id_employee" FROM "Employees" ORDER BY "id_employee"
LIMIT 1;')
    if not emp:
        return False

    st = fetch_one('''
        SELECT "id_stage_status"
        FROM "StageStatus"
        WHERE "stage_status_name" = 'Не начат'
        LIMIT 1;
    ''')
    if not st:
        st = fetch_one('SELECT "id_stage_status" FROM "StageStatus" ORDER BY
"id_stage_status" LIMIT 1;')

```



```

    if not st:
        return False

    execute('''WITH st_src AS (
        SELECT stt."stage_type_id", stt."execution_order"
        FROM "StageType" stt
        WHERE LOWER(COALESCE(stt."stage_level", '')) = 'frame'
    ),
    st_src2 AS (
        SELECT stt."stage_type_id", stt."execution_order"
        FROM "StageType" stt
        WHERE LOWER(COALESCE(stt."stage_level", '')) = 'episode'
    )
    INSERT INTO "Stage"
    ("id_project", "id_episode", "id_frame",
    "id_responsible_employee", "stage_type_id", "id_stage_status",
    "start_datetime", "actual_end_datetime", "planned_end_date")
    SELECT
        NULL, NULL, %(fid)s,
        %(emp)s, x."stage_type_id", %(sid)s,
        NULL, NULL, %(planned)s
    FROM (
        SELECT * FROM st_src
        UNION ALL
        SELECT * FROM st_src2
        WHERE NOT EXISTS (SELECT 1 FROM st_src)
    ) x
    ORDER BY x."execution_order" NULLS LAST, x."stage_type_id";''', {"fid": id_frame,
    "emp": emp["id_employee"], "sid": st["id_stage_status"], "planned": planned_end_date})

    return True

def fetch_all(sql, params=None):
    with get_conn() as conn, conn.cursor() as cur:
        cur.execute(sql, params or {})
        return cur.fetchall()

def fetch_one(sql, params=None):
    with get_conn() as conn, conn.cursor() as cur:
        cur.execute(sql, params or {})
        return cur.fetchone()

def execute(sql, params=None):
    with get_conn() as conn, conn.cursor() as cur:
        cur.execute(sql, params or {})

@bp.get("/<frame_token>")
def view_frame(frame_token: str):
    try:
        id_frame = parse_token(frame_token, "frame")
    except ValueError:
        flash("Неверная ссылка на кадр.", "error")
        return redirect(url_for("project.list_projects"))

    fr = fetch_one('''
        SELECT
            f.*,
            e."episode_name",
            e."episode_number_in_project",
            p."project_name",
            e."id_episode",
            p."id_project"
        FROM "Frame" f
        JOIN "Episode" e ON e."id_episode" = f."id_episode"
        JOIN "Project" p ON p."id_project" = e."id_project"
        WHERE f."id_frame" = %(fid)s;
    ''', {"fid": id_frame})

```

```

if not fr:
    flash("Кадр не найден.", "error")
    return redirect(url_for("project.list_projects"))

project_token = make_token("project", fr["id_project"])
episode_token = make_token("episode", fr["id_episode"])

# stages for this frame (to demonstrate procedures/functions/triggers from DB)
stages = fetch_all('''
    SELECT
        s."id_stage",
        s."id_responsible_employee",
        st."stage_type_name",
        COALESCE(ss."stage_status_name", '-') AS stage_status_name,
        s."planned_end_date",
        s."start_datetime",
        s."actual_end_datetime",
        COALESCE((emp."surname" || ' ' || emp."name"), '-') AS responsible,
        (
            SELECT dp."progress_percent"
            FROM "DailyProgress" dp
            WHERE dp."id_stage" = s."id_stage"
            ORDER BY dp."progress_date" DESC, dp."id_progress" DESC
            LIMIT 1
        ) AS last_progress_percent
    FROM "Stage" s
    JOIN "StageType" st ON st."stage_type_id" = s."stage_type_id"
    LEFT JOIN "StageStatus" ss ON ss."id_stage_status" = s."id_stage_status"
    LEFT JOIN "Employees" emp ON emp."id_employee" = s."id_responsible_employee"
    WHERE s."id_frame" = %(fid)s
    ORDER BY st."execution_order", s."id_stage";
''', {"fid": id_frame})

# Если этапов у кадра нет (например, кадр был создан до внедрения автосоздания) -
создаём их автоматически
if not stages:
    created = ensure_frame_stages(id_frame, fr.get("planned_due_date"))
    if created:
        stages = fetch_all('''
            SELECT
                s."id_stage",
                s."id_responsible_employee",
                st."stage_type_name",
                COALESCE(ss."stage_status_name", '-') AS stage_status_name,
                s."planned_end_date",
                s."start_datetime",
                s."actual_end_datetime",
                COALESCE((emp."surname" || ' ' || emp."name"), '-') AS
responsible,
                (
                    SELECT dp."progress_percent"
                    FROM "DailyProgress" dp
                    WHERE dp."id_stage" = s."id_stage"
                    ORDER BY dp."progress_date" DESC, dp."id_progress" DESC
                    LIMIT 1
                ) AS last_progress_percent
            FROM "Stage" s
            JOIN "StageType" st ON st."stage_type_id" = s."stage_type_id"
            LEFT JOIN "StageStatus" ss ON ss."id_stage_status" =
s."id_stage_status"
            LEFT JOIN "Employees" emp ON emp."id_employee" =
s."id_responsible_employee"
            WHERE s."id_frame" = %(fid)s
            ORDER BY st."execution_order", s."id_stage";
''', {"fid": id_frame})

for s in stages:
    s["token"] = make_token("stage", s["id_stage"])

```

```

        stage_statuses = fetch_all('SELECT "id_stage_status","stage_status_name" FROM
"StageStatus" ORDER BY "id_stage_status";')
        check_results = fetch_all('SELECT "id_check_result","check_result_name" FROM
"CheckResult" ORDER BY "id_check_result";')
    try:
        checkers = fetch_all('''
            SELECT
                e."id_employee",
                (e."surname" || ' ' || e."name") AS fio,
                sp."specialization_name" AS specialization
            FROM "Employees" e
            LEFT JOIN "Spesialization" sp
                ON sp."id_spesialization" = e."id_spesialization"
            WHERE COALESCE(e."is_active", TRUE) = TRUE
            ORDER BY e."surname","name";
        ''')
    except Exception:
        checkers = fetch_all('''
            SELECT
                e."id_employee",
                (e."surname" || ' ' || e."name") AS fio,
                NULL AS specialization
            FROM "Employees" e
            WHERE COALESCE(e."is_active", TRUE) = TRUE
            ORDER BY e."surname","name";
        ''')

    employees_by_stage = {}
    for s in stages:
        allowed = allowed_specializations(s.get("stage_type_name"))
        if allowed:
            employees_by_stage[s["id_stage"]] = [
                e for e in checkers if normalize_value(e.get("specialization")) in
allowed
            ]
        else:
            employees_by_stage[s["id_stage"]] = checkers

    return render_template(
        "frames_view.html",
        fr=fr,
        frame_token=frame_token,
        project_token=project_token,
        episode_token=episode_token,
        stages=stages,
        stage_statuses=stage_statuses,
        check_results=check_results,
        checkers=checkers,
        employees_by_stage=employees_by_stage,
        employees=checkers
    )

@bp.get("/new/<episode_token>")
def new_frame_form(episode_token: str):
    try:
        id_episode = parse_token(episode_token, "episode")
    except ValueError:
        flash("Неверная ссылка на эпизод.", "error")
        return redirect(url_for("project.list_projects"))

    ep = fetch_one('''
        SELECT e."id_episode", e."episode_name", e."episode_number_in_project",
            p."id_project", p."project_name"
        FROM "Episode" e
        JOIN "Project" p ON p."id_project" = e."id_project"
        WHERE e."id_episode"=%(eid)s;
    ''', {"eid": id_episode})

```

```

if not ep:
    flash("Эпизод не найден.", "error")
    return redirect(url_for("project.list_projects"))

    return render_template("frames_form.html", mode="new", ep=ep,
episode_token=episode_token, fr=None, frame_token=None)

@bp.post("/new/<episode_token>")
def create_frame(episode_token: str):
    try:
        id_episode = parse_token(episode_token, "episode")
    except ValueError:
        flash("Неверная ссылка на эпизод.", "error")
        return redirect(url_for("project.list_projects"))

    data = {
        "id_episode": id_episode,
        "frame_number_in_episode": request.form.get("frame_number_in_episode") or
None,
        "planned_due_date": request.form.get("planned_due_date") or None,
        "actual_done_date": request.form.get("actual_done_date") or None,
    }

    if not data["frame_number_in_episode"]:
        flash("Поле «Номер кадра в эпизоде» обязательно.", "error")
        return redirect(url_for("frame.new_frame_form", episode_token=episode_token))

    row = fetch_one('''
        INSERT INTO "Frame"
        ("id_episode", "frame_number_in_episode", "planned_due_date", "actual_done_date")
        VALUES

        (%(id_episode)s, %(frame_number_in_episode)s, %(planned_due_date)s, %(actual_done_date)s)
        RETURNING "id_frame";
        ''', data)

    fid = row.get("id_frame") if row else None

    if fid:
        ensure_frame_stages(fid, data.get("planned_due_date"))
        # Автосоздание этапов кадра (Stage), чтобы таблица Stage не была пустой
        if fid:
            cnt = fetch_one('SELECT COUNT(*)::int AS c FROM "Stage" WHERE
            "id_frame"=%(fid)s;', {"fid": fid})
            if cnt and cnt.get("c", 0) == 0:
                created = False

                # 1) попытка вызвать процедуру БД (если она существует)
                try:
                    execute('CALL "sp_create_default_stages_for_frame"(%(fid)s);', {"fid":
fid})

                    created = True
                except Exception:
                    created = False

                # 2) фолбэк: создаём этапы через INSERT..SELECT
                if not created:
                    try:
                        emp = fetch_one('''
                            SELECT "id_employee"
                            FROM "Employees"
                            WHERE COALESCE("is_active", FALSE) = TRUE
                            ORDER BY "id_employee"
                            LIMIT 1;
                        ''')
                        if not emp:
                            emp = fetch_one('SELECT "id_employee" FROM "Employees" ORDER
BY "id_employee" LIMIT 1;')

```

```

        if not emp:
            flash("Кадр создан, но этапы не добавлены: таблица Employees
пуста.", "error")
            return redirect(url_for("episode.view_episode",
episode_token=episode_token))

        st = fetch_one('''
        SELECT "id_stage_status"
        FROM "StageStatus"
        WHERE "stage_status_name" = 'Не начат'
        LIMIT 1;
        ''')
        if not st:
            st = fetch_one('SELECT "id_stage_status" FROM "StageStatus"
ORDER BY "id_stage_status" LIMIT 1;')

        if not st:
            flash("Кадр создан, но этапы не добавлены: таблица StageStatus
пуста.", "error")
            return redirect(url_for("episode.view_episode",
episode_token=episode_token))

        execute('''
        WITH st_src AS (
            SELECT stt."stage_type_id", stt."execution_order"
            FROM "StageType" stt
            WHERE LOWER(COALESCE(stt."stage_level", '')) = 'frame'
        ),
        st_src2 AS (
            SELECT stt."stage_type_id", stt."execution_order"
            FROM "StageType" stt
            WHERE LOWER(COALESCE(stt."stage_level", '')) = 'episode'
        )
        INSERT INTO "Stage"
        ("id_project", "id_episode", "id_frame",
        "id_responsible_employee", "stage_type_id", "id_stage_status",
        "start_datetime", "actual_end_datetime", "planned_end_date")
        SELECT
            NULL, NULL, %(fid)s,
            %(emp)s, x."stage_type_id", %(sid)s,
            NULL, NULL, %(planned)s
        FROM (
            SELECT * FROM st_src
            UNION ALL
            SELECT * FROM st_src2
            WHERE NOT EXISTS (SELECT 1 FROM st_src)
        ) x
        ORDER BY x."execution_order" NULLS LAST, x."stage_type_id";
        ''', {"fid": fid, "emp": emp["id_employee"], "sid":
st["id_stage_status"], "planned": data["planned_due_date"]})

        created = True
        except Exception as e:
            flash(f"Кадр создан, но этапы не добавлены: {e}", "error")

        if created:
            flash("Кадр добавлен, этапы созданы автоматически.", "ok")
        else:
            flash("Кадр добавлен, но этапы не создались (проверь
StageType/StageStatus/Employees).", "error")
        else:
            flash("Кадр добавлен.", "ok")
    else:
        flash("Кадр добавлен.", "ok")

    return redirect(url_for("episode.view_episode", episode_token=episode_token))
@bp.get("/<frame_token>/edit")
def edit_frame_form(frame_token: str):

```

```

try:
    id_frame = parse_token(frame_token, "frame")
except ValueError:
    flash("Неверная ссылка на кадр.", "error")
    return redirect(url_for("project.list_projects"))

fr = fetch_one('SELECT * FROM "Frame" WHERE "id_frame"=%(fid)s;', {"fid":
id_frame})
if not fr:
    flash("Кадр не найден.", "error")
    return redirect(url_for("project.list_projects"))

episode_token = make_token("episode", fr["id_episode"])
ep = fetch_one('''
    SELECT e."id_episode", e."episode_name", e."episode_number_in_project",
        p."id_project", p."project_name"
    FROM "Episode" e
    JOIN "Project" p ON p."id_project" = e."id_project"
    WHERE e."id_episode"=%(eid)s;
''', {"eid": fr["id_episode"]})

return render_template("frames_form.html", mode="edit", ep=ep,
episode_token=episode_token, fr=fr, frame_token=frame_token)

@bp.post("/<frame_token>/edit")
def update_frame(frame_token: str):
    try:
        id_frame = parse_token(frame_token, "frame")
    except ValueError:
        flash("Неверная ссылка на кадр.", "error")
        return redirect(url_for("project.list_projects"))

    current = fetch_one('SELECT "id_episode" FROM "Frame" WHERE "id_frame"=%(fid)s;',
{"fid": id_frame})
    if not current:
        flash("Кадр не найден.", "error")
        return redirect(url_for("project.list_projects"))

    episode_token = make_token("episode", current["id_episode"])

    data = {
        "id_frame": id_frame,
        "frame_number_in_episode": request.form.get("frame_number_in_episode") or
None,
        "planned_due_date": request.form.get("planned_due_date") or None,
        "actual_done_date": request.form.get("actual_done_date") or None,
    }

    if not data["frame_number_in_episode"]:
        flash("Поле «Номер кадра в эпизоде» обязательно.", "error")
        return redirect(url_for("frame.edit_frame_form", frame_token=frame_token))

    execute('''
        UPDATE "Frame"
        SET "frame_number_in_episode"=%(frame_number_in_episode)s,
            "planned_due_date"=%(planned_due_date)s,
            "actual_done_date"=%(actual_done_date)s
        WHERE "id_frame"=%(id_frame)s;
    ''', data)

    flash("Кадр обновлён.", "ok")
    return redirect(url_for("frame.view_frame", frame_token=frame_token))

@bp.post("/<frame_token>/delete")
def delete_frame(frame_token: str):
    try:
        id_frame = parse_token(frame_token, "frame")
    except ValueError:
        flash("Неверная ссылка на кадр.", "error")

```

```

        return redirect(url_for("project.list_projects"))

    fr = fetch_one('SELECT "id_episode" FROM "Frame" WHERE "id_frame"=%(fid)s;',
{"fid": id_frame})
    if not fr:
        flash("Кадр не найден.", "error")
        return redirect(url_for("project.list_projects"))

    episode_token = make_token("episode", fr["id_episode"])
    execute('''
        DELETE FROM "StageCheck"
        WHERE "id_stage" IN (
            SELECT s."id_stage"
            FROM "Stage" s
            WHERE s."id_frame" = %(fid)s
        );
    ''', {"fid": id_frame})
    execute('''
        DELETE FROM "DailyProgress"
        WHERE "id_stage" IN (
            SELECT s."id_stage"
            FROM "Stage" s
            WHERE s."id_frame" = %(fid)s
        );
    ''', {"fid": id_frame})
    execute('DELETE FROM "Stage" WHERE "id_frame"=%(fid)s;', {"fid": id_frame})
    execute('DELETE FROM "Frame" WHERE "id_frame"=%(fid)s;', {"fid": id_frame})
    flash("Кадр удалён.", "ok")
    return redirect(url_for("episode.view_episode", episode_token=episode_token))

@bp.post("/<frame_token>/stages/<stage_token>/status")
def update_stage_status(frame_token: str, stage_token: str):
    try:
        id_frame = parse_token(frame_token, "frame")
        id_stage = parse_token(stage_token, "stage")
    except ValueError:
        flash("Неверная ссылка.", "error")
        return redirect(url_for("project.list_projects"))

    id_stage_status = request.form.get("id_stage_status")
    if not id_stage_status:
        flash("Не выбран статус.", "error")
        return redirect(url_for("frame.view_frame", frame_token=frame_token))

    try:
        execute('''
            UPDATE "Stage"
            SET "id_stage_status" = %(sid)s
            WHERE "id_stage" = %(id_stage)s AND "id_frame" = %(id_frame)s;
        ''', {"sid": id_stage_status, "id_stage": id_stage, "id_frame": id_frame})

        ss = fetch_one('''
            SELECT "stage_status_name"
            FROM "StageStatus"
            WHERE "id_stage_status" = %(sid)s;
        ''', {"sid": id_stage_status})
        name = ss["stage_status_name"] if ss else None
        if name:
            n = name.lower()
            if "не нач" in n:
                pct = 0
            elif "в работ" in n:
                pct = 50
            elif "требуется" in n or "доработ" in n:
                pct = 70
            elif "заверш" in n or "готов" in n:
                pct = 100
            else:

```

```

        pct = 0
    else:
        pct = 0

    execute('''
        INSERT INTO "DailyProgress"
        ("id_employee","id_stage","progress_date","progress_percent")
        VALUES (
            (SELECT "id_responsible_employee" FROM "Stage" WHERE
            "id_stage"=%(st)s),
            %(st)s,
            CURRENT_DATE,
            %(pct)s
        );
    ''', {"st": id_stage, "pct": pct})

    flash(f"Статус этапа обновлён, прогресс: {pct}%", "ok")
except Exception as e:
    flash(f"Ошибка при обновлении статуса: {e}", "error")

return redirect(url_for("frame.view_frame", frame_token=frame_token))

@bp.post("/<frame_token>/stages/<stage_token>/responsible")
def update_stage_responsible(frame_token: str, stage_token: str):
    try:
        id_frame = parse_token(frame_token, "frame")
        id_stage = parse_token(stage_token, "stage")
    except ValueError:
        flash("Неверная ссылка.", "error")
        return redirect(url_for("project.list_projects"))

    employee_id = request.form.get("id_responsible_employee") or None
    if employee_id:
        ok, error = validate_stage_responsible(id_stage, int(employee_id))
        if not ok:
            flash(error, "error")
            return redirect(url_for("frame.view_frame", frame_token=frame_token))
    execute('''
        UPDATE "Stage"
        SET "id_responsible_employee" = %(eid)s
        WHERE "id_stage" = %(id_stage)s AND "id_frame" = %(id_frame)s;
    ''', {"eid": employee_id, "id_stage": id_stage, "id_frame": id_frame})

    flash("Ответственный назначен.", "ok")
    return redirect(url_for("frame.view_frame", frame_token=frame_token))

@bp.post("/<frame_token>/stages/<stage_token>/check")
def add_stage_check(frame_token: str, stage_token: str):
    try:
        id_frame = parse_token(frame_token, "frame")
        id_stage = parse_token(stage_token, "stage")
    except ValueError:
        flash("Неверная ссылка.", "error")
        return redirect(url_for("project.list_projects"))

    checker = request.form.get("id_checker_employee")
    result = request.form.get("id_check_result")
    comment = (request.form.get("comment") or "").strip() or None

    if not checker or not result:
        flash("Для проверки нужно выбрать проверяющего и результат.", "error")
        return redirect(url_for("frame.view_frame", frame_token=frame_token))

    try:
        execute('''
            INSERT INTO "StageCheck"

```



```

("id_checker_employee","id_stage","id_check_result","comment","check_datetime")
VALUES
    (%(checker)s,%(stage)s,%(res)s,%(comment)s,NOW());
'', {"checker": checker, "stage": id_stage, "res": result, "comment":
comment})
    flash("Проверка сохранена. (AFTER-триггер мог обновить эпизод и график
публикаций)", "ok")
    except Exception as e:
        flash(f"Ошибка при сохранении проверки: {e}", "error")

    return redirect(url_for("frame.view_frame", frame_token=frame_token))

```

Г4) Файл main.py

```

from flask import Blueprint, redirect, url_for

bp = Blueprint("main", __name__)

@bp.get("/")
def index():
    return redirect(url_for("project.list_projects"))

```

Г5) Файл project.py

```

from datetime import date, timedelta

from flask import Blueprint, render_template, request, redirect, url_for, flash
from db import get_conn
from tokens import make_token, parse_token
from routes.role_rules import validate_stage_responsible, allowed_specializations,
normalize_value

bp = Blueprint("project", __name__, url_prefix="/projects")

def ensure_project_stages(id_project: int) -> bool:
    """Создаёт этапы (Stage) для проекта, если их нет. Возвращает True, если этапы
    добавлены."""
    cnt = fetch_one('SELECT COUNT(*)::int AS c FROM "Stage" WHERE "id_project"=%(pid)s
AND "id_episode" IS NULL AND "id_frame" IS NULL;', {"pid": id_project})
    if cnt and cnt.get("c", 0) > 0:
        return False

    emp = fetch_one('''
        SELECT "id_employee"
        FROM "Employees"
        WHERE COALESCE("is_active", FALSE) = TRUE
        ORDER BY "id_employee"
        LIMIT 1;
    ''')
    if not emp:
        emp = fetch_one('SELECT "id_employee" FROM "Employees" ORDER BY "id_employee"
LIMIT 1;')
    if not emp:
        return False

    st = fetch_one('''
        SELECT "id_stage_status"
        FROM "StageStatus"
        WHERE "stage_status_name" = 'Не начал'
        LIMIT 1;
    ''')
    if not st:
        st = fetch_one('SELECT "id_stage_status" FROM "StageStatus" ORDER BY
"id_stage_status" LIMIT 1;')
    if not st:

```

```

        return False

    planned = fetch_one('SELECT "planned_end_date" FROM "Project" WHERE
        "id_project"=%(pid)s;', {"pid": id_project})

    execute('''
        INSERT INTO "Stage"
        ("id_project","id_episode","id_frame",
        "id_responsible_employee","stage_type_id","id_stage_status",
        "start_datetime","actual_end_datetime","planned_end_date")
        SELECT
        %(pid)s, NULL, NULL,
        %(emp)s, stt."stage_type_id", %(sid)s,
        NULL, NULL, %(planned)s
        FROM "StageType" stt
        WHERE LOWER(COALESCE(stt."stage_level", '')) = 'project'
        ORDER BY stt."execution_order" NULLS LAST, stt."stage_type_id";
        ''', {"pid": id_project, "emp": emp["id_employee"], "sid": st["id_stage_status"],
        "planned": (planned or {}).get("planned_end_date")})
    return True

# --- Helpers ---
def fetch_all(sql, params=None):
    with get_conn() as conn, conn.cursor() as cur:
        cur.execute(sql, params or {})
        return cur.fetchall()

def fetch_one(sql, params=None):
    with get_conn() as conn, conn.cursor() as cur:
        cur.execute(sql, params or {})
        return cur.fetchone()

def execute(sql, params=None):
    with get_conn() as conn, conn.cursor() as cur:
        cur.execute(sql, params or {})

def _token_for_row(row):
    row["token"] = make_token("project", row["id_project"])
    return row

def _progress_project(id_project: int):
    """Возвращает прогресс проекта (общий и на сегодня).
    Прогресс считается как средний процент по всем этапам
    (включая уровни проекта, эпизодов и кадров).
    Ежедневный прогресс - разница прогресса на сегодня и вчера.
    """
    def _overall_as_of(as_of_date: date) -> float:
        row = fetch_one('''
            WITH stage_project AS (
                SELECT
                    s."id_stage",
                    s."id_stage_status",
                    COALESCE(s."id_project", e."id_project", e2."id_project") AS
id_project
            FROM "Stage" s
            LEFT JOIN "Episode" e ON e."id_episode" = s."id_episode"
            LEFT JOIN "Frame" f ON f."id_frame" = s."id_frame"
            LEFT JOIN "Episode" e2 ON e2."id_episode" = f."id_episode"
            WHERE s."id_project" = %(pid)s
                OR e."id_project" = %(pid)s
                OR e2."id_project" = %(pid)s
            ),
            last_progress AS (
                SELECT
                    dp."id_stage",
                    dp."progress_percent",
                    ROW_NUMBER() OVER (PARTITION BY dp."id_stage" ORDER BY
dp."progress_date" DESC, dp."id_progress" DESC) AS rn
        ''')

```

```

FROM "DailyProgress" dp
JOIN stage_project sp ON sp."id_stage" = dp."id_stage"
WHERE dp."progress_date" <= %(as_of)s
),
stage_p AS (
SELECT
    sp."id_stage",
    COALESCE(
        lp."progress_percent"::numeric,
        CASE
            WHEN ss."stage_status_name" ILIKE '%%не начат%%' THEN 0
            WHEN ss."stage_status_name" ILIKE '%%в работе%%' THEN 50
            WHEN ss."stage_status_name" ILIKE '%%доработ%%' THEN 70
            WHEN ss."stage_status_name" ILIKE '%%провер%%' THEN 90
            WHEN ss."stage_status_name" ILIKE '%%заверш%%'
              OR ss."stage_status_name" ILIKE '%%принят%%' THEN 100
            ELSE 0
        END::numeric
    ) AS p
FROM stage_project sp
LEFT JOIN last_progress lp ON lp."id_stage" = sp."id_stage" AND lp.rn =
1
    LEFT JOIN "StageStatus" ss ON ss."id_stage_status" =
sp."id_stage_status"
)
SELECT COALESCE(ROUND(AVG(p), 2), 0) AS overall_percent
FROM stage_p;
'', {"pid": id_project, "as_of": as_of_date})
return float(row["overall_percent"]) if row and row["overall_percent"] is not
None else 0.0

overall = _overall_as_of(date.today())

daily_row = fetch_one(''
WITH stage_project AS (
SELECT s."id_stage"
FROM "Stage" s
LEFT JOIN "Episode" e ON e."id_episode" = s."id_episode"
LEFT JOIN "Frame" f ON f."id_frame" = s."id_frame"
LEFT JOIN "Episode" e2 ON e2."id_episode" = f."id_episode"
WHERE s."id_project" = %(pid)s
    OR e."id_project" = %(pid)s
    OR e2."id_project" = %(pid)s
),
today_dp AS (
SELECT
    dp."id_stage",
    dp."progress_percent",
    ROW_NUMBER() OVER (PARTITION BY dp."id_stage" ORDER BY dp."progress_date"
DESC, dp."id_progress" DESC) AS rn
FROM "DailyProgress" dp
JOIN stage_project sp ON sp."id_stage" = dp."id_stage"
WHERE dp."progress_date" = CURRENT_DATE
)
SELECT
    COUNT(*) AS total_stages,
    COALESCE(SUM(COALESCE(t.progress_percent, 0)), 0) AS today_sum
FROM stage_project sp
LEFT JOIN today_dp t ON t."id_stage" = sp."id_stage" AND t.rn = 1;
'', {"pid": id_project})

total = float(daily_row["total_stages"]) if daily_row and
daily_row["total_stages"] is not None else 0.0
today_sum = float(daily_row["today_sum"]) if daily_row and daily_row["today_sum"]
is not None else 0.0
today = round((today_sum / total), 2) if total > 0 else 0.0

return {"overall": round(overall, 2), "today": today}

```

```

@bp.get("/")
def list_projects():
    rows = fetch_all('''
        SELECT
            p."id_project",
            p."project_name",
            p."start_date",
            p."planned_end_date",
            p."actual_end_date",
            ps."project_status_name"
        FROM "Project" p
        LEFT JOIN "ProjectStatus" ps
            ON ps."id_project_status" = p."id_project_status"
        ORDER BY p."id_project" DESC;
    ''')
    # attach tokens + progress
    out = []
    for r in rows:
        r = _token_for_row(r)
        pr = _progress_project(r["id_project"])
        r["progress_overall"] = pr["overall"]
        r["progress_today"] = pr["today"]
        out.append(r)
    return render_template("projects_list.html", rows=out)

@bp.get("/new")
def new_project_form():
    statuses = fetch_all('SELECT "id_project_status", "project_status_name" FROM
"ProjectStatus" ORDER BY 1;')
    return render_template("projects_form.html", mode="new", statuses=statuses,
item=None, token=None)

@bp.post("/new")
def create_project():
    data = {
        "project_name": request.form.get("project_name", "").strip(),
        "description": request.form.get("description", "").strip() or None,
        "start_date": request.form.get("start_date") or None,
        "planned_end_date": request.form.get("planned_end_date") or None,
        "actual_end_date": request.form.get("actual_end_date") or None,
        "id_project_status": request.form.get("id_project_status") or None,
    }

    if not data["project_name"]:
        flash("Поле «Название проекта» обязательно.", "error")
        return redirect(url_for("project.new_project_form"))

    execute('''
        INSERT INTO "Project"

("project_name","description","start_date","planned_end_date","actual_end_date","id_pr
oject_status")
        VALUES

(%(project_name)s,%(description)s,%(start_date)s,%(planned_end_date)s,%(actual_end_dat
e)s,%(id_project_status)s);
    ''', data)

    flash("Проект создан.", "ok")
    return redirect(url_for("project.list_projects"))

@bp.get("/<token>")
def view_project(token: str):
    try:
        id_project = parse_token(token, "project")
    except ValueError:
        flash("Неверная ссылка на проект.", "error")
        return redirect(url_for("project.list_projects"))

```

```

item = fetch_one('''
    SELECT
        p.*,
        ps."project_status_name"
    FROM "Project" p
    LEFT JOIN "ProjectStatus" ps
        ON ps."id_project_status" = p."id_project_status"
    WHERE p."id_project" = %(id_project)s;
''', {"id_project": id_project})
# Если этапов проекта нет - создаём автоматически
ensure_project_stages(id_project)

if not item:
    flash("Проект не найден.", "error")
    return redirect(url_for("project.list_projects"))

progress = _progress_project(id_project)

statuses = fetch_all('SELECT "id_project_status","project_status_name" FROM
"ProjectStatus" ORDER BY 1;')

episodes = fetch_all('''
    SELECT
        e."id_episode",
        e."episode_number_in_project",
        e."episode_name",
        e."planned_release_date",
        e."actual_release_date",
        e."ready_for_release_flag",
        es."episode_status_name"
    FROM "Episode" e
    LEFT JOIN "EpisodeStatus" es ON es."id_episode_status" = e."id_episode_status"
    WHERE e."id_project" = %(pid)s
    ORDER BY e."episode_number_in_project";
''', {"pid": id_project})

for e in episodes:
    e["token"] = make_token("episode", e["id_episode"])

item_token = token # keep same token for URLs/forms
project_stages = fetch_all('''
    SELECT
        s."id_stage",
        s."id_responsible_employee",
        st."stage_type_name",
        COALESCE(ss."stage_status_name", '-') AS stage_status_name,
        s."planned_end_date",
        COALESCE((emp."surname" || ' ' || emp."name"), '-') AS responsible,
        (
            SELECT dp."progress_percent"
            FROM "DailyProgress" dp
            WHERE dp."id_stage" = s."id_stage"
            ORDER BY dp."progress_date" DESC, dp."id_progress" DESC
            LIMIT 1
        ) AS last_progress_percent
    FROM "Stage" s
    JOIN "StageType" st ON st."stage_type_id" = s."stage_type_id"
    LEFT JOIN "StageStatus" ss ON ss."id_stage_status" = s."id_stage_status"
    LEFT JOIN "Employees" emp ON emp."id_employee" = s."id_responsible_employee"
    WHERE s."id_project" = %(pid)s
    ORDER BY st."execution_order", s."id_stage";
''', {"pid": id_project})

for s in project_stages:
    s["token"] = make_token("stage", s["id_stage"])

try:
    employees = fetch_all(''''

```

```

        SELECT
            e."id_employee",
            (e."surname" || ' ' || e."name") AS fio,
            sp."specialization_name" AS specialization
        FROM "Employees" e
        LEFT JOIN "Specialization" sp
            ON sp."id_specialization" = e."id_specialization"
        WHERE COALESCE(e."is_active", TRUE) = TRUE
        ORDER BY e."surname", "name";
    '')
except Exception:
    employees = fetch_all('''
        SELECT
            e."id_employee",
            (e."surname" || ' ' || e."name") AS fio,
            NULL AS specialization
        FROM "Employees" e
        WHERE COALESCE(e."is_active", TRUE) = TRUE
        ORDER BY e."surname", "name";
    ''')

employees_by_stage = {}
for s in project_stages:
    allowed = allowed_specializations(s.get("stage_type_name"))
    if allowed:
        employees_by_stage[s["id_stage"]] = [
            e for e in employees if normalize_value(e.get("specialization")) in
allowed
        ]
    else:
        employees_by_stage[s["id_stage"]] = employees

# Этапы проекта/эпизодов/кадров для отображения на странице проекта (все привязаны по
id_project)
stage_statuses = fetch_all('SELECT "id_stage_status", "stage_status_name" FROM
"StageStatus" ORDER BY 1;')

stages_all = fetch_all('''
    SELECT
        s."id_stage",
        s."id_episode",
        s."id_frame",
        stt."stage_type_name" AS stage_type_name,
        ss."stage_status_name" AS stage_status_name,
        (e."surname" || ' ' || e."name") AS responsible,
        COALESCE(dp_last."progress_percent", CASE
            WHEN ss."stage_status_name" ILIKE '%%не начат%%' THEN 0
            WHEN ss."stage_status_name" ILIKE '%%в работе%%' THEN 50
            WHEN ss."stage_status_name" ILIKE '%%доработ%%' THEN 70
            WHEN ss."stage_status_name" ILIKE '%%провер%%' THEN 90
            WHEN ss."stage_status_name" ILIKE '%%заверш%%' OR ss."stage_status_name"
ILIKE '%%готов%%' OR ss."stage_status_name" ILIKE '%%принят%%' THEN 100
            ELSE 0
        END) AS last_progress_percent,
        s."planned_end_date"
    FROM "Stage" s
    JOIN "StageType" stt ON stt."stage_type_id" = s."stage_type_id"
    LEFT JOIN "StageStatus" ss ON ss."id_stage_status" = s."id_stage_status"
    LEFT JOIN "Employees" e ON e."id_employee" = s."id_responsible_employee"
    LEFT JOIN LATERAL (
        SELECT dp."progress_percent"
        FROM "DailyProgress" dp
        WHERE dp."id_stage" = s."id_stage"
        ORDER BY dp."progress_date" DESC, dp."id_progress" DESC
        LIMIT 1
    ) dp_last ON TRUE
    WHERE s."id_project" = %(pid)s
    ORDER BY
        (CASE WHEN s."id_episode" IS NULL AND s."id_frame" IS NULL THEN 1
            WHEN s."id_episode" IS NOT NULL AND s."id_frame" IS NULL THEN 2

```

```

        ELSE 3 END),
        stt."execution_order" NULLS LAST, s."id_stage";
    '', {"pid": id_project})

for s in stages_all:
    s["token"] = make_token("stage", s["id_stage"])

episode_stages = [s for s in stages_all if s.get("id_episode") and not
s.get("id_frame")]
frame_stages = [s for s in stages_all if s.get("id_frame")]

return render_template(
    "projects_view.html",
    item=item,
    token=item_token,
    progress=progress,
    episodes=episodes,
    project_stages=project_stages,
    employees=employees,
    employees_by_stage=employees_by_stage,
    stage_statuses=stage_statuses,
    stages_all=stages_all,
    episode_stages=episode_stages,
    frame_stages=frame_stages,
    statuses=statuses,
)

@bp.post("/stage/<stage_token>/status")
def update_project_stage_status(stage_token: str):
    try:
        id_stage = parse_token(stage_token, "stage")
    except ValueError:
        flash("Неверная ссылка на этап.", "error")
        return redirect(url_for("project.list_projects"))

    new_status = request.form.get("id_stage_status") or None

    try:
        execute('''
            UPDATE "Stage"
            SET "id_stage_status" = %(sid)s
            WHERE "id_stage" = %(stid)s;
        ''', {"sid": new_status, "stid": id_stage})

        execute('''
            INSERT INTO "DailyProgress"
            ("id_employee", "id_stage", "progress_date", "progress_percent")
            SELECT
                s."id_responsible_employee",
                s."id_stage",
                CURRENT_DATE,
                CASE
                    WHEN ss."stage_status_name" ILIKE '%%не начат%%' THEN 0
                    WHEN ss."stage_status_name" ILIKE '%%в работе%%' THEN 50
                    WHEN ss."stage_status_name" ILIKE '%%доработ%%' THEN 70
                    WHEN ss."stage_status_name" ILIKE '%%провер%%' THEN 90
                    WHEN ss."stage_status_name" ILIKE '%%заверш%%' OR ss."stage_status_name" ILIKE
                    '%%готов%%' OR ss."stage_status_name" ILIKE '%%принят%%' THEN 100
                ELSE 0
            END AS progress_percent
            FROM "Stage" s
            LEFT JOIN "StageStatus" ss ON ss."id_stage_status" = s."id_stage_status"
            WHERE s."id_stage" = %(stid)s;
        ''', {"stid": id_stage})

```

```

        flash("Статус этапа обновлён.", "ok")
    except Exception as err:
        flash(f"Ошибка при обновлении статуса этапа: {err}", "error")

    return redirect(request.form.get("next") or url_for("project.list_projects"))
@bp.post("/<token>/status")
def update_project_status(token: str):
    try:
        id_project = parse_token(token, "project")
    except ValueError:
        flash("Неверная ссылка на проект.", "error")
        return redirect(url_for("project.list_projects"))

    new_status = request.form.get("id_project_status") or None

    try:
        execute('''
            UPDATE "Project"
            SET "id_project_status" = %(sid)s
            WHERE "id_project" = %(pid)s;
        ''', {"sid": new_status, "pid": id_project})
        flash("Статус проекта обновлён. (Триггеры БД выполнены)", "ok")
    except Exception as e:
        flash(f"Ошибка при обновлении статуса: {e}", "error")

    return redirect(url_for("project.view_project", token=token))
@bp.post("/<token>/stages/<stage_token>/responsible")
def update_project_stage_responsible(token: str, stage_token: str):
    try:
        id_project = parse_token(token, "project")
        id_stage = parse_token(stage_token, "stage")
    except ValueError:
        flash("Неверная ссылка.", "error")
        return redirect(url_for("project.list_projects"))

    employee_id = request.form.get("id_responsible_employee") or None
    if employee_id:
        ok, error = validate_stage_responsible(id_stage, int(employee_id))
        if not ok:
            flash(error, "error")
            return redirect(url_for("project.view_project", token=token))
    execute('''
        UPDATE "Stage"
        SET "id_responsible_employee" = %(eid)s
        WHERE "id_stage" = %(id_stage)s AND "id_project" = %(pid)s;
    ''', {"eid": employee_id, "id_stage": id_stage, "pid": id_project})

    flash("Ответственный назначен.", "ok")
    return redirect(url_for("project.view_project", token=token))

@bp.get("/<token>/edit")
def edit_project_form(token: str):
    try:
        id_project = parse_token(token, "project")
    except ValueError:
        flash("Неверная ссылка на проект.", "error")
        return redirect(url_for("project.list_projects"))

    item = fetch_one('SELECT * FROM "Project" WHERE "id_project"=%(id_project)s;',
{"id_project": id_project})
    if not item:
        flash("Проект не найден.", "error")
        return redirect(url_for("project.list_projects"))

    statuses = fetch_all('SELECT "id_project_status", "project_status_name" FROM
"ProjectStatus" ORDER BY 1;')
    return render_template("projects_form.html", mode="edit", statuses=statuses,
item=item, token=token)

```



```

@bp.post("/<token>/edit")
def update_project(token: str):
    try:
        id_project = parse_token(token, "project")
    except ValueError:
        flash("Неверная ссылка на проект.", "error")
        return redirect(url_for("project.list_projects"))

    data = {
        "id_project": id_project,
        "project_name": request.form.get("project_name", "").strip(),
        "description": request.form.get("description", "").strip() or None,
        "start_date": request.form.get("start_date") or None,
        "planned_end_date": request.form.get("planned_end_date") or None,
        "actual_end_date": request.form.get("actual_end_date") or None,
        "id_project_status": request.form.get("id_project_status") or None,
    }

    if not data["project_name"]:
        flash("Поле «Название проекта» обязательно.", "error")
        return redirect(url_for("project.edit_project_form", token=token))

    execute('''
        UPDATE "Project"
        SET "project_name"=%(project_name)s,
            "description"=%(description)s,
            "start_date"=%(start_date)s,
            "planned_end_date"=%(planned_end_date)s,
            "actual_end_date"=%(actual_end_date)s,
            "id_project_status"=%(id_project_status)s
        WHERE "id_project"=%(id_project)s;
    ''', data)

    flash("Изменения сохранены.", "ok")
    return redirect(url_for("project.view_project", token=token))

@bp.post("/<token>/delete")
def delete_project(token: str):
    try:
        id_project = parse_token(token, "project")
    except ValueError:
        flash("Неверная ссылка на проект.", "error")
        return redirect(url_for("project.list_projects"))

    try:
        with get_conn() as conn, conn.cursor() as cur:
            cur.execute('''
                DELETE FROM "StageStatusHistory"
                WHERE "id_stage" IN (
                    SELECT s."id_stage"
                    FROM "Stage" s
                    LEFT JOIN "Episode" e ON e."id_episode" = s."id_episode"
                    LEFT JOIN "Frame" f ON f."id_frame" = s."id_frame"
                    LEFT JOIN "Episode" e2 ON e2."id_episode" = f."id_episode"
                    WHERE s."id_project" = %(pid)s
                        OR e."id_project" = %(pid)s
                        OR e2."id_project" = %(pid)s
                );
            ''', {"pid": id_project})
            cur.execute('''
                DELETE FROM "StageCheck"
                WHERE "id_stage" IN (
                    SELECT s."id_stage"
                    FROM "Stage" s
                    LEFT JOIN "Episode" e ON e."id_episode" = s."id_episode"
                    LEFT JOIN "Frame" f ON f."id_frame" = s."id_frame"
                    LEFT JOIN "Episode" e2 ON e2."id_episode" = f."id_episode"
                    WHERE s."id_project" = %(pid)s
                        OR e."id_project" = %(pid)s
            ''')

```

```

        OR e2."id_project" = %(pid)s
    );
    '', {"pid": id_project})
cur.execute('''
    DELETE FROM "DailyProgress"
    WHERE "id_stage" IN (
        SELECT s."id_stage"
        FROM "Stage" s
        LEFT JOIN "Episode" e ON e."id_episode" = s."id_episode"
        LEFT JOIN "Frame" f ON f."id_frame" = s."id_frame"
        LEFT JOIN "Episode" e2 ON e2."id_episode" = f."id_episode"
        WHERE s."id_project" = %(pid)s
            OR e."id_project" = %(pid)s
            OR e2."id_project" = %(pid)s
    );
    '', {"pid": id_project})
cur.execute('''
    DELETE FROM "Stage" s
    WHERE s."id_project" = %(pid)s
    OR s."id_episode" IN (
        SELECT e."id_episode" FROM "Episode" e WHERE e."id_project" =
%(pid)s
    )
    OR s."id_frame" IN (
        SELECT f."id_frame"
        FROM "Frame" f
        JOIN "Episode" e ON e."id_episode" = f."id_episode"
        WHERE e."id_project" = %(pid)s
    );
    '', {"pid": id_project})
cur.execute('''
    DELETE FROM "PublicationSchedule"
    WHERE "id_episode" IN (
        SELECT e."id_episode" FROM "Episode" e WHERE e."id_project" =
%(pid)s
    );
    '', {"pid": id_project})
cur.execute('''
    DELETE FROM "Frame"
    WHERE "id_episode" IN (
        SELECT e."id_episode" FROM "Episode" e WHERE e."id_project" =
%(pid)s
    );
    '', {"pid": id_project})
cur.execute('''
    DELETE FROM "Episode" WHERE "id_project" = %(pid)s;
    '', {"pid": id_project})
cur.execute('''
    DELETE FROM "Project" WHERE "id_project" = %(pid)s;
    '', {"pid": id_project})
flash("Проект удалён.", "ok")
except Exception as err:
    flash(f"Ошибка при удалении проекта: {err}", "error")
return redirect(url_for("project.list_projects"))

```

Г6) Файл publication.py

```

from flask import Blueprint, render_template, request, redirect, url_for, flash

from db import get_conn
from tokens import make_token, parse_token

bp = Blueprint("publication", __name__, url_prefix="/publications")

def fetch_all(sql, params=None):
    with get_conn() as conn, conn.cursor() as cur:
        cur.execute(sql, params or {})
        return cur.fetchall()

```

```

def fetch_one(sql, params=None):
    with get_conn() as conn, conn.cursor() as cur:
        cur.execute(sql, params or {})
        return cur.fetchone()

def execute(sql, params=None):
    with get_conn() as conn, conn.cursor() as cur:
        cur.execute(sql, params or {})

def _attach_tokens(row):
    row["token"] = make_token("publication", row["id_publication_schedule"])
    row["episode_token"] = make_token("episode", row["id_episode"])
    return row

@bp.get("")
def list_publications():
    q = request.args.get("q", "").strip()
    where_clause = ""
    params = {}
    if q:
        where_clause = 'WHERE p."project_name" ILIKE %(q)s'
        params["q"] = f"%{q}%"

    rows = fetch_all(f'''
        SELECT DISTINCT ON (ps."id_episode")
            ps."id_publication_schedule",
            ps."id_episode",
            p."project_name",
            e."episode_name",
            pst."publication_status_name",
            ps."planned_publication_date",
            ps."actual_publication_date"
        FROM "PublicationSchedule" ps
        JOIN "Episode" e ON e."id_episode" = ps."id_episode"
        JOIN "Project" p ON p."id_project" = e."id_project"
        LEFT JOIN "PublicationStatus" pst ON pst."id_publication_status" =
ps."id_publication_status"
        {where_clause}
        ORDER BY ps."id_episode", ps."id_publication_schedule" DESC;
    ''', params)

    rows = [_attach_tokens(r) for r in rows]
    return render_template("publications_list.html", rows=rows, q=q)

@bp.get("/history/<episode_token>")
def publication_history(episode_token: str):
    try:
        id_episode = parse_token(episode_token, "episode")
    except ValueError:
        flash("Неверная ссылка на эпизод.", "error")
        return redirect(url_for("publication.list_publications"))

    ep = fetch_one(f'''
        SELECT e."episode_name", p."project_name"
        FROM "Episode" e
        JOIN "Project" p ON p."id_project" = e."id_project"
        WHERE e."id_episode" = %(eid)s;
    ''', {"eid": id_episode})
    if not ep:
        flash("Эпизод не найден.", "error")
        return redirect(url_for("publication.list_publications"))

    rows = fetch_all(f'''

```

```

        SELECT
            ps."id_publication_schedule",
            pst."publication_status_name",
            ps."planned_publication_date",
            ps."actual_publication_date"
        FROM "PublicationSchedule" ps
        LEFT JOIN "PublicationStatus" pst ON pst."id_publication_status" =
ps."id_publication_status"
        WHERE ps."id_episode" = %(eid)s
        ORDER BY ps."id_publication_schedule" DESC;
        '', {"eid": id_episode}))

    return render_template(
        "publications_history.html",
        ep=ep,
        episode_token=episode_token,
        rows=rows,
    )

@bp.get("/new")
def new_publication_form():
    episodes = fetch_all(''
        SELECT
            e."id_episode",
            e."episode_name",
            e."episode_number_in_project",
            p."project_name"
        FROM "Episode" e
        JOIN "Project" p ON p."id_project" = e."id_project"
        ORDER BY p."project_name", e."episode_number_in_project", e."episode_name";
        '')
    statuses = fetch_all('SELECT "id_publication_status","publication_status_name"
FROM "PublicationStatus" ORDER BY 1;')
    return render_template("publications_form.html", mode="new", item=None,
token=None, episodes=episodes, statuses=statuses)

@bp.post("/new")
def create_publication():
    data = {
        "id_episode": request.form.get("id_episode") or None,
        "id_publication_status": request.form.get("id_publication_status") or None,
        "planned_publication_date": request.form.get("planned_publication_date") or
None,
        "actual_publication_date": request.form.get("actual_publication_date") or
None,
    }

    if not data["id_episode"]:
        flash("Выберите эпизод.", "error")
        return redirect(url_for("publication.new_publication_form"))

    execute(''
        INSERT INTO "PublicationSchedule"

("id_episode","id_publication_status","planned_publication_date","actual_publication_d
ate")

        VALUES

        (%(id_episode)s,%(id_publication_status)s,%(planned_publication_date)s,%(actual_public
ation_date)s);
        '', data)

    flash("Запись публикации добавлена.", "ok")
    return redirect(url_for("publication.list_publications"))

@bp.get("/<token>/edit")

```

```

def edit_publication_form(token: str):
    try:
        id_publication_schedule = parse_token(token, "publication")
    except ValueError:
        flash("Неверная ссылка на публикацию.", "error")
        return redirect(url_for("publication.list_publications"))

    item = fetch_one('SELECT * FROM "PublicationSchedule" WHERE
"id_publication_schedule" = %(pid)s;', {"pid": id_publication_schedule})
    if not item:
        flash("Запись публикации не найдена.", "error")
        return redirect(url_for("publication.list_publications"))

    episodes = fetch_all('''
        SELECT
            e."id_episode",
            e."episode_name",
            e."episode_number_in_project",
            p."project_name"
        FROM "Episode" e
        JOIN "Project" p ON p."id_project" = e."id_project"
        ORDER BY p."project_name", e."episode_number_in_project", e."episode_name";
    ''')
    statuses = fetch_all('SELECT "id_publication_status","publication_status_name"
FROM "PublicationStatus" ORDER BY 1;')
    return render_template("publications_form.html", mode="edit", item=item,
token=token, episodes=episodes, statuses=statuses)

@bp.post("/<token>/edit")
def update_publication(token: str):
    try:
        id_publication_schedule = parse_token(token, "publication")
    except ValueError:
        flash("Неверная ссылка на публикацию.", "error")
        return redirect(url_for("publication.list_publications"))

    data = {
        "id_publication_schedule": id_publication_schedule,
        "id_episode": request.form.get("id_episode") or None,
        "id_publication_status": request.form.get("id_publication_status") or None,
        "planned_publication_date": request.form.get("planned_publication_date") or
None,
        "actual_publication_date": request.form.get("actual_publication_date") or
None,
    }

    if not data["id_episode"]:
        flash("Выберите эпизод.", "error")
        return redirect(url_for("publication.edit_publication_form", token=token))

    execute('''
        UPDATE "PublicationSchedule"
        SET "id_episode"=%(id_episode)s,
            "id_publication_status"=%(id_publication_status)s,
            "planned_publication_date"=%(planned_publication_date)s,
            "actual_publication_date"=%(actual_publication_date)s
        WHERE "id_publication_schedule"=%(id_publication_schedule)s;
    ''', data)

    flash("Запись публикации обновлена.", "ok")
    return redirect(url_for("publication.list_publications"))

@bp.post("/<token>/delete")
def delete_publication(token: str):
    try:
        id_publication_schedule = parse_token(token, "publication")
    except ValueError:

```

```

        flash("Неверная ссылка на публикацию.", "error")
        return redirect(url_for("publication.list_publications"))

    execute('DELETE FROM "PublicationSchedule" WHERE
"id_publication_schedule"=%(pid)s;', {"pid": id_publication_schedule})
    flash("Запись публикации удалена.", "ok")
    return redirect(url_for("publication.list_publications"))

```

Г7) Файл report.py

```

from datetime import date

from flask import Blueprint, render_template, request

from db import get_conn

bp = Blueprint("report", __name__, url_prefix="/reports")

def fetch_all(sql, params=None):
    with get_conn() as conn, conn.cursor() as cur:
        if params is None:
            cur.execute(sql)
        else:
            cur.execute(sql, params)
    return cur.fetchall()

def _get_date_param(name: str, default: date) -> date:
    raw = (request.args.get(name) or "").strip()
    if not raw:
        return default
    try:
        return date.fromisoformat(raw)
    except ValueError:
        return default

def _get_month_param(name: str, default: date) -> date:
    raw = (request.args.get(name) or "").strip()
    if not raw:
        return date(default.year, default.month, 1)
    try:
        year_str, month_str = raw.split("-", 1)
        year = int(year_str)
        month = int(month_str)
        return date(year, month, 1)
    except (ValueError, TypeError):
        return date(default.year, default.month, 1)

def _month_value(value: date) -> str:
    return f"{value.year:04d}-{value.month:02d}"

@bp.get("/")
def reports():
    lag_days = int(request.args.get("lag_days", "10") or 10)

    today = date.today()
    date_a = _get_date_param("date_a", today)
    month_v = _get_month_param("month_v", today)
    month_g = _get_month_param("month_g", today)
    month_e = _get_month_param("month_e", today)

    month_start = month_v
    if month_v.month == 12:
        next_month = date(month_v.year + 1, 1, 1)

```

```

else:
    next_month = date(month_v.year, month_v.month + 1, 1)
if month_g.month == 12:
    month_g_next = date(month_g.year + 1, 1, 1)
else:
    month_g_next = date(month_g.year, month_g.month + 1, 1)
if month_e.month == 12:
    month_e_next = date(month_e.year + 1, 1, 1)
else:
    month_e_next = date(month_e.year, month_e.month + 1, 1)

# a) Progress per project (for selected day: sum of per-stage progress / total)
progress_rows = fetch_all('''
    WITH stage_project AS (
        SELECT
            s."id_stage",
            COALESCE(s."id_project", e."id_project", e2."id_project") AS
id_project
        FROM "Stage" s
        LEFT JOIN "Episode" e ON e."id_episode" = s."id_episode"
        LEFT JOIN "Frame" f ON f."id_frame" = s."id_frame"
        LEFT JOIN "Episode" e2 ON e2."id_episode" = f."id_episode"
    ),
    today_dp AS (
        SELECT
            dp."id_stage",
            dp."progress_percent",
            ROW_NUMBER() OVER (PARTITION BY dp."id_stage" ORDER BY
dp."progress_date" DESC, dp."id_progress" DESC) AS rn
        FROM "DailyProgress" dp
        JOIN stage_project sp ON sp."id_stage" = dp."id_stage"
        WHERE dp."progress_date" = %(as_of)s
    ),
    totals AS (
        SELECT
            sp.id_project,
            COUNT(*) AS total_stages,
            COALESCE(SUM(COALESCE(t.progress_percent, 0)), 0) AS today_sum
        FROM stage_project sp
        LEFT JOIN today_dp t ON t."id_stage" = sp."id_stage" AND t.rn = 1
        GROUP BY sp.id_project
    )
    SELECT
        p."id_project",
        p."project_name",
        CASE
            WHEN COALESCE(t.total_stages, 0) = 0 THEN 0
            ELSE ROUND((t.today_sum::numeric / t.total_stages::numeric), 2)
        END AS overall_percent
    FROM "Project" p
    LEFT JOIN totals t ON t.id_project = p."id_project"
    ORDER BY p."project_name";
''', {"as_of": date_a})

# b) Projects lagging more than N days
lag_rows = fetch_all('''
    SELECT
        p."project_name",
        p."planned_end_date",
        p."actual_end_date",
        (COALESCE(p."actual_end_date", CURRENT_DATE) - p."planned_end_date")::int
AS delay_days
    FROM "Project" p
    WHERE p."planned_end_date" IS NOT NULL
    AND (COALESCE(p."actual_end_date", CURRENT_DATE) - p."planned_end_date") >
%(lag_days)s
    ORDER BY delay_days DESC;
''', {"lag_days": lag_days})

# c) Frames completed by each artist in current month

```

```

frames_by_artist = fetch_all('''
    SELECT
        e."surname" || ' ' || e."name" AS artist,
        COUNT(DISTINCT s."id_frame") AS frames_done
    FROM "Stage" s
    JOIN "Employees" e ON e."id_employee" = s."id_responsible_employee"
    WHERE s."id_frame" IS NOT NULL
        AND s."actual_end_datetime" >= %(start)s
        AND s."actual_end_datetime" < %(end)s
    GROUP BY artist
    ORDER BY frames_done DESC, artist;
''', {"start": month_start, "end": next_month})

# d) Avg time for stage "композитинг" per project (selected month)
avg_paint_time = fetch_all('''
    WITH stage_project AS (
        SELECT
            s."id_stage",
            s."stage_type_id",
            s."start_datetime",
            s."actual_end_datetime",
            COALESCE(s."id_project", e."id_project", e2."id_project") AS
id_project
        FROM "Stage" s
        LEFT JOIN "Episode" e ON e."id_episode" = s."id_episode"
        LEFT JOIN "Frame" f ON f."id_frame" = s."id_frame"
        LEFT JOIN "Episode" e2 ON e2."id_episode" = f."id_episode"
    )
    SELECT
        p."project_name",
        AVG(
            CASE
                WHEN st."stage_type_id" IS NOT NULL
                THEN EXTRACT(EPOCH FROM (sp."actual_end_datetime" -
sp."start_datetime")) / 3600.0
                ELSE NULL
            END
        ) AS avg_hours
    FROM "Project" p
    LEFT JOIN stage_project sp ON sp.id_project = p."id_project"
    AND sp."start_datetime" IS NOT NULL
    AND sp."actual_end_datetime" IS NOT NULL
    AND sp."actual_end_datetime" >= %(start)s
    AND sp."actual_end_datetime" < %(end)s
    LEFT JOIN "StageType" st ON st."stage_type_id" = sp."stage_type_id"
    AND st."stage_type_name" ILIKE '%%композит%%'
    GROUP BY p."project_name"
    ORDER BY p."project_name";
''', {"start": month_g, "end": month_g_next})

# e) Episodes waiting QC on compositing
qc_rows = fetch_all('''
    WITH stage_ctx AS (
        SELECT
            s."id_stage",
            s."id_episode",
            s."id_frame"
        FROM "Stage" s
    ),
    ep_ctx AS (
        SELECT
            sc."id_stage",
            COALESCE(sc."id_episode", f."id_episode") AS id_episode
        FROM stage_ctx sc
        LEFT JOIN "Frame" f ON f."id_frame" = sc."id_frame"
    )
    SELECT DISTINCT
        p."project_name",
        e."episode_name",
        f."frame_number_in_episode",

```



```

        ss."stage_status_name"
FROM "Stage" s
JOIN "StageType" st ON st."stage_type_id" = s."stage_type_id"
JOIN ep_ctx ec ON ec."id_stage" = s."id_stage"
JOIN "Episode" e ON e."id_episode" = ec.id_episode
JOIN "Project" p ON p."id_project" = e."id_project"
LEFT JOIN "Frame" f ON f."id_frame" = s."id_frame"
LEFT JOIN "StageStatus" ss ON ss."id_stage_status" = s."id_stage_status"
WHERE st."stage_type_name" ILIKE '%%композит%%'
      AND ss."stage_status_name" ILIKE '%%провер%%'
      AND (s."start_datetime" IS NULL OR s."start_datetime" < %(end)s)
ORDER BY p."project_name", e."episode_name", f."frame_number_in_episode";
'', {"end": month_e_next})

# f) Workload distribution
load_rows = fetch_all('''
SELECT
    e."surname" || ' ' || e."name" AS artist,
    COUNT(DISTINCT s."id_stage") AS active_stages
FROM "Stage" s
JOIN "Employees" e ON e."id_employee" = s."id_responsible_employee"
JOIN "StageStatus" ss ON ss."id_stage_status" = s."id_stage_status"
WHERE ss."stage_status_name" NOT IN ('Принят', 'Завершён')
GROUP BY artist
ORDER BY active_stages DESC, artist;
''')

# g) Forecast finish date (pace from all historical progress)
forecast_rows = fetch_all('''
WITH stage_project AS (
    SELECT
        s."id_stage",
        s."id_stage_status",
        COALESCE(s."id_project", e."id_project", e2."id_project") AS
id_project
FROM "Stage" s
LEFT JOIN "Episode" e ON e."id_episode" = s."id_episode"
LEFT JOIN "Frame" f ON f."id_frame" = s."id_frame"
LEFT JOIN "Episode" e2 ON e2."id_episode" = f."id_episode"
),
project_bounds AS (
    SELECT
        sp.id_project,
        MIN(dp."progress_date") AS start_date
FROM stage_project sp
JOIN "DailyProgress" dp ON dp."id_stage" = sp."id_stage"
GROUP BY sp.id_project
),
progress_today AS (
    SELECT
        p."id_project",
        COALESCE(ROUND(AVG(
            COALESCE(
                ld."progress_percent",
                CASE
                    WHEN ss."stage_status_name" ILIKE '%не начат%' THEN 0
                    WHEN ss."stage_status_name" ILIKE '%в работе%' THEN 50
                    WHEN ss."stage_status_name" ILIKE '%доработ%' THEN 70
                    WHEN ss."stage_status_name" ILIKE '%заверш%' OR
ss."stage_status_name" ILIKE '%принят%' THEN 100
                    WHEN ss."stage_status_name" ILIKE '%провер%' THEN 90
                    ELSE 0
                END
            )
        ), 2), 0) AS overall_percent
FROM "Project" p
LEFT JOIN stage_project sp ON sp.id_project = p."id_project"
LEFT JOIN LATERAL (
    SELECT dp."progress_percent"
FROM "DailyProgress" dp

```

```

        WHERE dp."id_stage" = sp."id_stage"
        AND dp."progress_date" <= CURRENT_DATE
        ORDER BY dp."progress_date" DESC, dp."id_progress" DESC
        LIMIT 1
    ) ld ON true
    LEFT JOIN "StageStatus" ss ON ss."id_stage_status" = sp."id_stage_status"
    GROUP BY p."id_project"
),
progress_start AS (
    SELECT
        pb.id_project,
        pb.start_date,
        COALESCE(ROUND(AVG(
            COALESCE(
                ld."progress_percent",
                CASE
                    WHEN ss."stage_status_name" ILIKE '%не начат%' THEN 0
                    WHEN ss."stage_status_name" ILIKE '%в работе%' THEN 50
                    WHEN ss."stage_status_name" ILIKE '%доработ%' THEN 70
                    WHEN ss."stage_status_name" ILIKE '%заверш%' OR
ss."stage_status_name" ILIKE '%принят%' THEN 100
                    WHEN ss."stage_status_name" ILIKE '%провер%' THEN 90
                    ELSE 0
                END
            ), 2), 0) AS overall_percent
    FROM project_bounds pb
    JOIN stage_project sp ON sp.id_project = pb.id_project
    LEFT JOIN LATERAL (
        SELECT dp."progress_percent"
        FROM "DailyProgress" dp
        WHERE dp."id_stage" = sp."id_stage"
        AND dp."progress_date" <= pb.start_date
        ORDER BY dp."progress_date" DESC, dp."id_progress" DESC
        LIMIT 1
    ) ld ON true
    LEFT JOIN "StageStatus" ss ON ss."id_stage_status" = sp."id_stage_status"
    GROUP BY pb.id_project, pb.start_date
)
SELECT
    p."project_name",
    COALESCE(pt.overall_percent, 0) AS overall_percent,
    CASE
        WHEN ps.start_date IS NULL THEN NULL
        ELSE GREATEST(0, (COALESCE(pt.overall_percent, 0) -
COALESCE(ps.overall_percent, 0)) / NULLIF((CURRENT_DATE - ps.start_date), 0))
    END AS daily_rate,
    CASE
        WHEN GREATEST(0, (COALESCE(pt.overall_percent, 0) -
COALESCE(ps.overall_percent, 0)) / NULLIF((CURRENT_DATE - ps.start_date), 0)) > 0
        THEN (CURRENT_DATE + CEIL((100 - COALESCE(pt.overall_percent, 0)) /
GREATEST(0, (COALESCE(pt.overall_percent, 0) - COALESCE(ps.overall_percent, 0)) /
NULLIF((CURRENT_DATE - ps.start_date), 0))) * INTERVAL '1 day')::date
        ELSE NULL
    END AS прогноз
FROM "Project" p
LEFT JOIN progress_today pt ON pt."id_project" = p."id_project"
LEFT JOIN progress_start ps ON ps.id_project = p."id_project"
ORDER BY p."project_name";
'''

# h) Defects and reworks by stage type (include all stage types, no period)
defect_rows = fetch_all(''
    SELECT
        st."stage_type_name",
        COUNT(*) FILTER (WHERE cr."check_result_name" ILIKE '%брак%') AS
defects,
        COUNT(*) FILTER (WHERE cr."check_result_name" ILIKE '%доработ%') AS
reworks,

```

```

COUNT(*) FILTER (WHERE cr."check_result_name" ILIKE '%%не принят%%') AS
rejected
FROM "StageType" st
LEFT JOIN "Stage" s ON s."stage_type_id" = st."stage_type_id"
LEFT JOIN "StageCheck" sc
    ON sc."id_stage" = s."id_stage"
LEFT JOIN "CheckResult" cr ON cr."id_check_result" = sc."id_check_result"
GROUP BY st."stage_type_name"
ORDER BY st."stage_type_name";
'''

return render_template(
    "reports.html",
    lag_days=lag_days,
    month_start=month_start,
    date_a=date_a,
    month_v_value=_month_value(month_v),
    month_g_value=_month_value(month_g),
    month_e_value=_month_value(month_e),
    progress_rows=progress_rows,
    lag_rows=lag_rows,
    frames_by_artist=frames_by_artist,
    avg_paint_time=avg_paint_time,
    qc_rows=qc_rows,
    load_rows=load_rows,
    forecast_rows=forecast_rows,
    defect_rows=defect_rows,
)

```

Г8) Файл role_rules.py

```

import pycpg2

from db import get_conn

ROLE_RULES = {
    "разработка идеи и сценария": {"сценарист"},
    "концепт-арт и дизайн": {"художник-постановщик"},
    "создание раскадровки": {"художник-постановщик"},
    "аниматик / превизуализация": {"художник-постановщик"},
    "озвучивание": {"актёр озвучивания"},
    "звук и музыка": {"звукорежиссёр"},
    "финальная приёмка эпизода": {"режиссёр"},
    "моделирование": {"3d-моделлер"},
    "анимация": {"аниматор"},
    "композитинг": {"компоузер (композитинг)"},
    "визуальные эффекты (vfx)": {"vfx-специалист"},
    "техническая подготовка": {"технический художник"},
    "рендеринг": {"специалист по рендерингу"},
}

def _normalize(value: str | None) -> str:
    if not value:
        return ""
    return " ".join(value.lower().split())

def normalize_value(value: str | None) -> str:
    return _normalize(value)

def allowed_specializations(stage_type_name: str | None) -> set[str] | None:
    if not stage_type_name:
        return None
    return ROLE_RULES.get(_normalize(stage_type_name))

```

```

def _fetch_stage_type_name(stage_id: int) -> str | None:
    with get_conn() as conn, conn.cursor() as cur:
        cur.execute('''
            SELECT st."stage_type_name"
            FROM "Stage" s
            JOIN "StageType" st ON st."stage_type_id" = s."stage_type_id"
            WHERE s."id_stage" = %(sid)s;
        ''', {"sid": stage_id})
        row = cur.fetchone()
        return row["stage_type_name"] if row else None

def _fetch_employee_specialization(employee_id: int) -> str | None:
    try:
        with get_conn() as conn, conn.cursor() as cur:
            cur.execute('''
                SELECT sp."specialization_name" AS specialization
                FROM "Employees" e
                LEFT JOIN "Spesialization" sp
                ON sp."id_spesialization" = e."id_spesialization"
                WHERE e."id_employee" = %(eid)s;
            ''', {"eid": employee_id})
            row = cur.fetchone()
            return row["specialization"] if row else None
    except psycopg2.Error:
        return None

def validate_stage_responsible(stage_id: int, employee_id: int) -> tuple[bool, str | None]:
    stage_type_name = _fetch_stage_type_name(stage_id)
    if not stage_type_name:
        return False, "Этап не найден."

    allowed = ROLE_RULES.get(_normalize(stage_type_name))
    if not allowed:
        return False, f"Для этапа «{stage_type_name}» нет правила назначения."

    specialization = _fetch_employee_specialization(employee_id)
    if not specialization:
        return False, "У сотрудника не указана специализация."

    normalized_spec = _normalize(specialization)
    if normalized_spec not in allowed:
        needed = ", ".join(sorted(allowed))
        return (
            False,
            f"Нельзя назначить «{specialization}» на этап «{stage_type_name}». Требуется: {needed}.",
        )

    return True, None

```